

Associate Architect 양성과정 · Architecture 설계

# Day 3 · Architectural Tactics (1)

Tactics 개요 · Availability / Deployability / Energy Efficiency / Integrability / Modifiability / Performance / Safety / Security

- 3.1 Architectural Tactics · 3.2 Availability Tactics
- 3.3 Deployability · 3.4 Energy Efficiency · 3.5 Integrability · 3.6 Modifiability Tactics
- 3.7 Performance · 3.8 Safety · 3.9 Security Tactics

# 3.1

## Architectural Tactics

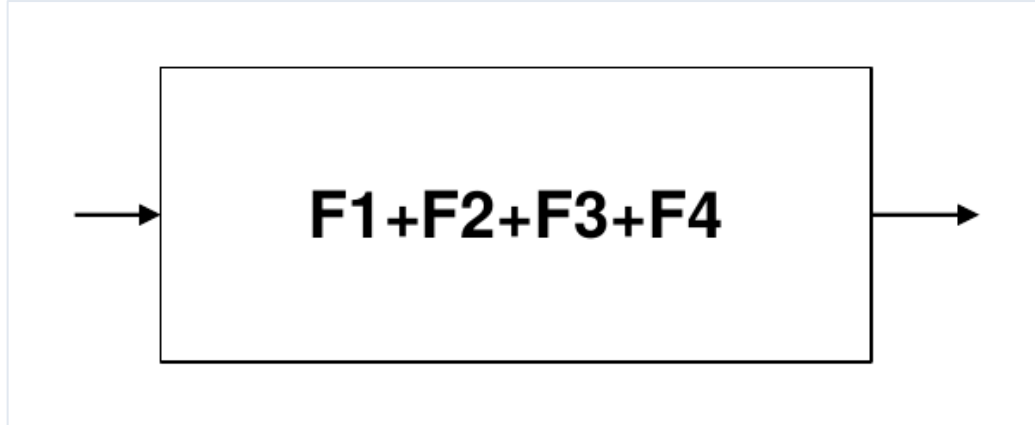
품질 속성을 달성하기 위한 Architect 의 설계 결정 단위, Tactic 을 이해한다

- 기능(Functionality)과 Architecture 의 관계
- Quality Attribute 와 Architecture
- Quality Attribute Scenario
- General vs. Concrete Scenario
- Tactic 의 정의와 계층 구조
- Quality Design Decision 의 범주

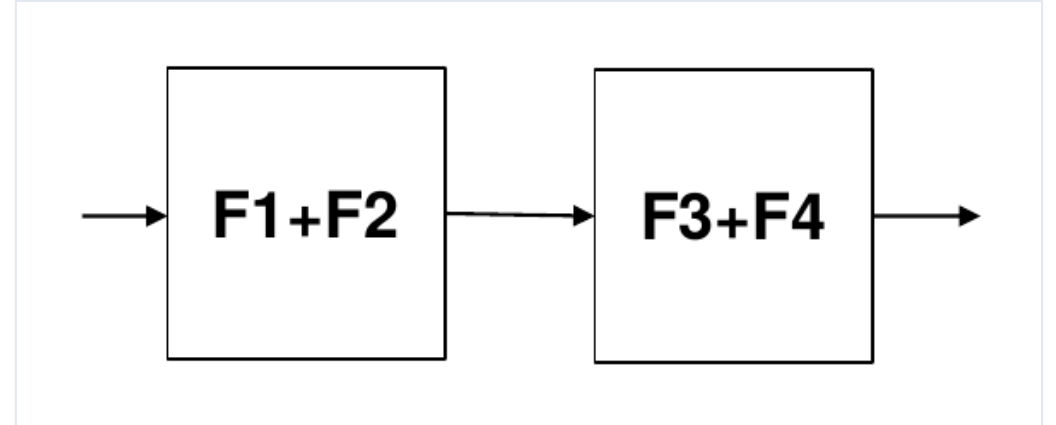
## 기능(Functionality)과 Architecture

### ■ 기능은 구조와 대체로 독립적이다

- 같은 기능(F1~F4)을 만족하는 구조는 여러 가지가 있을 수 있다
  - 하나의 component 에  $F1+F2+F3+F4$  를 모두 넣을 수도 있고
  - $F1+F2$  와  $F3+F4$  로 나누어 두 component 에 배치할 수도 있다
- 즉 기능 요구만으로는 어떤 구조를 택할지 결정할 수 없다



기능 전체를 하나의 구조에 배치



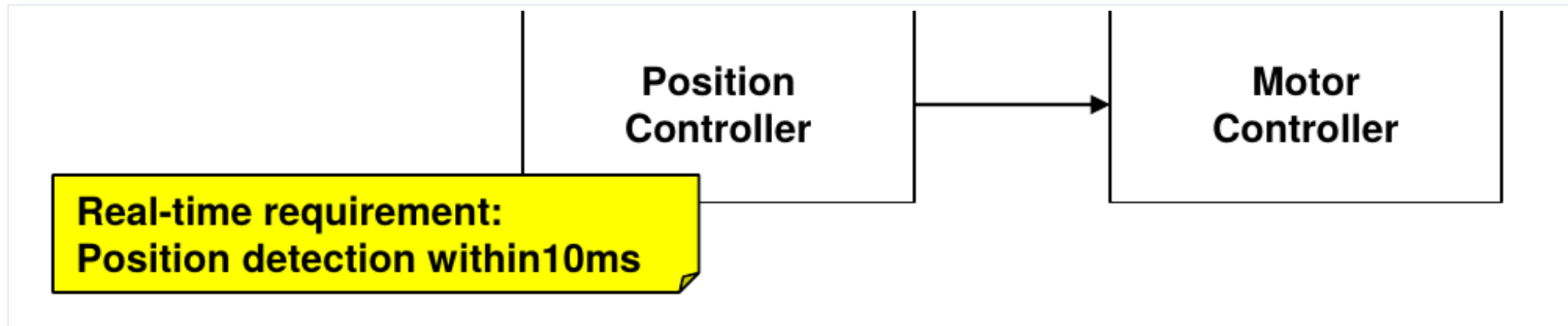
기능을 두 구조로 나누어 배치

**핵심** 구조를 결정짓는 것은 기능이 아니라 Quality Attribute(품질 속성)이다

## Quality Attribute 와 Architecture (1)

### ■ Quality Attribute 가 기능의 component 배치를 좌우한다

- Maintainability, Understandability 가 중요하면
  - 의미적 응집도(semantic coherence)를 기준으로 기능을 배치한다
  - 예: Layered Architecture
- Performance 가 중요하면
  - 시간 제약이 큰(time critical) 기능을 따로 분리한다
  - 예: 10ms 이내 위치 검출이 필요한 Position Controller 를 Motor Controller 와 분리



실시간 요구를 가진 기능을 별도 component 로 분리한 예

## Quality Attribute 와 Architecture (2)

### ■ Quality Attribute 는 Architecture 설계 초기부터 고려해야 한다

- 여러 품질 속성의 달성 여부는 초기 Architecture 결정에 좌우된다
- 기능을 먼저 구현한 뒤 나중에 품질 속성을 '덧붙이는' 것은 일반적으로 불가능하다

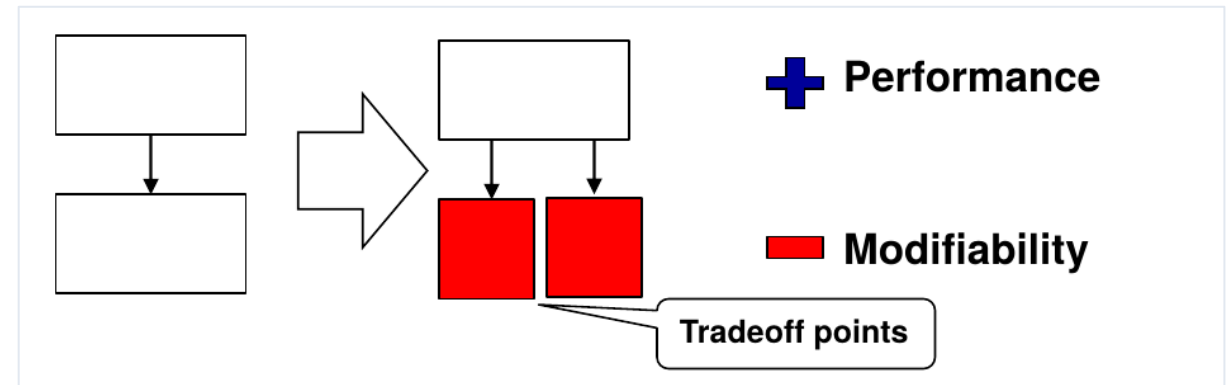
### ■ 품질 속성은 Architecture 와 구현 모두에 걸쳐 있다

- 어떤 품질 속성도 Architecture 설계에만, 또는 구현에만 전적으로 의존하지 않는다
- 예: Performance 요구는 두 가지 방법으로 달성할 수 있다
  - 기능의 배치(allocation of functionality) — Architecture 수준의 결정
  - 알고리즘 선택(choice of algorithms) — Architecture 와 무관한 결정

## Quality Attribute 와 Architecture (3) — Tradeoff

### ■ Architecture 선택은 어떤 품질 속성은 높이고 다른 것은 낮춘다

- 하나의 품질 속성을 개선하는 구조 변경이 다른 품질 속성을 높일 수도, 떨어뜨릴 수도 있다
  - 예: 하나의 component 를 둘로 나누면 Performance 는 좋아지지만 Modifiability 는 나빠질 수 있다
  - 이렇게 서로 충돌하는 지점을 Tradeoff point 라 한다
- 상세 설계와 구현 전에 이러한 품질 속성 간 tradeoff 를 균형 있게 조정하는 것이 Architecture 의 핵심 역할이다

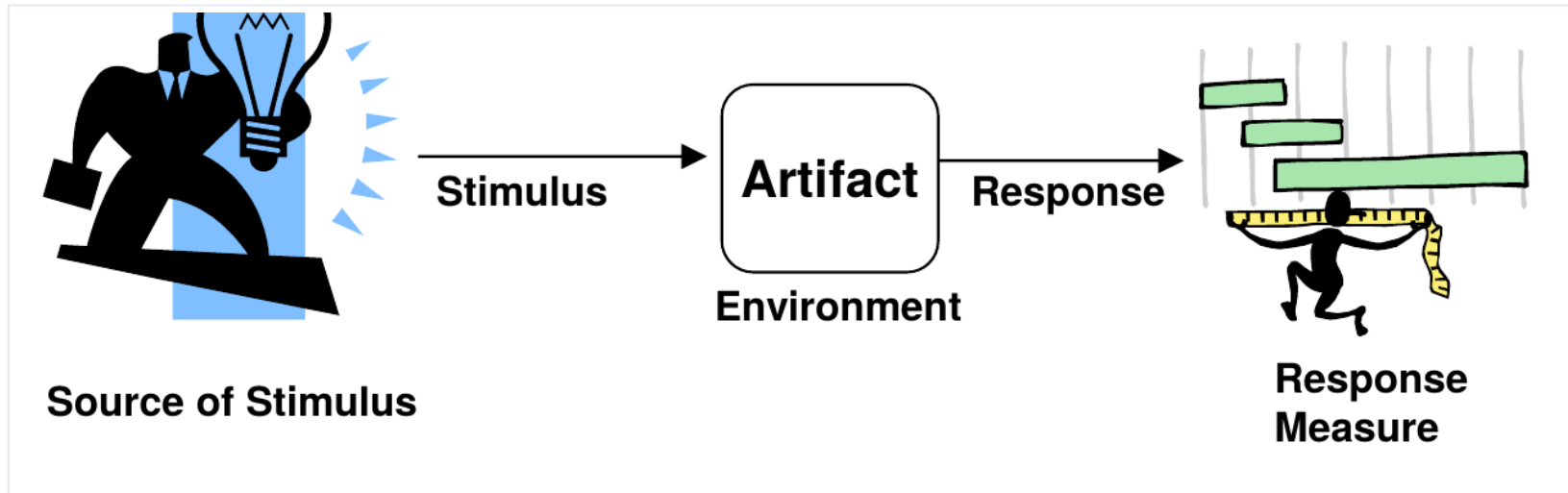


구조 변경에 따른 Performance(+)와 Modifiability(-)의 tradeoff

## 품질 요구는 어떻게 명세하는가?

### ■ "시스템은 수정 가능해야 한다" 는 말은 의미가 없다

- 막연한 표현으로는 요구를 검증할 수도, 설계에 반영할 수도 없다
- Quality Attribute Scenario(품질 속성 시나리오)는 품질 속성별 요구를 식별하고 명세하는 효과적인 방법이다
- 시나리오는 6개 부분으로 구성된다
  - Source of Stimulus, Stimulus, Artifact, Environment, Response, Response Measure



Quality Attribute Scenario 의 6개 구성 요소

# Quality Attribute Scenario 의 구성 요소

| 구성 요소              | 의미                                                                                                             |
|--------------------|----------------------------------------------------------------------------------------------------------------|
| Source of Stimulus | Stimulus(자극)를 만들어 낸 주체(사람, 시스템, 장치 등)                                                                          |
| Stimulus           | 시스템에 도착하는 자극. 품질 속성에 따라 달라진다 — Performance: 서비스 요청 / Modifiability: 변경 요청 / Availability: 결함 상황 / Security: 공격 |
| Artifact           | 자극이 도착하는 시스템 전체 또는 시스템의 일부                                                                                     |
| Environment        | 시나리오가 발생하는 상황(정상 운영, 과부하, 시작 중 등)                                                                              |
| Response           | 자극이 도착한 결과로 일어나는 활동                                                                                            |
| Response Measure   | Response 가 일어났을 때 측정 가능한 기준. 시나리오를 테스트할 수 있도록 어떤 형태로든 측정 가능해야 한다                                               |

**핵심** Stimulus 는 품질 속성마다 다르지만, 시나리오의 틀은 모든 품질 속성에 공통이다



## General Scenario 와 Concrete Scenario

### ■ General Quality Attribute Scenario

- 특정 시스템에 독립적(system independent)
- 어떤 시스템에도 잠재적으로 적용할 수 있는 일반적인 틀
- 각 구성 요소에 들어갈 수 있는 값의 종류를 열거한다

### ■ Concrete Quality Attribute Scenario

- 지금 고려하는 특정 시스템에 한정된 시나리오
- General Scenario 의 속성 특성(attribute characterization)에서 구체적인 값을 골라 인스턴스화한다
- 실제 요구 명세와 테스트의 기준이 된다

## General vs. Concrete Scenario — Availability 예

- "정상 운영 중인 process 가 예상치 못한 외부 message 를 받는다. process 는 운영자에게 수신 사실을 알리고 다운타임 없이 계속 동작한다"

| 구성 요소                   | General Scenario (선택지)                                             | Concrete Scenario (예)                         |
|-------------------------|--------------------------------------------------------------------|-----------------------------------------------|
| <b>Source</b>           | Internal, External                                                 | External (외부)                                 |
| <b>Stimulus</b>         | (Fault) Omission, Crash, Timing, Response                          | 예상치 못한 외부 message 수신                          |
| <b>Artifact</b>         | Process, Storage, Processor, Communication                         | Process                                       |
| <b>Environment</b>      | Normal, Degraded Operation                                         | Normal operation (정상 운영 중)                    |
| <b>Response</b>         | Record, Notify, Disable, Continue(Normal/Degraded), Be Unavailable | 운영자에게 message 수신을 알리고(Notify) 계속 동작(Continue) |
| <b>Response Measure</b> | Repair Time, Availability, Available/Degraded Time Interval        | 다운타임 없음(no downtime)                          |

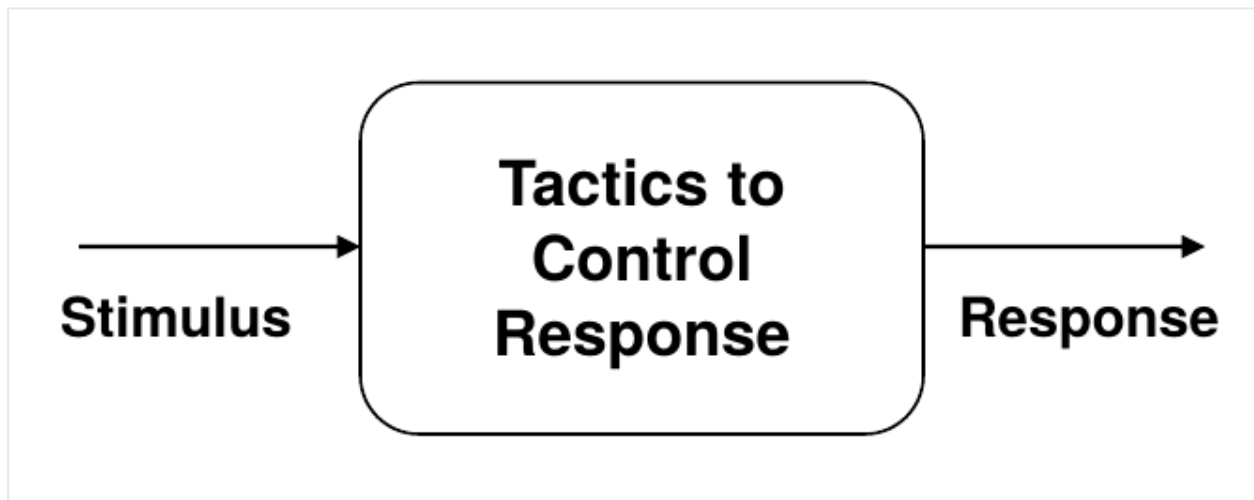
# Tactic 이란?

## ■ (Architectural) Tactic 의 정의

- 품질 속성 Response 의 달성에 영향을 주는 설계 결정(design decision)
- 하나의 품질 속성 Response 에만 집중하며, 다른 품질 속성과의 tradeoff 는 고려하지 않는다
  - Stimulus 가 들어왔을 때 원하는 Response 가 나오도록 제어하는 수단

## ■ Tactic 은 Architect 의 선택지(decision option)이다

- 각 tactic 은 Architect 가 고를 수 있는 하나의 설계 옵션이다
- 새로운 발명이 아니라, Architect 들이 오랫동안 실무에서 써 온 방법들을 정리한 것이다

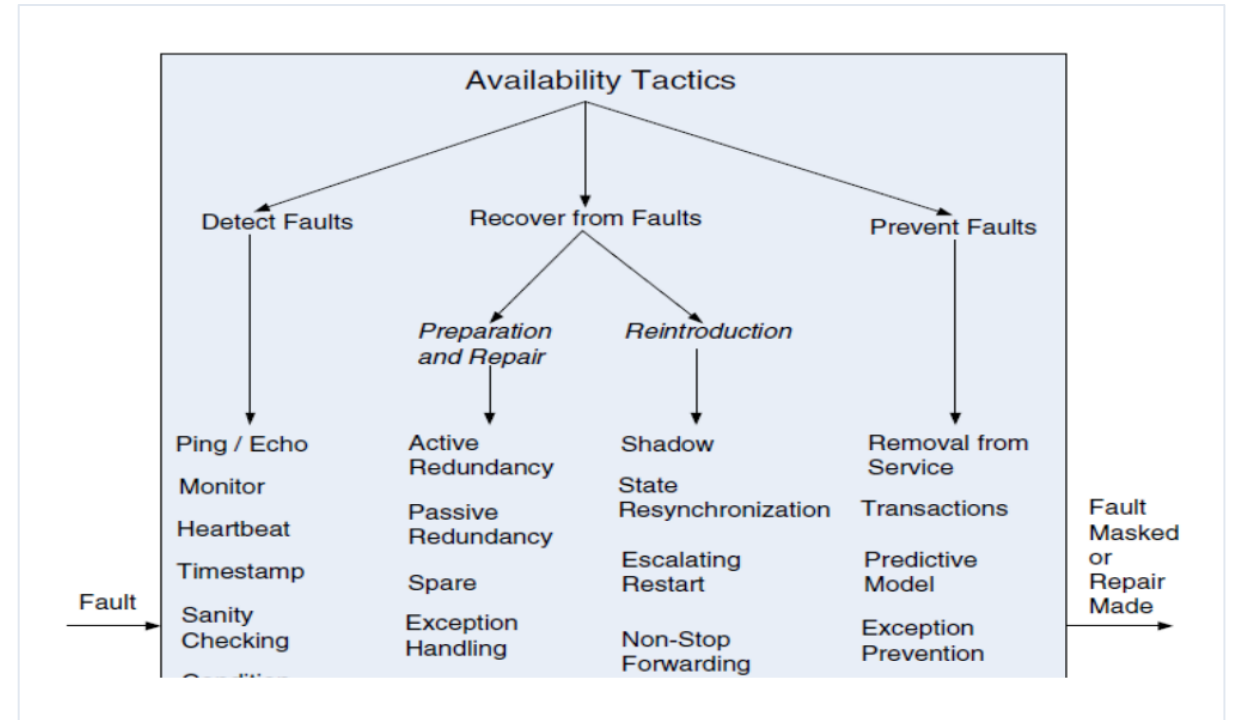


*Tactic: Stimulus 에 대한 Response 를 제어하는 설계 결정*

## Tactic 의 계층 구조

### ■ Tactic 은 다른 tactic 을 세분화(refine)할 수 있다

- 품질 속성마다 tactic 들을 특수화(specialization) 또는 분해(decomposition) 관계에 따라 계층으로 정리할 수 있다
- 예: Availability Tactics 는 Detect Faults / Recover from Faults / Prevent Faults 로 나뉘고, Recover 는 다시 Preparation and Repair / Reintroduction 으 로 나뉜다
- 맨 아래에 구체적인 tactic(Ping/Echo, Heartbeat, Active Redundancy 등)이 온다



Availability Tactic 계층도 (Fault → Tactics → Fault masked / Repair made)

## Quality Design Decision 의 7가지 범주

### ■ Allocation of responsibilities

- 책임을 어떤 요소에 할당할지

### ■ Coordination model

- 요소들이 어떻게 협력·통신할지

### ■ Data model

- 데이터를 어떻게 구조화하고 관리할지

### ■ Management of resources

- 자원을 어떻게 관리·배분할지

### ■ Mapping among architectural elements

- module, component, 하드웨어 사이의 대응

### ■ Binding time decisions / Choice of technology

- 결정을 언제 확정할지, 어떤 기술을 쓸지

**핵심** 이 범주들은 Architecture 설계 중 놓쳐서는 안 될 중요한 결정을 빠짐없이 찾아내도록 돕는다

# 3.2

## Availability Tactics

결함(fault)이 장애(failure)로 이어지지 않도록 감지·복구·예방하는 태틱

- Availability 의 정의와 Reliability 와의 차이
- ISO/IEC 25010 의 Reliability
- Availability General / Concrete Scenario
- Fault Detection Tactics
- Fault Recovery Tactics (Preparation and Repair / Reintroduction)
- Fault Prevention Tactics

# Availability(가용성)란?

## ■ 정의

- Availability: 필요할 때 소프트웨어가 자기 임무를 수행할 준비가 되어 있는 성질
  - Availability = Reliability(신뢰성) + 복구(recovery)의 개념
- 결함(fault)을 감추거나(mask) 고쳐서(repair), 지정된 기간 동안의 누적 서비스 중단 시간이 요구된 값을 넘지 않도록 하는 시스템의 능력

## ■ 핵심

- 근본적으로 Availability 는 결함을 완화하여 서비스 중단 시간(outage time)을 최소화하는 것이다

**핵심** 출처: L. Bass 외, Software Architecture in Practice 3rd Ed. (SEI Series), Addison Wesley

## Availability vs. Reliability

### ■ Availability (가용성)

- 장비가 동작 가능한 상태에 있는 시간의 비율
- $\text{Uptime} / \text{Total time (Uptime + Downtime)}$

### ■ Reliability (신뢰성)

- 장비가 의도한 기능을 얼마나 오래 수행하는가의 척도
- $\text{MTBF (Mean Time Between Failure)} = \text{총 서비스 시간} / \text{고장 횟수}$

**핵심** 신뢰성이 높은 장비는 보통 가용성도 높지만, 가용성이 높다고 반드시 신뢰성이 높은 것은 아니다 (빨리 복구되면 가용성은 높을 수 있다)



## ISO/IEC 25010 — Reliability 의 하위 특성

| 특성                     | 정의                                                       |
|------------------------|----------------------------------------------------------|
| <b>Reliability</b>     | 시스템·제품·component 가 지정된 조건에서 지정된 기간 동안 명시된 기능을 수행하는 정도    |
| <b>Maturity</b>        | 정상 운영 상태에서 신뢰성 요구를 충족하는 정도                               |
| <b>Availability</b>    | 필요할 때 시스템이 동작 가능하고 접근 가능한 정도                             |
| <b>Fault tolerance</b> | 하드웨어·소프트웨어 결함이 있어도 의도한 대로 동작하는 정도                        |
| <b>Recoverability</b>  | 중단이나 장애가 발생했을 때 영향받은 데이터를 복구하고 원하는 시스템 상태를 다시 세울 수 있는 정도 |

**핵심** ISO/IEC 25010 에서는 Availability 를 Reliability 의 하위 특성으로 본다

# Availability General Scenario (1)

| 구성 요소              | 가능한 값                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Source of Stimulus | Internal 또는 External: 사람, 하드웨어, 소프트웨어, 물리 인프라, 물리적 환경                                                                                  |
| Stimulus           | 다음 종류의 fault — Omission: component 가 입력에 응답하지 않음 / Crash: omission 이 반복됨 / Timing: 응답은 하지만 너무 이르거나 늦음 / Incorrect Response: 잘못된 값으로 응답 |
| Artifact           | Processor, 통신 채널, process, storage                                                                                                     |
| Environment        | fault 발생 시점의 시스템 상태 — 정상 운영, startup, shutdown, 수리 모드, 성능 저하(degraded) 운영, 과부하 운영                                                      |
| Response           | 장애 기록(logging), 특정 사용자·시스템에 통보, degraded mode 로 전환, 외부 시스템 종료, 수리 중 이용 불가 상태로 전환                                                       |
| Response Measure   | 가용성 비율(%), fault 감지 시간, fault 수리 시간, 시스템이 반드시 가용해야 하는 시간 구간, degraded mode 로 있어도 되는 시간 구간                                              |

## Availability General Scenario (2) — Stimulus 와 Response

### ■ Stimulus: fault 의 4가지 종류

- Omission: component 가 입력에 응답하지 못한다
- Crash: component 가 omission fault 를 반복해서 일으킨다
- Timing: 응답은 하지만 시점이 너무 이르거나 늦다
- Incorrect Response: 잘못된 값으로 응답한다

### ■ Response Measure 의 예

- 가용성 비율 (예: 99.99%)
- fault 를 감지하는 데 걸리는 시간 / 수리하는 데 걸리는 시간
- 시스템이 가용해야 하는 시간 구간, degraded mode 가 허용되는 시간 구간

**핵심** Availability 시나리오의 Stimulus 는 항상 '결함(fault)'이다

# Availability Concrete Scenario 예

| 구성 요소            | 값                                                      |
|------------------|--------------------------------------------------------|
| Source           | Heartbeat Monitor                                      |
| Stimulus         | 서버가 응답하지 않음 (Server Unresponsive)                      |
| Artifact         | Process                                                |
| Environment      | 정상 운영 중 (Normal Operation)                             |
| Response         | 운영자에게 알리고 계속 동작 (Inform Operator, Continue to Operate) |
| Response Measure | 다운타임 없음 (No Downtime)                                  |

**핵심** "Heartbeat monitor 가 정상 운영 중 서버가 응답하지 않음을 감지한다. 시스템은 운영자에게 알리고 다운타임 없이 계속 동작한다"

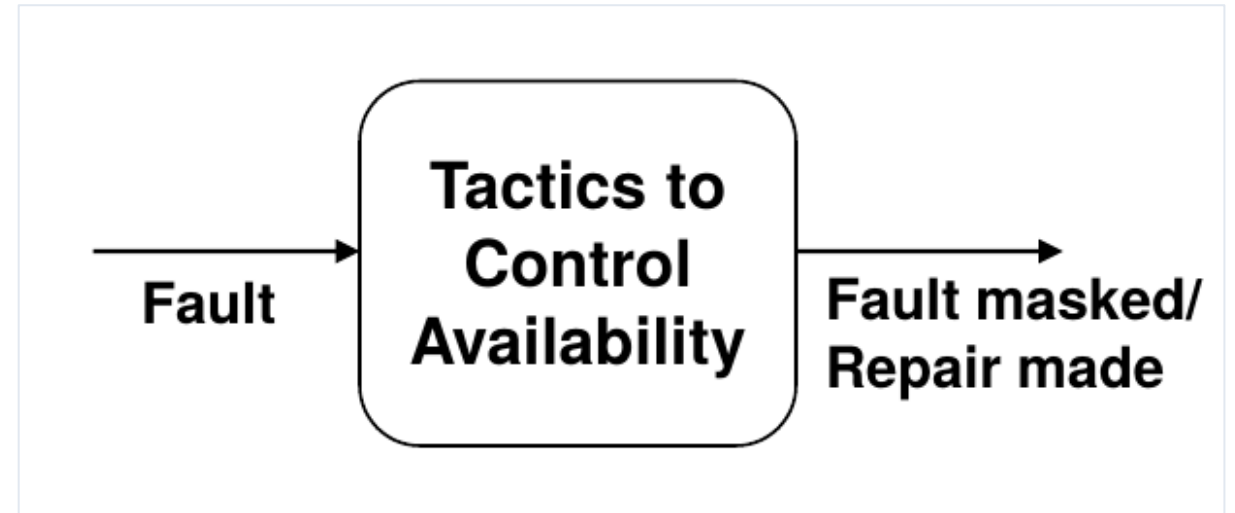
## Availability Tactics 의 목표와 접근 방법

### ■ Availability tactic 의 목표

- fault 가 failure(장애)로 이어지지 않게 한다
- 적어도 fault 의 영향 범위를 제한한다
- 수리(repair)가 가능하게 한다

### ■ 가용성을 유지하는 세 가지 접근

- 어떤 형태의 중복(redundancy)
- 장애를 감지하기 위한 어떤 형태의 모니터링 (monitoring)
- 장애가 감지되었을 때의 어떤 형태의 복구 (recovery)

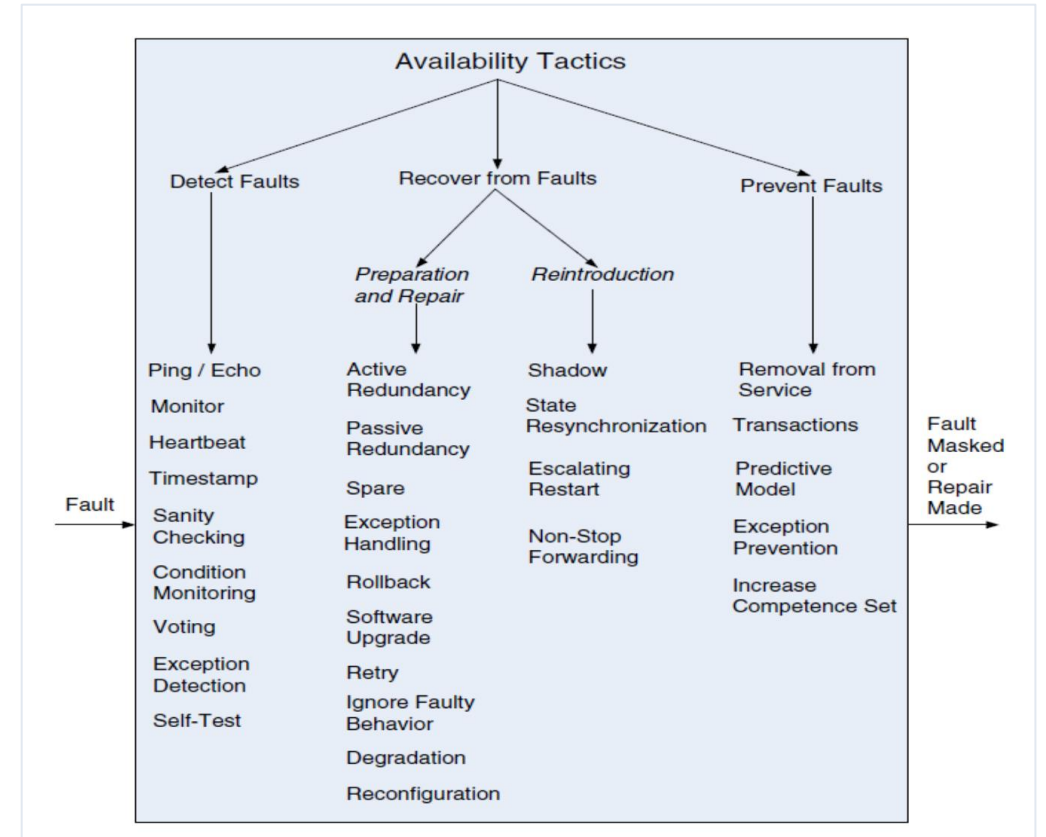


*Fault 가 들어오면 tactic 이 이를 감추거나 수리한다*

## Availability Tactics 계층도

### ■ 세 가지 큰 갈래

- Detect Faults: 결함 감지
  - Preparation and Repair (준비와 수리)
  - Reintroduction (재투입)
- Recover from Faults: 결함 복구
- Prevent Faults: 결함 예방



Availability Tactics 전체 계층도

## Fault Detection — Ping/Echo, Heartbeat

### ■ Ping/echo

- 한 component 가 감시 대상 component 에 ping 을 보내고, 정해진 시간 안에 echo 가 돌아오기를 기대한다
- ping 은 보통 system monitor 가 보낸다

### ■ Heartbeat

- component 가 주기적으로 heartbeat message 를 내보내고 system monitor 가 이를 듣는다
- heartbeat 를 다른 제어 message 에 실어 보내면 message overhead 를 줄일 수 있다

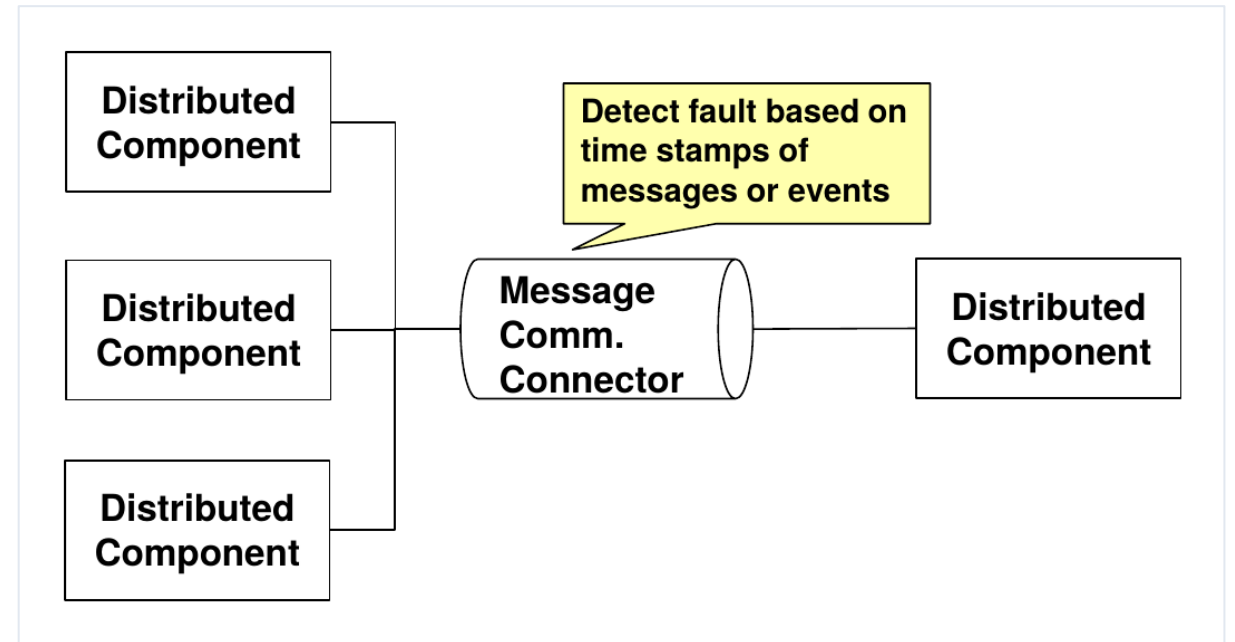


*Ping/echo(양방향)와 Heartbeat(단방향)의 차이*

## Fault Detection — Time Stamp

### ■ Time stamp

- event 에 time stamp 를 붙여 event 순서가 잘못된 경우를 감지한다
- 주로 분산 message-passing 시스템에서 사용한다
- message 나 event 의 time stamp 를 근거로 결함을 판단한다



분산 component 사이의 message time stamp 로 결함 감지



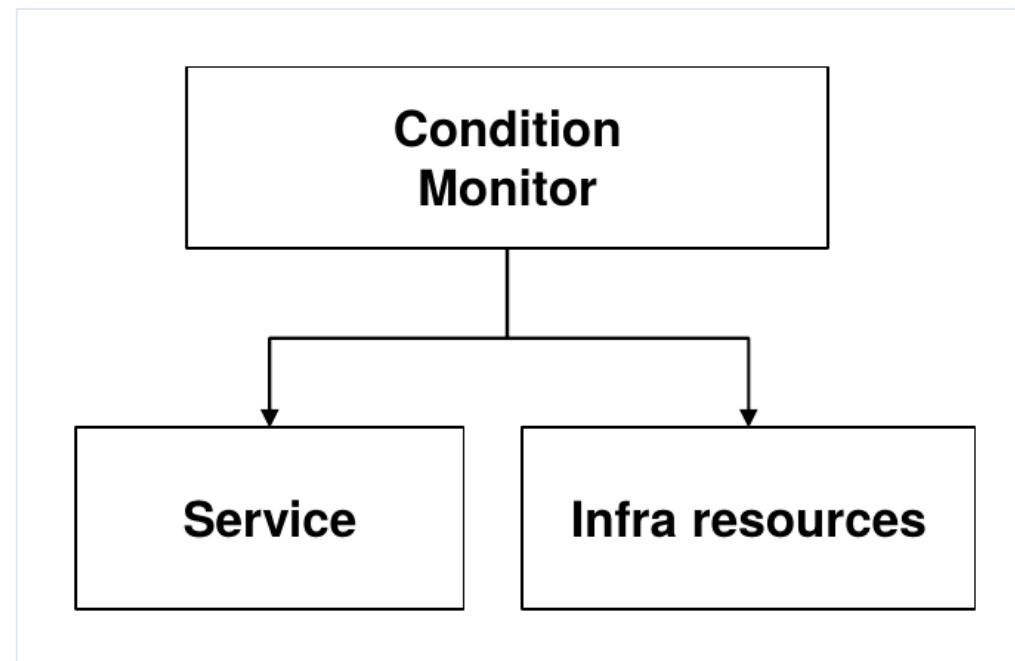
## Fault Detection — Condition Monitoring

### ■ Condition monitoring

- 서비스나 인프라(runtime/OS, network, storage)의 상태를 관찰하여 장애가 일어나기 전에 문제를 감지해 downtime 을 최소화하는 방법

### ■ 기법

- Real-time monitoring (실시간 모니터링)
- Periodic monitoring (주기적 모니터링)
- Remote monitoring (원격 모니터링)
- Predictive maintenance (예측 정비)



*Condition Monitor 가 Service 와 Infra resource 를 감시*

## Fault Detection — Exception Detection

### ■ Exception detection

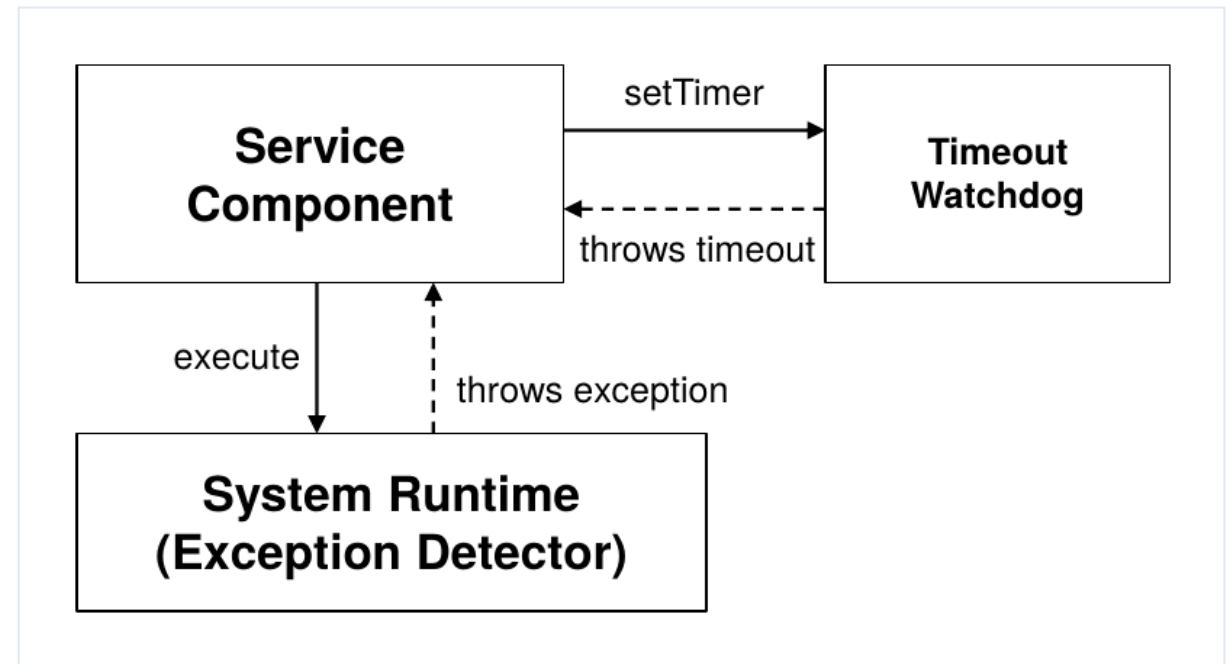
- 정상적인 실행 흐름을 바꾸는 시스템 상태를 감지하는 것

### ■ System exception

- 예: 0 으로 나누기, bus/address fault 등

### ■ Timeout exception

- 다른 component 의 응답을 기다리는 component 가 대기 시간이 임계값을 넘으면 exception 을 발생시킨다
  - 그림에서 Timeout Watchdog 가 timer 를 걸고 시간 초과 시 timeout 을 던진다



System Runtime(Exception Detector)과 Timeout Watchdog

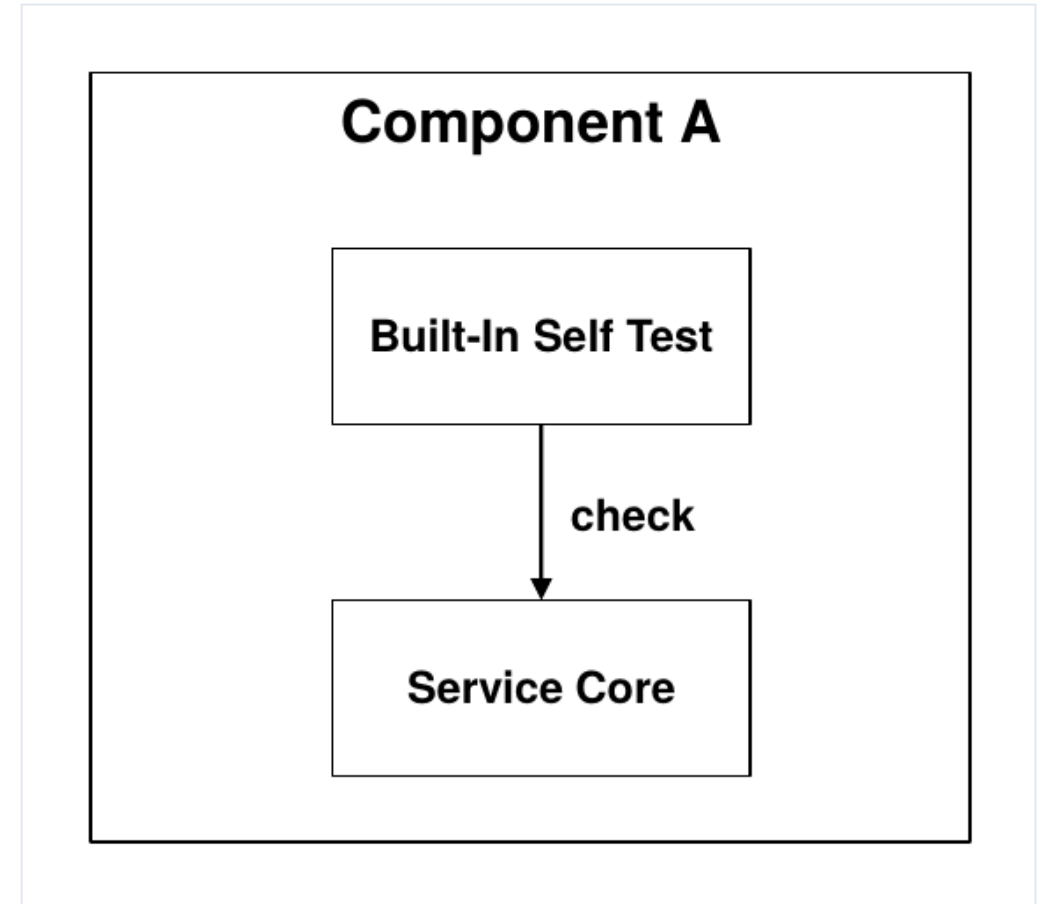
## Fault Detection — Self-test

### ■ Self-test

- component(대개는 subsystem 전체)가 스스로 올바르게 동작하는지 검사하는 절차를 실행한다
- Built-In Self Test 가 Service Core 를 check 하는 구조

### ■ 종류

- Startup self test: 시작 시 검사
- Periodic self test: 주기적 검사
- Continuous self test: 지속적 검사

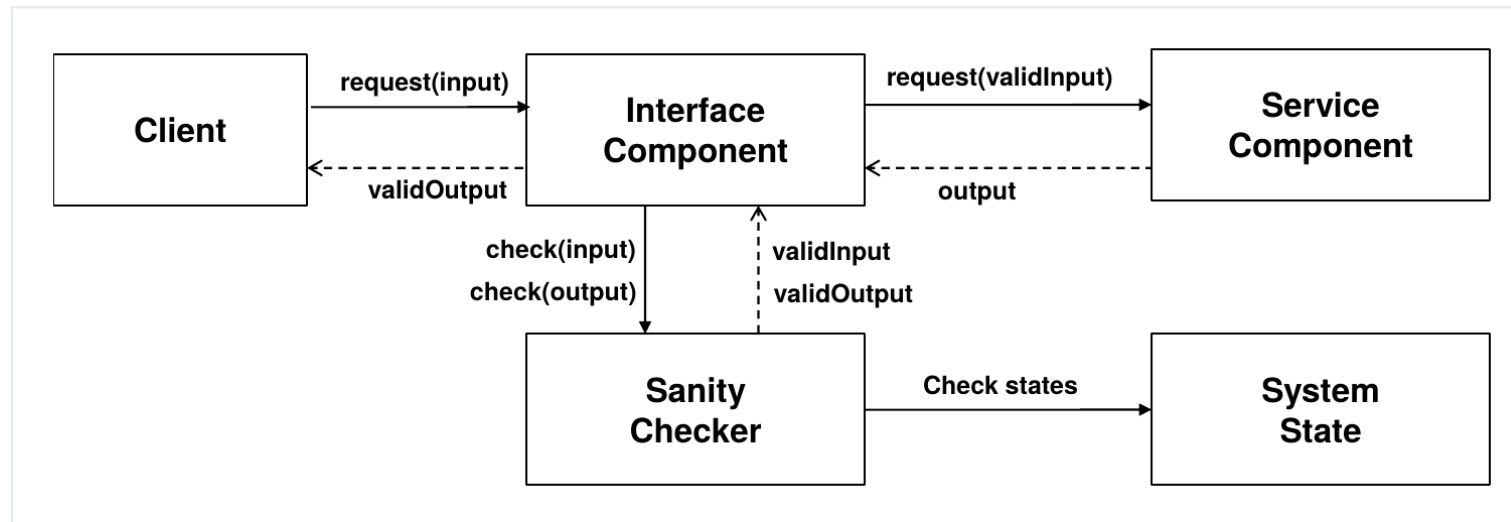


*Component 내부의 Built-In Self Test*

## Fault Detection — Sanity Checking

### ■ Sanity checking

- component 의 특정 작업이나 결과가 유효한지 검사하는 방법
- 내부 설계 지식, 현재 시스템 상태, 데이터 속성을 근거로 runtime 에 검사하여 장애를 일찍 감지한다
- 그림: Interface Component 가 Sanity Checker 로 input/output 을 검사한 뒤 유효한 값만 Service Component 로 전달한다



Sanity Checker 를 통한 입력·출력 유효성 검사

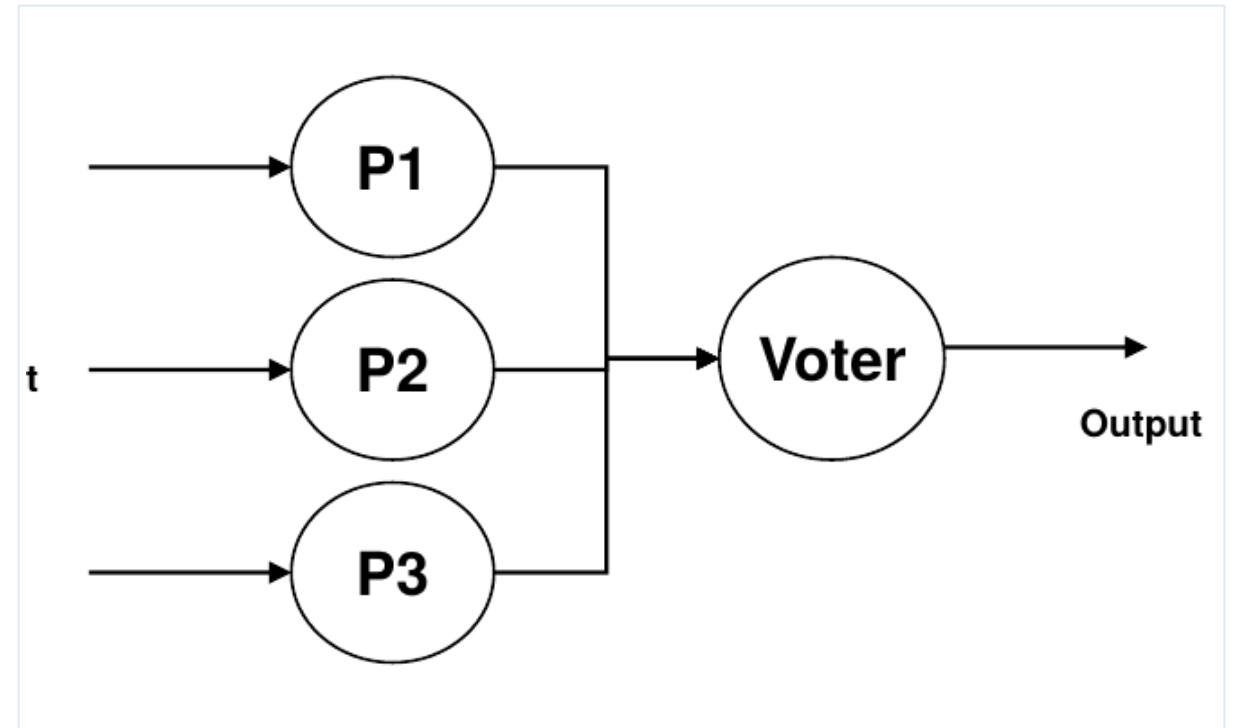
## Fault Detection — Voting

### ■ Voting

- 중복된 processor 위에서 도는 process 들이 같은 입력을 받아 단순한 출력값을 계산해 voter 에 보낸다
- voter 가 한 processor 의 동작이 다른 것들과 다르다고 판단하면 그 결과를 버린다
- voter 의 입력이 '동일/이탈'로 쉽게 분류될 수 있을 때 사용한다

### ■ 중복의 다양성(Diversity of Redundancy)

- 같은 소프트웨어를 다른 processor 에서 실행
- 다른 소프트웨어를 다른 processor 에서 실행



*P1, P2, P3 의 결과를 Voter 가 비교*

## Fault Detection Tactics 요약

### ■ 살아있음(aliveness) 감지

- Ping/echo
- Heartbeat

### ■ 오류로 이어질 수 있는 상황 감지

- Time stamp
- Condition monitoring
- Exception detection
- Self-test

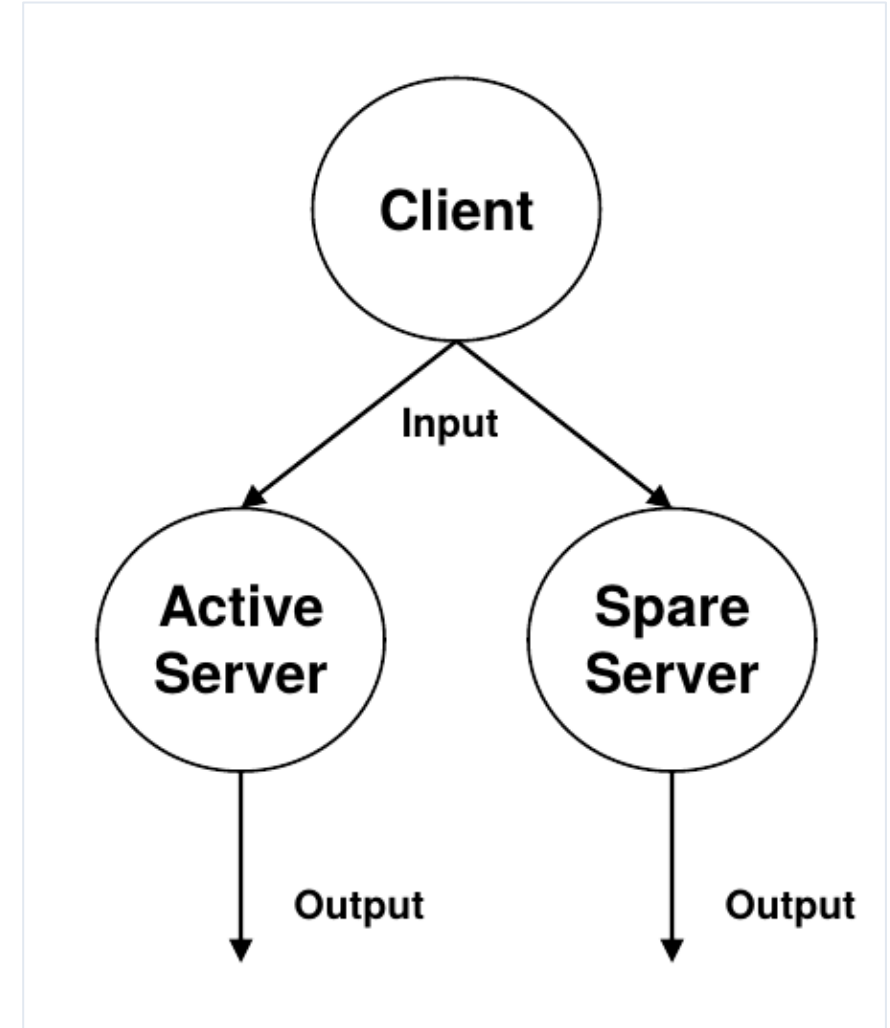
### ■ 시스템의 유효성 검사

- Sanity checking
- Voting

## Fault Recovery — Active Redundancy (Hot Spare)

### ■ Active Redundancy (hot spare)

- 모든 중복 component(active 와 spare)가 입력에 병렬로 응답한다
  - 출력은 그중 하나의 component 것만 사용한다
- spare 가 항상 같은 상태를 유지하므로, 장애 발생 시 수 밀리초 안에 역할을 넘겨받을 수 있다

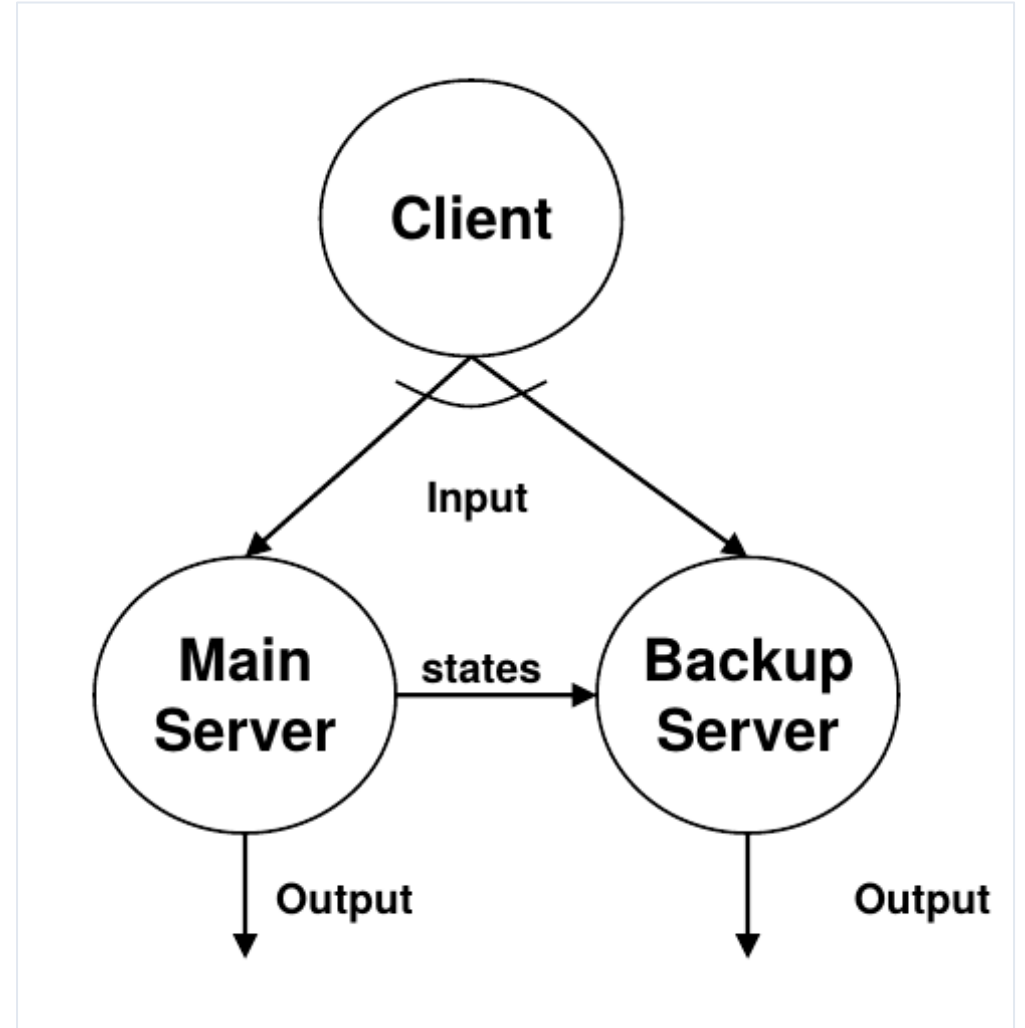


*Active Server 와 Spare Server 가 같은 입력을 동시에 처리*

## Fault Recovery — Passive Redundancy (Warm Spare)

### ■ Passive Redundancy (warm spare)

- 하나의 active component 만 event 에 응답하고, 중복 spare 에게 상태 변경(state update)을 주기적으로 알려 준다
- hot spare(가용성 높지만 비용 큼)와 cold spare(가용성 낮지만 훨씬 저렴) 사이의 균형 잡힌 해법



*Main Server 가 Backup Server 에 states 를 주기적으로 전달*



## Fault Recovery — Spare, Exception Handling, Rollback

### ■ Spare (cold spare)

- protection group 의 중복 spare 가 fail-over 가 일어나기 전까지는 서비스에서 빠져 있는(out of service) 구성
- 장애가 나면 spare 를 적절한 소프트웨어 구성으로 재부팅하고 상태를 초기화해야 한다
- 복구 성능이 나쁘기 때문에, 높은 신뢰성(MTBF) 요구만 있는 시스템에 더 적합하다
  - MTBF = Mean Time Between Failure (평균 고장 간격)

### ■ (Preparing) Exception handling

- 단순한 함수 return 값부터, 결함을 감추기 위한 정보를 담은 exception class 사용까지 범위가 넓다
- 보통 exception 의 원인을 고친 뒤 연산을 재시도(retry)하는 방식으로 결함을 감춘다

### ■ Rollback

- 장애가 감지되면 이전에 알려진 정상 상태(checkpoint)로 되돌린다
- checkpoint 는 고정된 위치에 저장하고 일정 간격으로 갱신하거나, 편리하거나 의미 있는 시점에 갱신한다

## Fault Recovery — Software Upgrade

### ■ Software upgrade

- 서비스에 영향을 주지 않으면서(in-service) 실행 코드 이미지를 업그레이드한다

### ■ 버그 수정(bug fix)의 경우

- Functional patch: 절차적 프로그래밍 환경에서 소프트웨어 함수의 새 버전을 동적으로 갱신
- Class patch: 객체지향 코드에서 runtime 에 class member(데이터와 함수)를 추가

### ■ 새 기능(new feature)의 경우

- Hitless ISSU(In-Service Software Upgrade): Active/Passive Redundancy tactic 을 활용해 서비스 중단 없이 업그레이드

## Fault Recovery — Retry, Ignore, Degradation, Reconfiguration

### ■ Retry

- 일시적(transient) 결함을 감추기 위해 연산을 재시도한다
- 재시도 횟수에는 제한을 두어야 한다
- 장애가 흔하고 예상되는 network 나 server farm 에서 사용한다

### ■ Ignore faulty behavior

- 특정 source 에서 온 message 가 가짜(spurious)라고 판단되면 그 message 를 무시한다
- 예: DoS(denial-of-service) 공격을 하는 외부 component 의 message 를 Access Control List filter 로 걸러낸다

### ■ Degradation

- component 장애가 있을 때 가장 중요한 시스템 기능은 유지하고, 덜 중요한 기능은 포기한다

### ■ Reconfiguration

- 남아 있는(제한된) 자원에 책임을 다시 할당하여 component 장애에서 복구한다
- 가능한 한 많은 기능을 유지하는 것이 목표다

## Preparation and Repair Tactics 요약

### ■ 중복 도입 (Introducing redundancy)

- Active redundancy
- Passive redundancy
- Spare

### ■ 계산 재시도 (Retrying a computation)

- Rollback
- Retry
- Reconfiguration

### ■ 복구 준비 (Preparing for recovery)

- Exception handling
- Software upgrade

### ■ 제한된 동작 (Limited functioning)

- Ignore faulty behavior
- Degradation

**핵심** Preparation and Repair tactic 은 중복 도입, 계산 재시도, 복구 준비를 기반으로 한다

## Fault Recovery: Reintroduction — Shadow Operation

### ■ Reintroduction(재투입)이란

- 장애가 났던 component 를 고친 뒤 다시 시스템에 투입하는 것

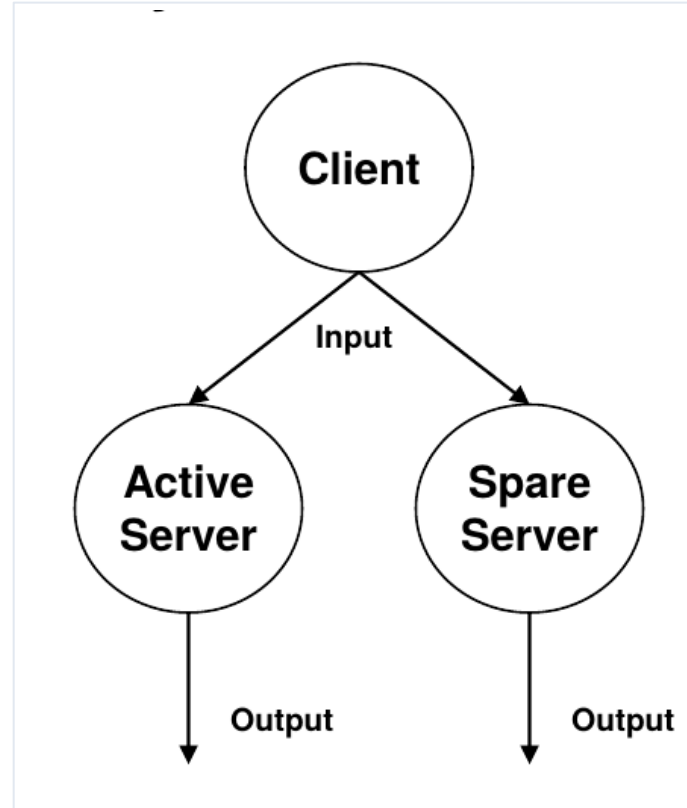
### ■ Shadow operation

- 이전에 장애가 났거나 in-service 업그레이드된 component 를 active 역할로 되돌리기 전에 잠시 "shadow mode"로 실행한다
- 그 동안 component 의 동작이 올바른지 모니터링할 수 있다

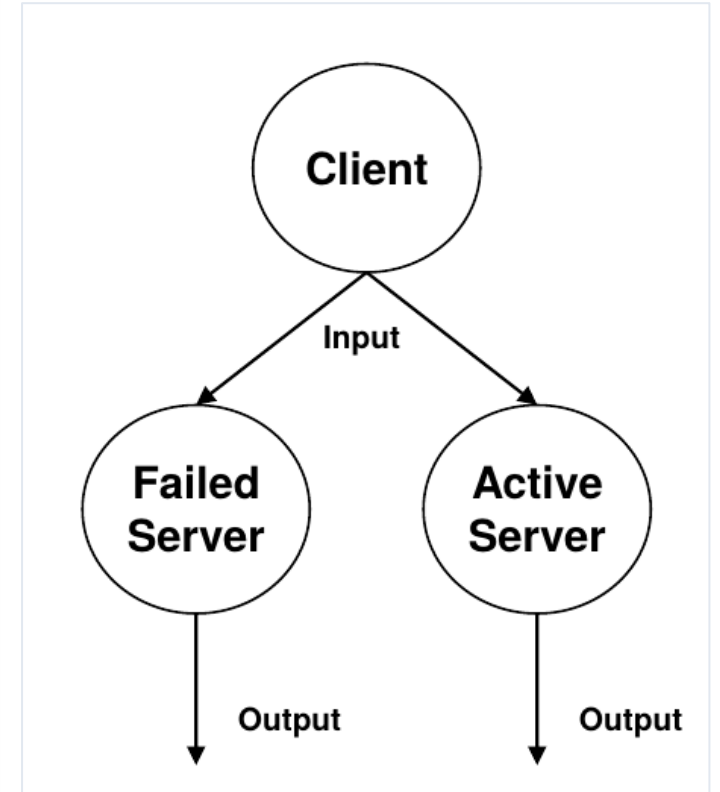
## Reintroduction — State Resynchronization (1)

### ■ State Resynchronization

- Active Redundancy 및 Passive Redundancy tactic 의 재투입 짝(partner) 이 되는 tactic
- Active Redundancy 와 함께 쓰면 상태 재 동기화가 자연스럽게(organically) 일어난다
  - spare 가 같은 입력을 계속 처리하고 있으므로, 고장난 서버를 고쳐 다시 넣기만 하면 된다



정상: Active/Spare 가 같이 입력 처리

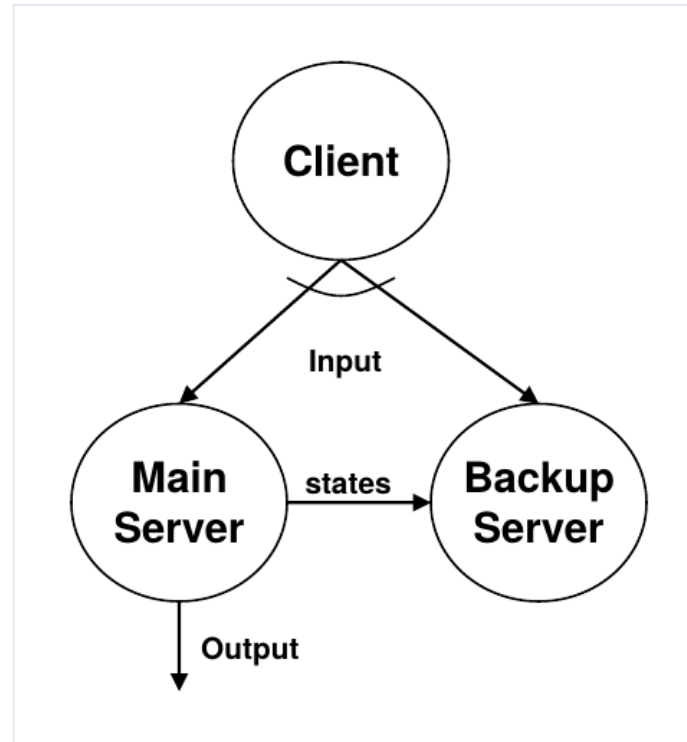


장애 후: Spare 가 Active 로 전환

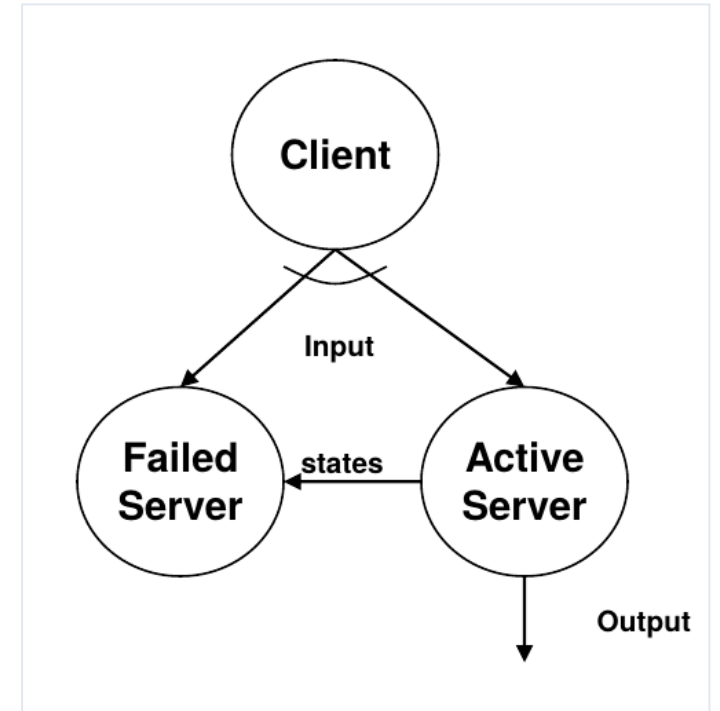
## Reintroduction — State Resynchronization (2)

### ■ Passive Redundancy(warm sparing)와 함께 쓸 때

- State Resynchronization 은 active component 가 standby component 에 주기적으로 보내는 상태 정보에만 의존한다
  - Backup 은 마지막으로 받은 states 를 기준으로 Active 역할을 이어받는다



정상: Main 이 Backup 에 states 전달

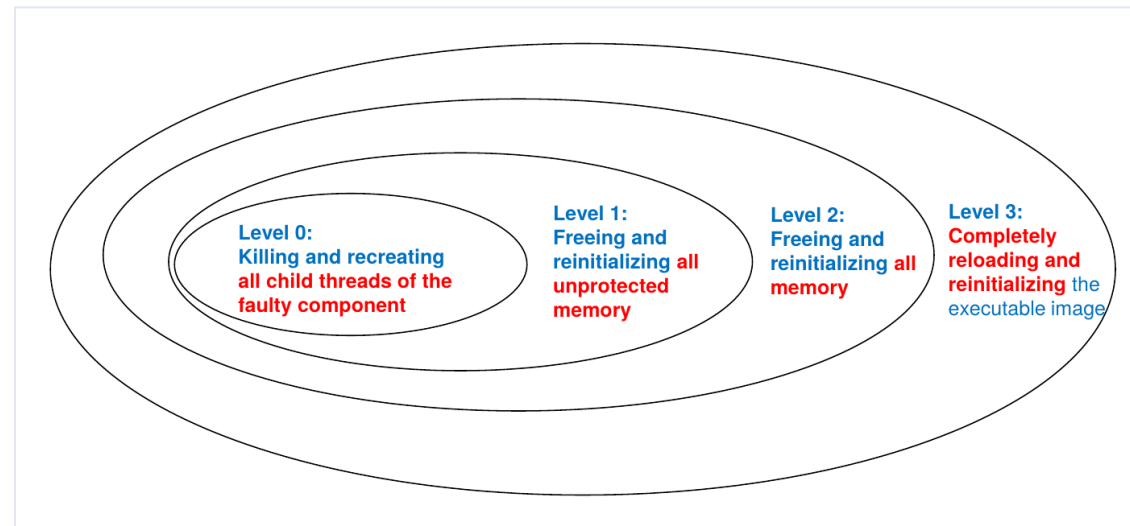


장애 후: Backup 이 Active 로 전환

## Reintroduction — Escalating Restart

### ■ Escalating Restart

- 재시작하는 component 의 단위(granularity)를 단계적으로 키워 가며 결함에서 복구한다 — 서비스에 미치는 영향을 최소화
- Level 0: 결함 component 의 child thread 를 모두 죽이고 재생성
- Level 1: 보호되지 않은(unprotected) memory 를 해제·재초기화
- Level 2: 모든 memory 를 해제·재초기화
- Level 3: 실행 이미지(executable image) 전체를 다시 로드·재초기화



Level 0 → 3 으로 갈수록 재시작 범위가 커진다



## Fault Prevention — Removal, Predictive Model, Exception Prevention

### ■ Removal from service

- 잠재적인 시스템 장애를 완화하기 위해 시스템 component 를 일시적으로 서비스에서 제외(out-of-service)한다
- 예: memory leak 을 없애기 위해 component 를 서비스에서 빼고 reset 한다

### ■ Predictive model

- 시스템 process 의 건강 상태(health)를 모니터링한다
- 미래의 결함을 예측할 수 있는 조건이 감지되면 선제적으로 교정 조치를 취한다

### ■ Exception prevention

- 시스템 exception 이 아예 일어나지 않도록 막는다

### ■ 예: Smart Pointer / Wrapper

- pointer 처럼 동작하면서 추가 기능을 제공하는 추상 데이터 타입
  - 자동 메모리 관리(automatic memory management)
  - 경계 검사(bounds checking)

## Fault Prevention — Increase Competence Set

### ■ Increase competence set

- 더 많은 예외 상황을 정상 동작의 일부로 처리하도록 component 를 설계한다

### ■ 예: 공유 자원 접근이 막혔을 때

- Case 1: component 가 exception 을 던진다
- Case 2: 다음에 접근 가능해지면 스스로 작업을 완료하겠다는 표시와 함께 즉시 return 한다
- Case 2 의 component 가 Case 1 보다 더 큰 competence set 을 가진다

**핵심** Competence set 이 크다 = 예외 상황을 '예외'가 아닌 '정상 처리 범위'로 다룬다

## Fault Prevention Tactics 요약

### ■ Runtime 모니터링 기반

- Removal from service
- Predictive model

### ■ 결함 제거 메커니즘 기반

- Exception prevention (예: Smart Pointer)

### ■ 참고

- Increase competence set 은 Fault Recovery(Preparation and Repair) tactic 으로 분류할 수도 있다

# 3.3

## Deployability Tactics

소프트웨어를 예측 가능한 시간과 비용으로 운영 환경에 배포하기 위한 تاک

- Deployment 와 Deployability 의 개념
- Deployability 일반/구체 시나리오
- Manage Deployment Pipeline: Scale Rollouts, Rollback, Script Deployment Commands
- Manage Deployed System: Manage Service Interactions, Package Dependencies, Toggle Features

## Deployment(배포)과 배포 파이프라인

### ■ 배포(Deployment)란?

- 개발된 소프트웨어를 사용 가능한 상태로 만드는 일련의 활동
  - 소프트웨어를 개발 환경에서 실제 운영 환경으로 이동·설치·구성·활성화하는 과정

### ■ 배포 파이프라인(Deployment Pipeline)

- 배포 과정을 여러 단계로 세분화하고, 가능한 한 자동화하여 흐름을 구성한 것
- 커밋 → 빌드 → 자동화된 테스트 → 승인/검토 → 스테이징 → 프로덕션 배포



배포 파이프라인의 단계

## Deployability(배포용이성)의 정의와 Architecture 고려사항

### ■ Deployability(배포용이성)

- 소프트웨어가 예측 가능하고 허용 가능한 시간과 비용 내에 운영 환경에 배포될 수 있는 능력
- 운영 중단을 최소화하고, 배포로 인한 장애와 롤백을 신속하게 처리한다

### ■ Architecture 고려사항

- 배포 단위: 전체 시스템, 일부 구성요소, 독립 배포 가능한 단위(Microservice 구조)
- 모니터링 및 롤백: 배포된 항목의 운영을 모니터링하고, 실패한 배포를 롤백할 수 있는 기능을 제공한다
- 효율성: 합리적인 수준의 시간과 노력으로 배포를 지원한다

**핵심** Deployability 는 '얼마나 빠르고 안전하게, 되돌릴 수 있게 배포하는가'의 문제이다

# Deployability 일반 시나리오 (1)

| 부분           | 설명        | 가능한 값                                                                                                                                                                                                                   |
|--------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 자극원(Source)  | 배포 요청의 주체 | 최종 사용자, 개발자, 시스템 관리자, 운영 인력, 컴포넌트 마켓플레이스, 제품 소유자                                                                                                                                                                        |
| 자극(Stimulus) | 배포 요청     | (최종 사용자) 신규 기능 활성화 요청 / (개발자) 코드 변경 결과의 배포 요청 / (시스템 관리자) 인프라 설정 변경(예: 보안 패치)에 따른 운영 서비스 재배포 요청 / (운영 인력) 일부 사용자에게만 새 버전 배포 요청 / (컴포넌트 마켓플레이스) 외부 구성요소(플러그인, 라이브러리 등) 변경에 따른 배포 요청 / (제품 소유자) 비즈니스 요구나 릴리즈 계획에 따른 배포 결정 |
| 대상(Artifact) | 배포 대상     | 서비스 컴포넌트(혹은 모듈), 시스템의 플랫폼 컴포넌트, UI 컴포넌트, 시스템과 상호 운영되는 다른 시스템의 컴포넌트                                                                                                                                                      |

## Deployability 일반 시나리오 (2)

### ■ 시나리오의 여섯 부분

- Source → Stimulus → Artifact → Environment → Response → Response Measure 순서로 배포 요청과 그 결과를 기술한다

| 부분                          | 설명                    | 가능한 값                                                                   |
|-----------------------------|-----------------------|-------------------------------------------------------------------------|
| 환경<br>(Environment)         | 배포 환경                 | 스테이징 환경, 실제 운영 환경                                                       |
| 응답(Response)                | 배포 요청에 따른 시스템의 반응     | 새로운 컴포넌트가 배포된다 / 새로 배포된 컴포넌트를 모니터링한다 / 배포된 컴포넌트를 이전 배포로 롤백한다            |
| 응답 측정<br>(Response Measure) | 배포에 대한 비용, 시간, 효과성 측정 | 시간(배포 시간, 서비스 중단 시간, 롤백 시간) / 노력 비용 / 배포 성공·실패율 / 다른 기능과 품질에 영향을 미치는 정도 |



## 구체적인 Deployability 시나리오

### ■ 시나리오 예

- "인증 및 권한 부여 서비스의 신규 릴리스가 컴포넌트 마켓플레이스에서 사용 가능해졌다. 새 서비스의 테스트와 실제 운영 환경 배포는 40시간 이내, 노력은 120 맨아워 이내로 끝나야 한다. 배포로 인한 결함이 없어야 하고 SLA 도 준수해야 한다"

| 부분              | 값                                       |
|-----------------|-----------------------------------------|
| 자극원(Source)     | 컴포넌트 마켓플레이스                             |
| 자극(Stimulus)    | 인증 및 권한 부여 서비스의 신규 릴리스                  |
| 대상(Artifact)    | 인증/권한 부여 서비스                            |
| 환경(Environment) | 실제 운영 환경                                |
| 응답(Response)    | 새로운 서비스를 자체적으로 테스트하고 실제 운영 서버로 배포한다     |
| 응답 측정(Measure)  | 시간 40시간 이내, 노력 120 맨아워 이내, 결함 없이 SLA 준수 |

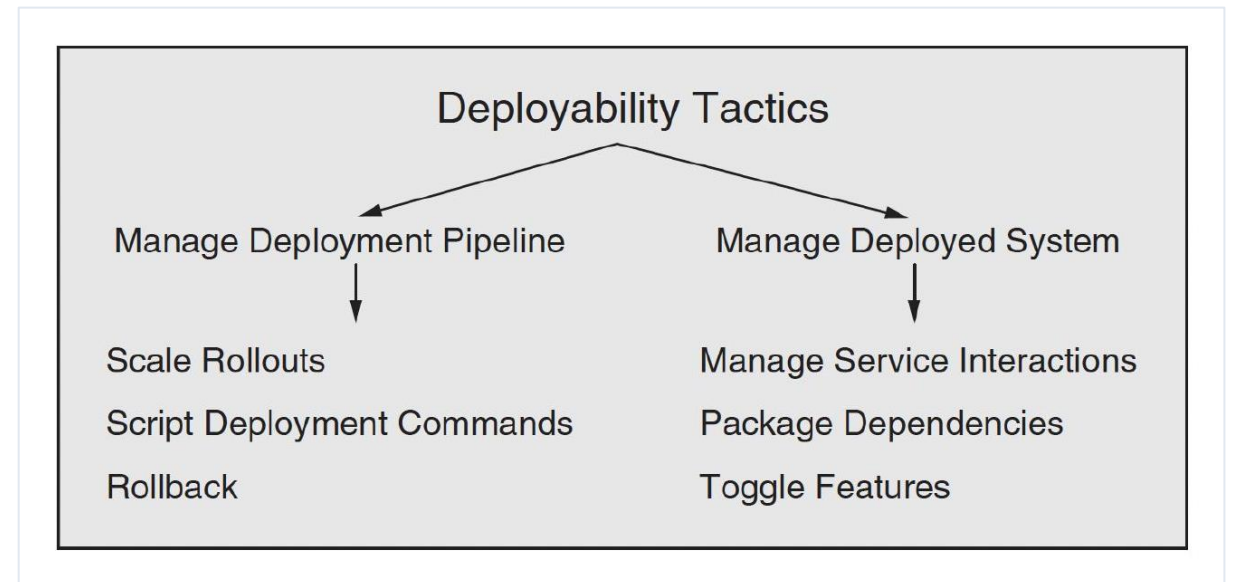
## Deployability Tactics 의 분류

### ■ Manage Deployment Pipeline (배포 파이프라인 관리)

- Scale Rollouts: 단계적으로 배포 범위를 넓힌다
- Rollback: 문제가 생기면 이전 버전으로 되돌린다
- Script Deployment Commands: 배포 명령을 스크립트화한다

### ■ Manage Deployed System (배포된 시스템 관리)

- Manage Service Interactions: 여러 버전의 서비스가 공존하도록 상호작용을 중재한다
- Package Dependencies: 종속 요소를 함께 패키징한다
- Toggle Features: 실행 시간에 기능을 켜고 끈다



Deployability Tactics 계층도

## Scale Rollouts (단계적 롤아웃)

### ■ 기본 개념 — 네 단계

- 초기 배포 단계: 새 버전(V2)을 소수의 인스턴스나 사용자 그룹에 먼저 배포한다
- 모니터링 및 검증 단계: 성능 저하, 오류율, 사용자 피드백 등을 관찰한다
- 점진적 확대 단계: 문제가 없음이 확인되면 배포 비율을 점차 늘린다
- 완료 및 안정화: 전체 시스템으로 확장을 마친 뒤 상태를 점검하고 보고한다

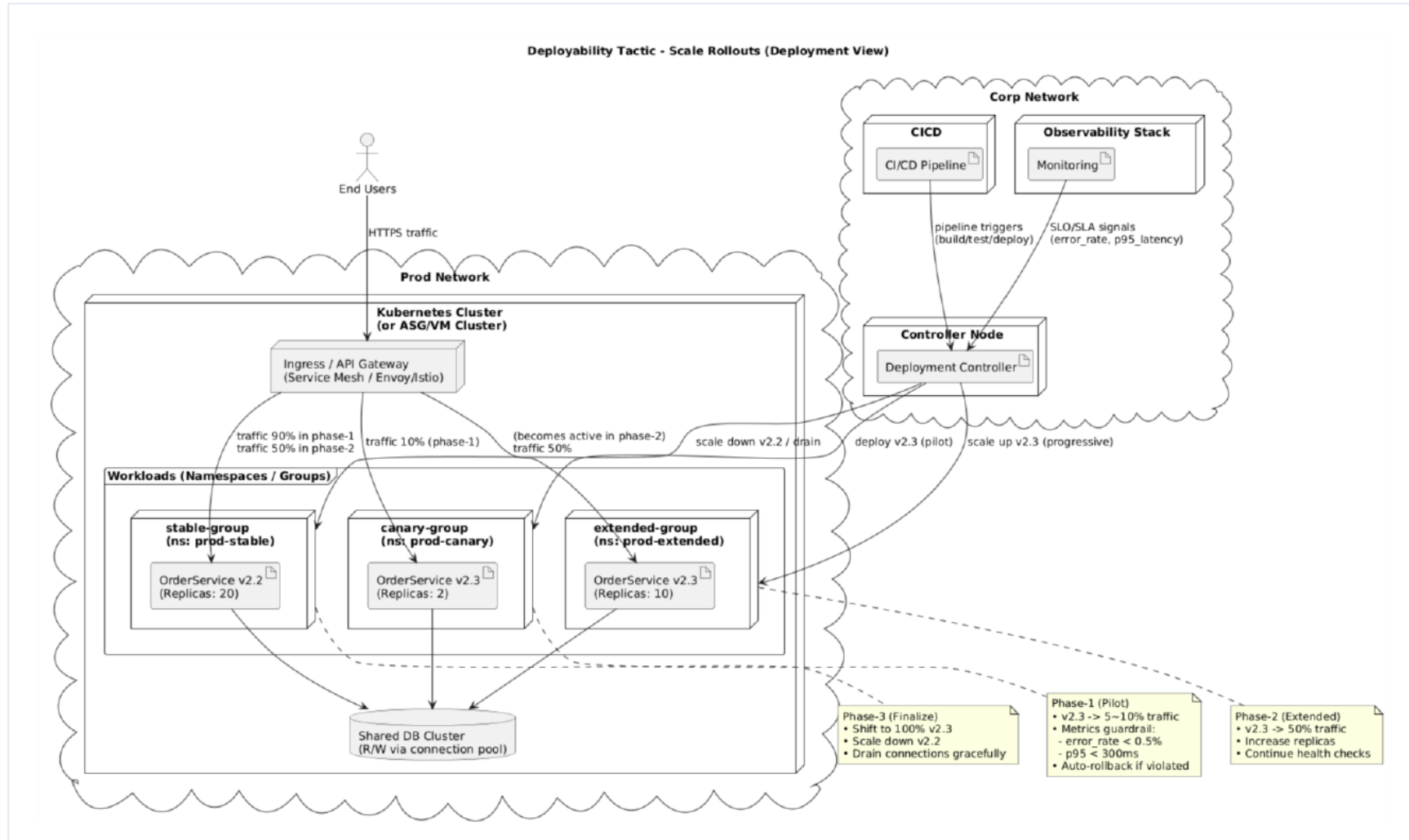
### ■ 관련 패턴 — 카나리아 테스트(Canary Test)

- 제한된 사용자(1~10%)를 대상으로 새 릴리즈를 먼저 테스트하여 이상 유무를 조기에 탐지하는 구체적 기법

### ■ 카나리아 적용(Kubernetes 예)

- Ingress/API Gateway 가 트래픽을 stable 그룹과 canary 그룹으로 나누어 라우팅한다
- Deployment Controller 가 관측 지표를 보고 canary 비율을 단계적으로(Phase 1 → 2 → 3) 확대한다
- 이상이 감지되면 canary 그룹을 줄이거나 제거해 되돌린다

## Scale Rollouts — 카나리아 적용 예시



Kubernetes 카나리아 배포: Gateway 가 stable/canary 그룹으로 트래픽을 나누고, Controller 가 지표를 보며 canary 비율을 단계적으로 확대한다

# Rollback (롤백)

## ■ 기본 개념

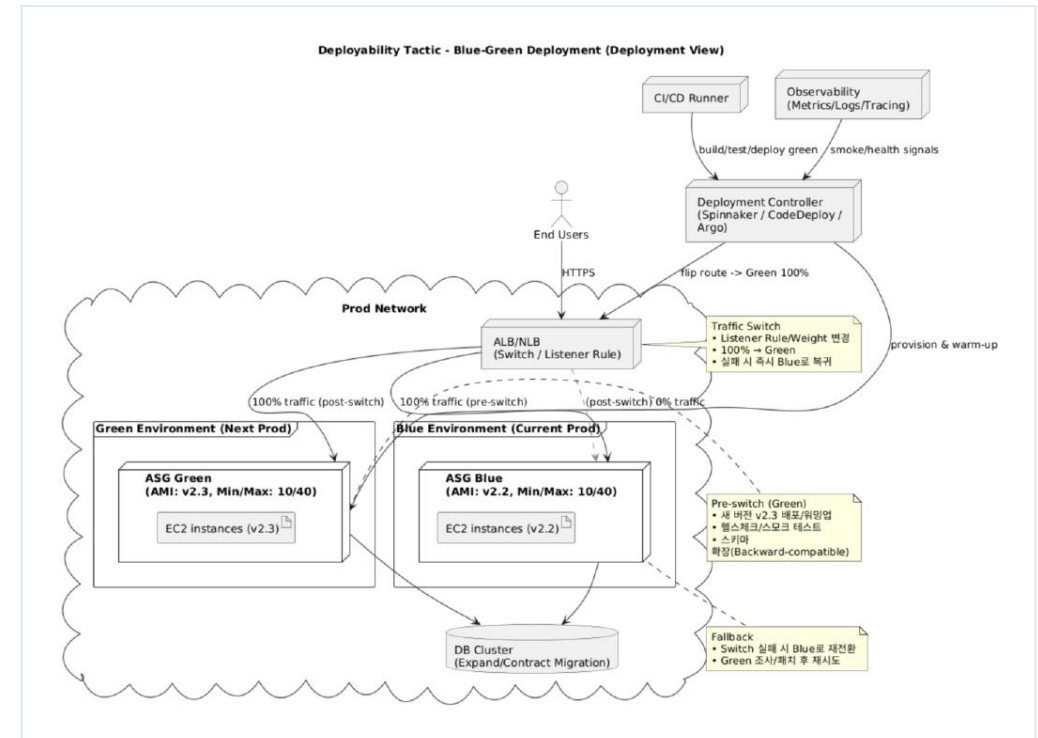
- 새 버전의 배포 중 또는 배포 이후 문제가 발생하면, 시스템을 이전의 안정된 버전으로 신속하게 복원하는 방법
- 롤백 시 배포로 이루어진 모든 업데이트 결과를 전부 추적하여 되돌릴 수 있어야 한다

| 구현 유형                             | 설명                                         | 적용 예시                                                        |
|-----------------------------------|--------------------------------------------|--------------------------------------------------------------|
| <b>Versioned Rollback</b>         | 이전 애플리케이션 버전(컨테이너 이미지, 바이너리 등)을 유지하고 즉시 복원 | [Kubernetes] kubectl rollout undo, [AWS CodeDeploy] rollback |
| <b>Blue-Green Rollback</b>        | 이전 버전 환경(Blue)을 유지하고 트래픽만 전환               | Blue-Green 배포 전략의 핵심                                         |
| <b>Snapshot Rollback</b>          | DB 나 상태 저장소의 스냅샷을 복구                       | RDS Snapshot Restore, EBS Snapshot                           |
| <b>Transaction-Level Rollback</b> | 배포 중 트랜잭션 단위로 취소/보상 수행                     | Saga Pattern                                                 |

## Rollback — Blue-Green Rollback 적용 예시

### ■ Blue-Green 배포와 롤백

- Blue(현재 운영, v2.2)와 Green(다음 운영, v2.3) 환경을 나란히 두고 Traffic Switch 로 트래픽을 전환한다
- Green 에 100% 전환 후 문제가 생기면 Blue 로 트래픽만 다시 돌려 즉시 롤백한다
- DB 스키마는 Expand & Contract 방식으로 양쪽 버전과 호환되게 유지한다



Deployability Tactic – Blue-Green Deployment (Deployment View)

## Rollback — Saga Rollback 적용

### ■ Saga 패턴

- 로컬 트랜잭션: 각 서비스가 자체 DB 에서 원자적으로 완료한다
- 보상 트랜잭션(Compensation): 실패 시 이미 끝난 앞 단계들을 역순으로 되돌리는 취소 작업
- 최종적 일관성: 순간적으로 불일치가 있어도 재시도/보상 후 일관된 상태로 수렴한다

### ■ 배포 관점의 의미

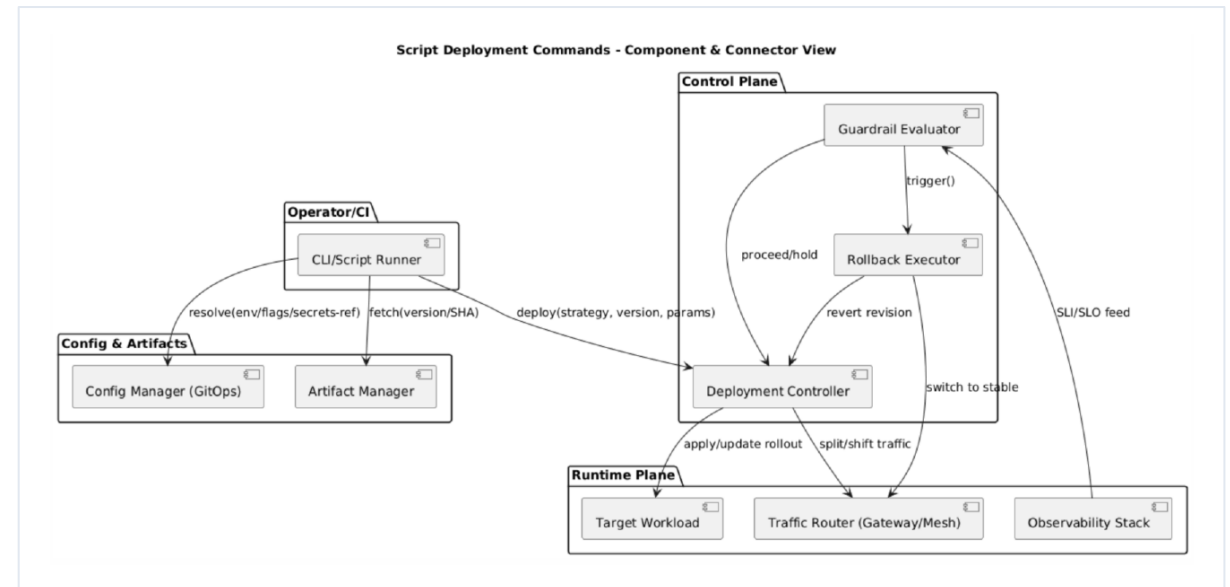
- Saga 패턴은 배포 중/직후에 이미 발생한 도메인 부작용(결제, 재고, 포인트 적립 등)을 보상 트랜잭션으로 되돌리는 비즈니스 레벨 롤백이다

**핵심** 코드 버전만 되돌리는 것이 아니라, 이미 일어난 비즈니스 효과까지 되돌려야 완전한 롤백이다

# Script Deployment Commands

## ■ 기본 개념

- 빌드/패키징/배포/검증/롤백까지를 스크립트 명령 세트로 일관되게 수행하도록 만드는택틱
- Operator/CI 가 CLI/Script Runner 를 실행하면 Config Manager, Artifact Manager, Deployment Controller 가 정해진 순서대로 동작한다
- Guardrail Evaluator 가 SLI/SLO 를 검사하고, 실패하면 Rollback Executor 가 자동으로 되돌린다



Script Deployment Commands – Component & Connector View



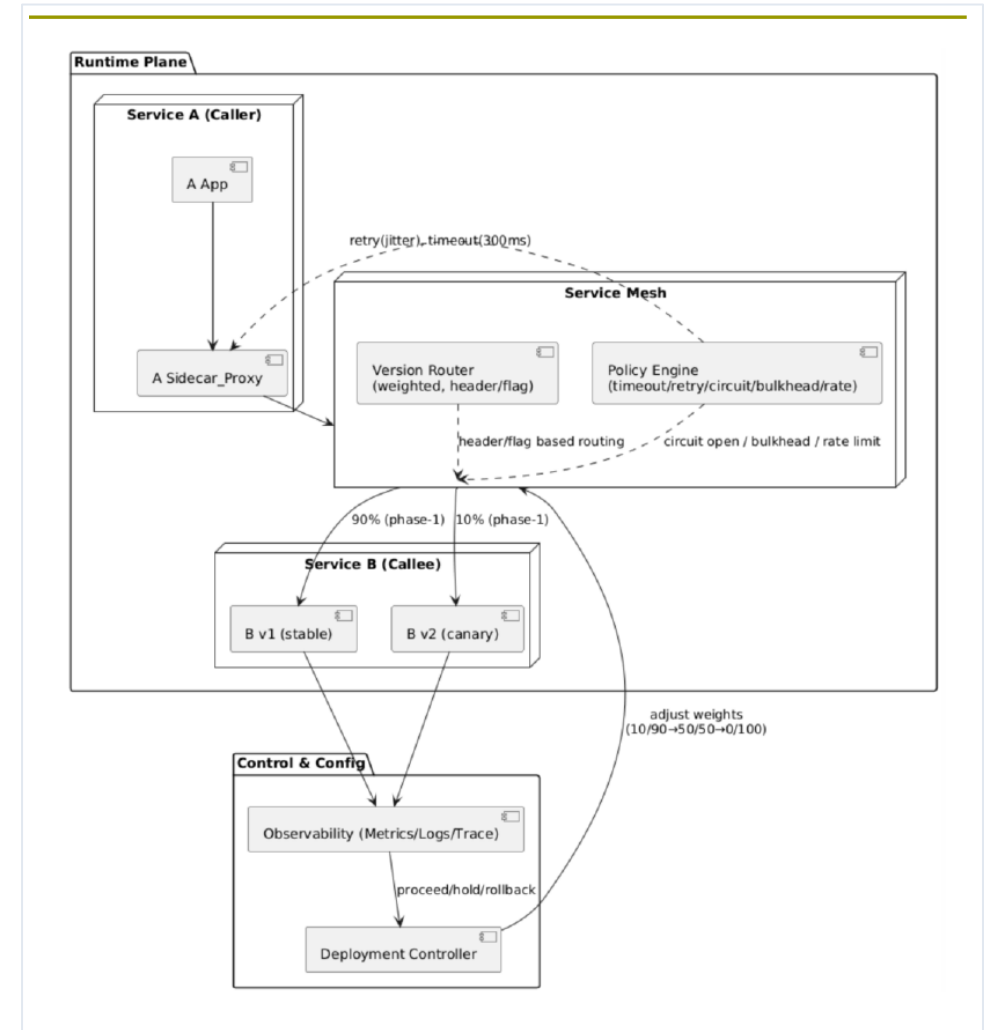
# Manage Service Interactions

## ■ 기본 개념

- 동일한 서비스의 여러 버전이 동시에 운영 중인 상황에서, 버전 호환성 문제를 선제적으로 피하기 위해 서비스 간 상호작용을 중재하는택틱

## ■ 다이어그램 설명

- Sidecar Proxy + Service Mesh 가 버전 라우팅과 호출 안정화 정책을 집행한다
- 관측 지표를 바탕으로 컨트롤러가 배포 비율을 자동 조정한다(진행/보류/롤백)
- 결과적으로 무중단·저위험 배포, 연쇄 장애 방지, 구·신버전 공존이 가능해진다



Service Mesh 를 이용한 버전 간 상호작용 관리

# Package Dependencies

## ■ 기본 개념

- 배포 시점에 애플리케이션이 필요로 하는 모든 종속 요소를 함께 패키징하는 태틱
- 개발 환경에서 실제 운영 환경으로 옮길 때 종속 요소의 버전이 일관되게 유지된다

## ■ 종속 요소(Dependencies)

- 소스 코드 라이브러리뿐 아니라 빌드.런타임.플랫폼.운영까지, 배포 성공에 영향을 주는 모든 의존 대상을 포함한다

## ■ 종속 요소를 패키지화하는 방법

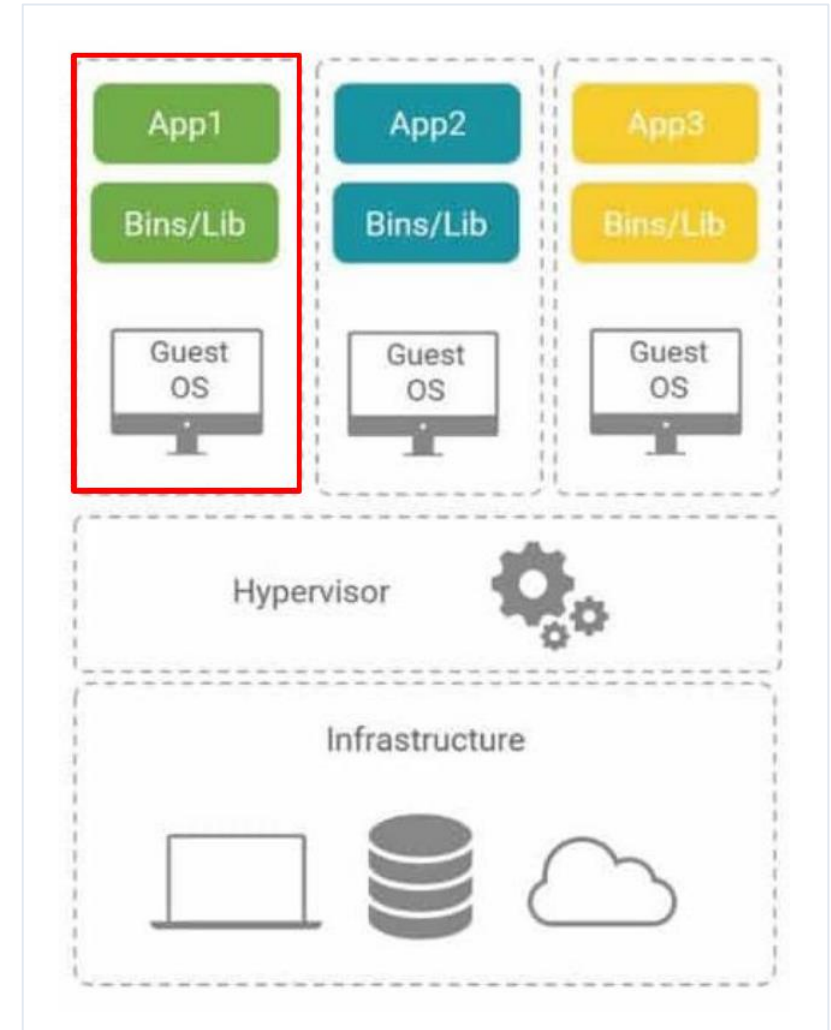
- 가상머신(Virtual Machine) 기반 패키징
- 컨테이너(Container) 기반 패키징

**핵심** "내 PC에서는 되는데..." 문제를 막으려면 앱과 함께 의존 요소까지 하나로 묶어 배포한다

## Package Dependencies — 가상머신 기반 패키징

### ■ 가상머신 기반 패키징

- Guest OS(커널 포함) + 시스템 패키지 + 앱을 통째로 이미지로 패키징
- 격리 경계: 하드웨어 수준(하이퍼바이저)
- 호스트와 완전히 분리되어 의존성 충돌 위험이 최소화

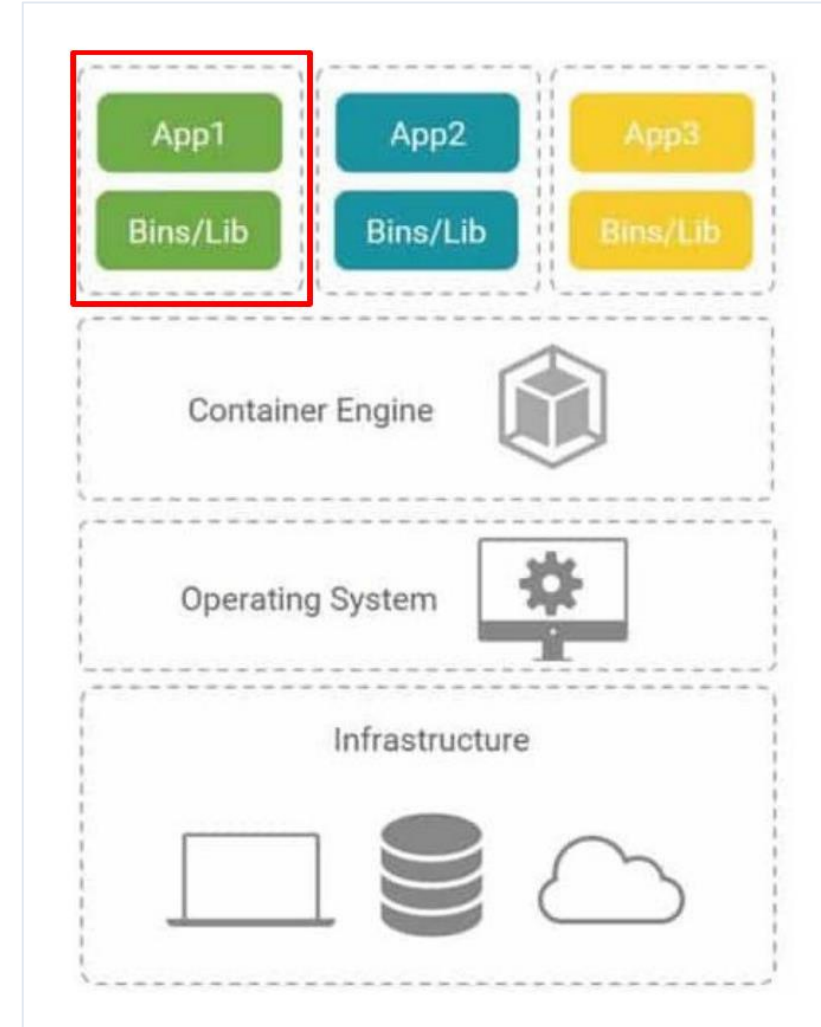


가상머신 기반 패키징

## Package Dependencies — 컨테이너 기반 패키징

### ■ 컨테이너 기반 패키징

- 언어 런타임 + 시스템 라이브러리 + 앱을 이미지로 패키징
- OS 커널은 호스트와 공유한다
- 격리 경계: 프로세스/파일시스템/네트워크 네임스페이스



컨테이너 기반 패키징

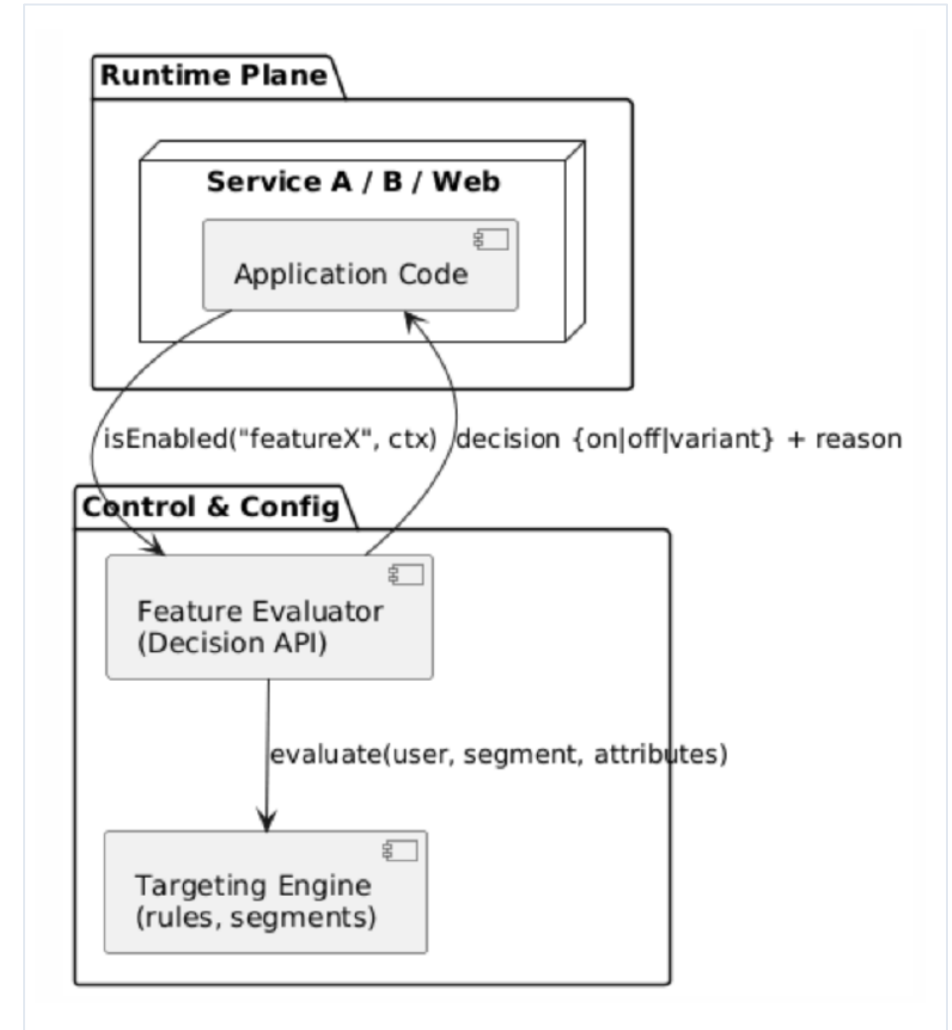
## Toggle Features (기능 활성화/비활성화)

### ■ 기본 개념

- 신규 기능을 배포한 이후에 '비상 정지 스위치'를 통해 실행 시간에 기능을 비활성화함으로써, 재배포의 비용과 위험을 제어하는 تاک

### ■ 동작 방식

- Application Code 는 isEnabled("featureX", ctx) 로 기능 활성화 여부를 묻는다
- Feature Evaluator(Decision API)가 사용자·세그먼트·속성을 평가한다
- Targeting Engine 이 규칙과 세그먼트에 따라 on/off/variant 를 결정한다



Feature Toggle 의 구조

# 3.4

## Energy Efficiency Tactics

기능을 수행하면서 소비하는 에너지를 최소화하기 위한 택틱

- Energy Efficiency 의 정의와 고려사항
- Energy Efficiency 일반/구체 시나리오
- Monitor Resources: Metering, Static Classification, Dynamic Classification
- Allocate Resources: Reduce Usage, Discovery, Schedule Resources

## Energy Efficiency(에너지 효율성)의 정의와 고려사항

### ■ Energy Efficiency(에너지 효율성)

- 소프트웨어 시스템이 주어진 기능을 수행하는 동안 소비되는 에너지의 양을 최소화할 수 있는 능력

### ■ Architecture 고려사항

- 소프트웨어가 실행될 때 CPU, 메모리, 네트워크, 스토리지 등의 사용량을 최소화하도록 설계하는 것
  - 필요한 만큼의 컴퓨팅 자원을 적시에 사용할 수 있는 방법은?
  - 데이터 처리를 위한 양과 시간을 최소화할 수 있는 방안은?
  - 네트워크 통신의 빈도와 통신량을 최소화할 수 있는 방안은?

**핵심** 에너지 효율은 결국 '자원을 얼마나 적게, 얼마나 적절한 때에 쓰는가'의 문제이다

# Energy Efficiency 일반 시나리오

| 부분                      | 설명                         | 가능한 값                                                  |
|-------------------------|----------------------------|--------------------------------------------------------|
| 자극원(Source)             | 에너지 절약 요청의 주체              | 최종 사용자, 시스템 관리자, 에너지 관리 시스템                            |
| 자극(Stimulus)            | 에너지 절약 요청                  | 불필요하게 높은 전력 소비(전체 사용량, 순간 최대 사용량, 평균 사용량)를 절약하길 원함     |
| 대상(Artifact)            | 관리할 대상                     | 특정 장치, 서버, VM, 클러스터                                    |
| 환경(Environment)         | 어떤 운영 상태에서 요청이 발생하는가?      | 정상 운전 상태, 혹은 부하가 낮거나 유휴(Idle) 상태의 실행 환경                |
| 응답(Response)            | 에너지 절약을 위한 행동              | 서비스 비활성화 / 런타임 서비스 할당 해제 / 서비스의 서버 할당 변경 / 서비스 수준 변경   |
| 응답 측정(Response Measure) | 얼마나 에너지를 절감했는가를 어떻게 측정하는가? | 절약된 에너지 소비량(평균/전체) / 배터리 소모 감소율(%) / CPU·GPU 사용 감소율(%) |



# 구체적인 에너지 절감 시나리오

## ■ 시나리오 예

- "시스템 관리자는 피크타임이 아닐 때 사용하지 않는 리소스를 해제하여 에너지를 절약하기를 원한다. 시스템은 데이터베이스 질의 응답 시간 요건인 2초를 유지하면서 리소스를 해제해, 필요한 전체 에너지의 50%를 평균적으로 절약한다"

| 부분              | 값                                   |
|-----------------|-------------------------------------|
| 자극원(Source)     | 시스템 관리자                             |
| 자극(Stimulus)    | 에너지 절약 요청                           |
| 대상(Artifact)    | 시스템                                 |
| 환경(Environment) | 피크타임이 아닐 때                          |
| 응답(Response)    | 사용하지 않는 리소스를 해제                     |
| 응답 측정(Measure)  | DB 질의 응답시간 2초 유지, 전체 에너지의 50% 평균 절약 |

## Energy Efficiency Tactics 의 분류

### ■ Monitor Resources (자원 모니터링)

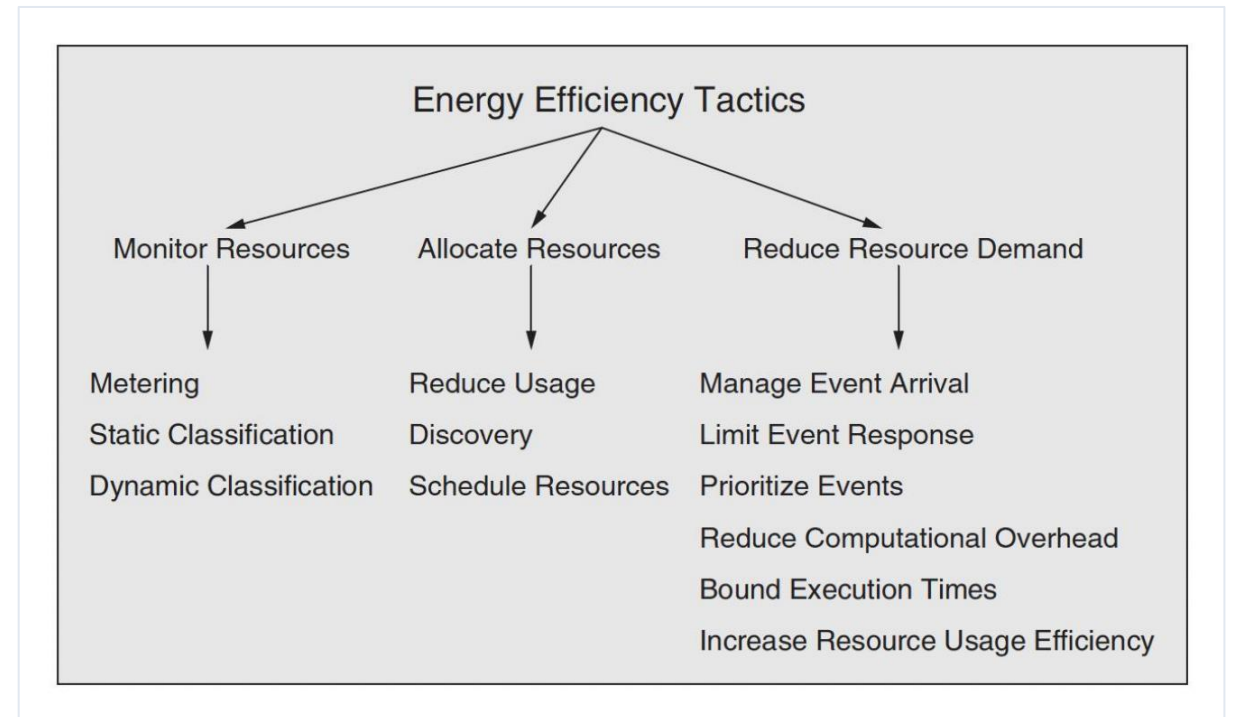
- Metering, Static Classification, Dynamic Classification

### ■ Allocate Resources (자원 할당)

- Reduce Usage, Discovery, Schedule Resources

### ■ Reduce Resource Demand (자원 요구 감소)

- Manage Event Arrival, Limit Event Response, Prioritize Events, Reduce Computational Overhead, Bound Execution Times, Increase Resource Usage Efficiency
- Performance Tactics 와 공유하는 태틱들이다



Energy Efficiency Tactics 계층도

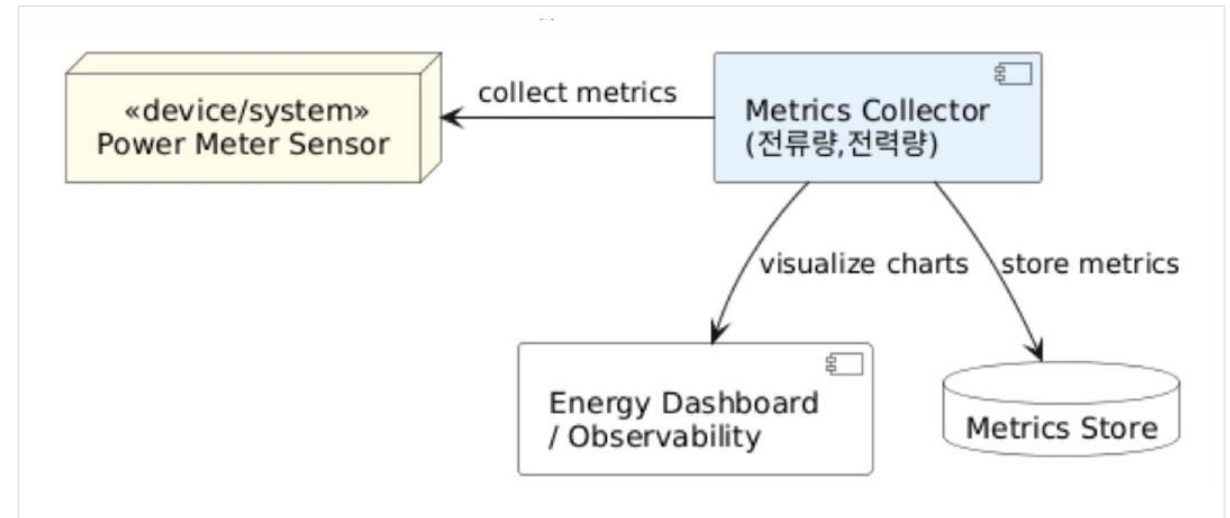
## Monitor Resources :: Metering

### ■ 기본 개념

- Metering 은 시스템의 에너지 소비를 센서 인프라를 통해 실시간 혹은 주기적으로 측정하고 모니터링하는택틱
- 데이터센터의 전력량계: 데이터센터 전체의 에너지 소비 측정
- 전류계나 전력계: 개별 서버나 하드 드라이브의 에너지 소비 측정
- 배터리 관리 시스템: 배터리로 운영되는 시스템의 에너지 소비 측정

### ■ 구조

- Power Meter Sensor → Metrics Collector → Metrics Store / Energy Dashboard



Metering 의 구성

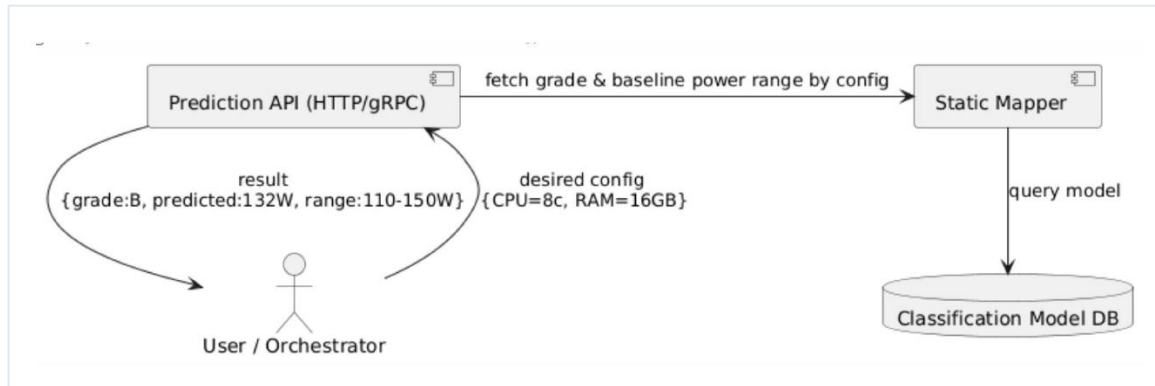
## Monitor Resources :: Static / Dynamic Classification

### ■ Static Classification (정적 분류)

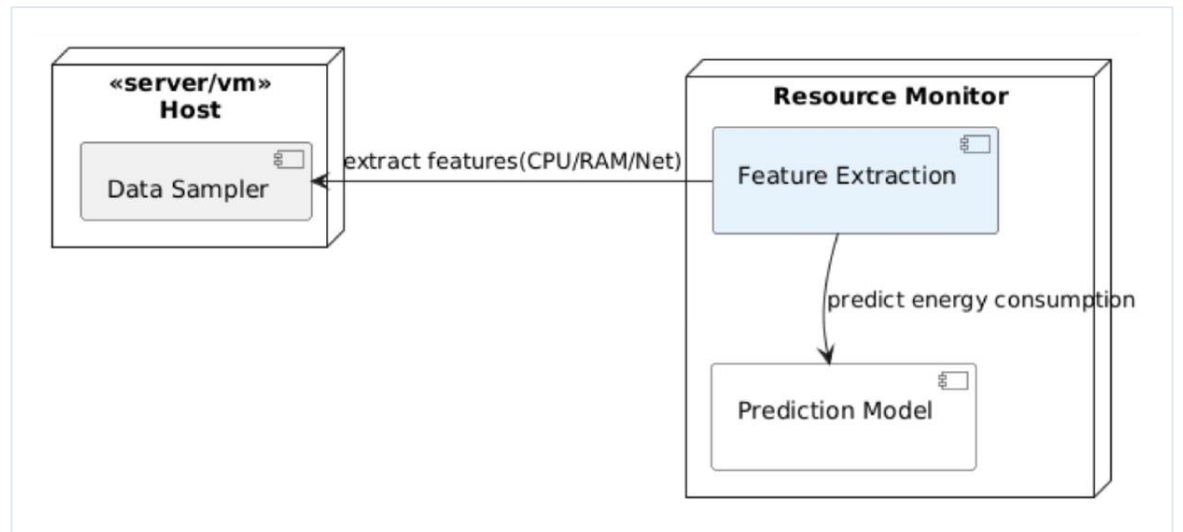
- 실시간 에너지 데이터 수집이 불가능한 경우(예: 원격 클라우드), 자원의 사전 정의되거나 고정된 특성(CPU 수, RAM 용량 등)을 기반으로 에너지 소비량을 예측한다

### ■ Dynamic Classification (동적 분류)

- 컴퓨팅 리소스의 정적 모델이 불충분한 경우, 동적 요인을 분석하여 실시간으로 에너지 소비를 예측한다
  - 작업부하(Workload): 현재 CPU 사용률, 메모리 사용량, 네트워크 트래픽 등



Static Classification: 고정 특성으로 예측



Dynamic Classification: 실시간 특성으로 예측

## Allocate Resources :: Reduce Usage

### ■ 에너지 사용 최소화 방법

- 화면 재생 빈도 줄이기
- 배경(백그라운드) 어둡게 하기
- 하드 드라이브 회전 속도 줄이기
- CPU 를 느린 클럭 속도로 실행
- 가상머신들을 최소 개수의 물리 서버에 배치
- 모바일 앱은 컴퓨팅의 일부를 클라우드에 배치

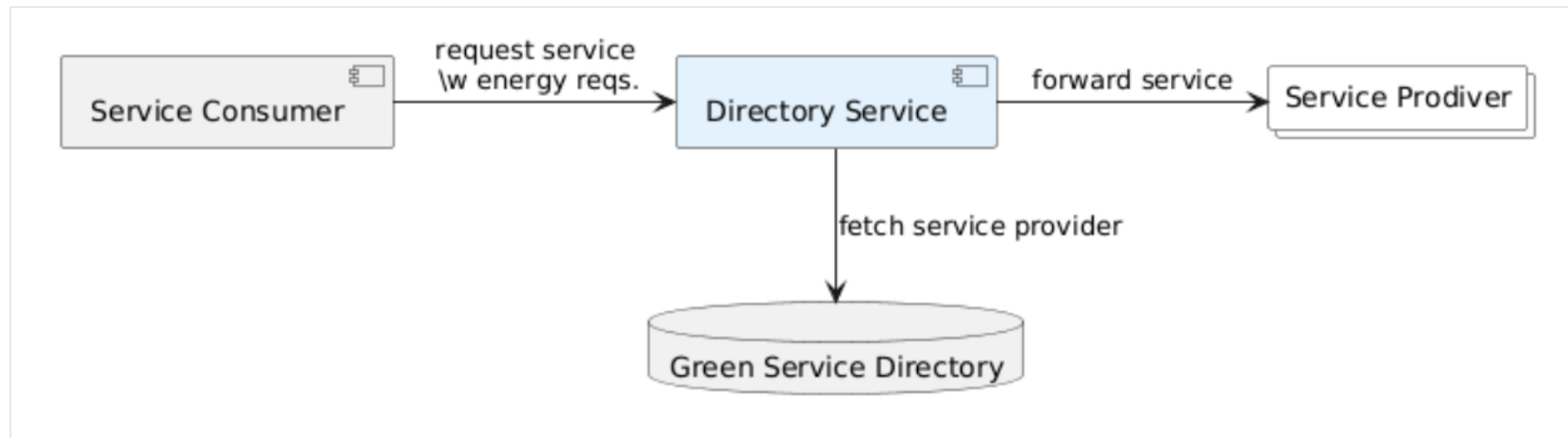
### ■ 불필요한 에너지 소비 제거 방법

- CPU 또는 서버 끄기
- 사용 중이지 않은 프로세서 블록에 공급되는 전류 끄기

## Allocate Resources :: Discovery

### ■ 기본 개념

- Discovery Service 는 클라이언트의 서비스 요청을 서비스 공급자와 매칭하는 역할을 한다
- Green Service Directory 는 각 서비스 공급자(VM, 서버, Microservice 등)의 정량화된 에너지 특징(예: 트랜잭션당 에너지 소비율)을 저장한다
- 서비스 요청이 에너지 정보(예: 최대 허용 에너지 소비량)를 포함하면, Discovery Service 는 Green Service Directory 의 에너지 특징을 기준으로 공급자를 선택한다

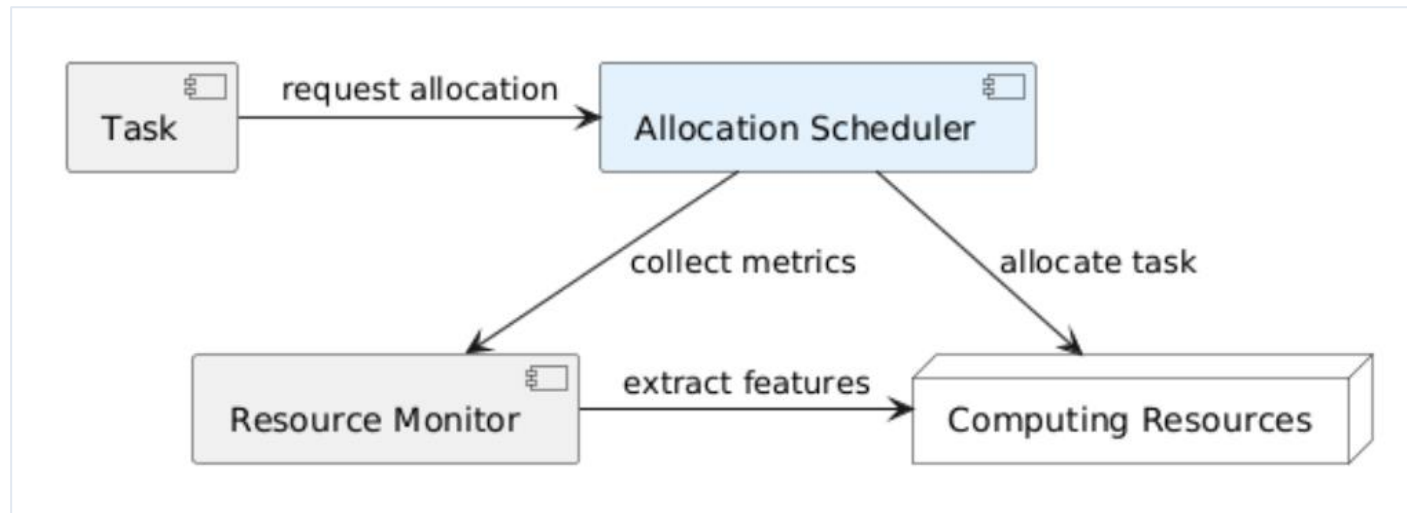


에너지 특징 기반의 서비스 Discovery

## Allocate Resources :: Schedule Resources

### ■ 기본 개념

- 에너지 인식 스케줄링(Energy-aware Scheduling)은 작업을 에너지 효율성 관점에서 컴퓨팅 리소스에 할당하는 것
- 하나 이상의 Monitor Resources 택틱을 사용해 데이터를 수집한다
- 에너지 효율성 관점에서 작업을 컴퓨팅 리소스에 할당한다
  - 에너지 효율이 높거나 에너지 비용이 낮은 리소스에 작업을 할당
  - 사용되는 컴퓨팅 리소스의 수를 최소화하도록 작업을 할당



*Allocation Scheduler 가 Resource Monitor 의 지표로 작업을 할당*

# 3.5

## Integrability Tactics

외부 컴포넌트를 쉽게, 예측 가능한 비용으로 시스템에 통합하기 위한 택틱

- 통합 문제와 통합 난이도(크기·거리)
- Integrability 일반/구체 시나리오
- Limit Dependencies: Encapsulate, Use an Intermediary, Restrict Communication Paths, Adhere to Standards, Abstract Common Services
- Adapt: Discover, Tailor Interface, Configure Behavior
- Coordinate: Orchestrate, Manage Resources



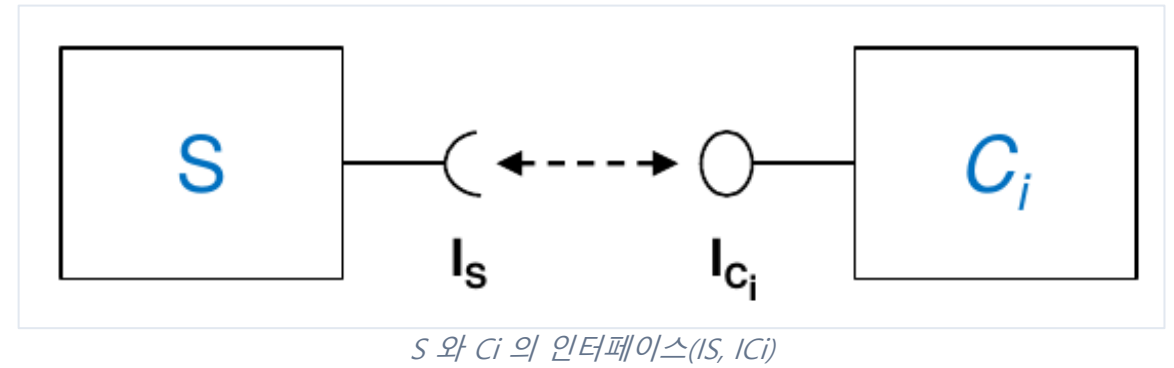
## Integrability(통합용이성) — 통합 문제

### ■ 통합 문제

- 시스템  $S$  가 단위 소프트웨어들의 집합  $\{C_i\}$  를 통합해야 하는 상황
  - $S$  는 통제할 수 있지만,  $\{C_i\}$  는 통제 범위 밖의 대상(예: 외부 벤더가 제공)일 수 있다
  - 실제 통합이 이루어지는 시점은 미래  $\rightarrow \{C_i\}$  에 대한 정보는 불완전하다

### ■ 시스템 $S$ 가 $\{C_i\}$ 를 쉽게 통합하려면?

- $S$  의 Architecture 가  $\{C_i\}$  와의 통합을 미리 예상하고 수용할 수 있도록 설계되어 있다면  $\rightarrow$  통합이 쉽다



**핵심** Integrability 는 '아직 모르는 컴포넌트'와의 결합을 얼마나 값싸게 만들 수 있는가의 문제이다

## 통합 난이도 — 크기와 거리

### ■ 통합 난이도 = S 와 {Ci} 인터페이스 간 크기와 거리의 함수

- 크기(Size): S 와 {Ci} 인터페이스 간의 의존관계 개수
- 거리(Distance): 각 의존관계에 있는 인터페이스 간의 차이를 해결하는 난이도

| 거리의 분류      | 예시                                                                    |
|-------------|-----------------------------------------------------------------------|
| 구문적 거리      | 인터페이스에 정의된 데이터 요소의 타입 및 개수 차이                                         |
| 데이터 의미론적 거리 | IS 측 고도 데이터(미터 단위) vs. I{Ci} 측 고도 데이터(피트 단위)                          |
| 동작적 의미론적 거리 | IS 는 항상 사용 가능 vs. I{Ci} 는 모드에 따라 사용 가능                                |
| 시간적 거리      | IS 는 이벤트 B 발생 이후 즉시 이벤트 A 발생 vs. I{Ci} 는 이벤트 B 발생 후 50ms 이내에 이벤트 A 발생 |
| 리소스 거리      | IS 는 리소스 R1 에 대한 독점 권한 요구 vs. I{Ci} 는 리소스 R1 에 대한 공유 접근 권한 기대         |

# Integrability 일반 시나리오 (1)

| 부분           | 설명                       | 가능한 값                                                   |
|--------------|--------------------------|---------------------------------------------------------|
| 자극원(Source)  | 통합 요청의 주체                | 다음 중 하나 이상: 시스템 이해관계자, 컴포넌트 마켓플레이스, 컴포넌트 벤더             |
| 자극(Stimulus) | 통합 요청의 종류                | 새로운 컴포넌트 추가 / 기존 컴포넌트의 신규 버전 통합 / 기존 컴포넌트들을 새로운 방법으로 통합 |
| 대상(Artifact) | 시스템의 어떤 부분이 통합에 영향을 받는가? | 전체 시스템 / 특정 컴포넌트 집합 / 컴포넌트 메타데이터 / 컴포넌트 환경설정            |

## Integrability 일반 시나리오 (2)

| 부분                      | 설명                    | 가능한 값                                                                                       |
|-------------------------|-----------------------|---------------------------------------------------------------------------------------------|
| 환경(Environment)         | 어떤 상태에서 통합 요청이 발생하는가? | 개발 / 통합 / 배포 / 런타임                                                                          |
| 응답(Response)            | 통합 요청에 대한 반응          | 다음 중 하나 이상: 변경이 (완료, 통합, 테스트, 배포)됐다 / 컴포넌트들이 새로운 환경 설정에서 올바르게 정보를 교환하고, 어떤 리소스 제한도 위반하지 않는다 |
| 응답 측정(Response Measure) | 응답을 어떻게 측정하는가?        | 다음 중 하나 이상의 관점에서 비용을 측정: 변경된 컴포넌트 수 / 변경된 코드의 퍼센트 / 변경된 코드 줄 수 / 노력 / 돈 / 실제 걸린 시간          |

## 구체적인 Integrability 시나리오

### ■ 시나리오 예

- "새로운 데이터 필터링 컴포넌트를 컴포넌트 마켓플레이스에서 구매할 수 있게 됐으며, 해당 컴포넌트를 한 달 안에 시스템에 통합하고 배포해야 한다. 이때 1MM 이상의 노력이 들어가는 안 된다"

| 부분              | 값                            |
|-----------------|------------------------------|
| 자극원(Source)     | 컴포넌트 마켓플레이스                  |
| 자극(Stimulus)    | 새로운 데이터 필터링 컴포넌트 등장          |
| 대상(Artifact)    | 시스템                          |
| 환경(Environment) | 개발                           |
| 응답(Response)    | 새로운 컴포넌트가 통합되고 배포됨           |
| 응답 측정(Measure)  | 한 달 내에 1MM(man-month) 이하의 노력 |

## Integrability Tactics 의 분류

### ■ Limit Dependencies (의존성 제한)

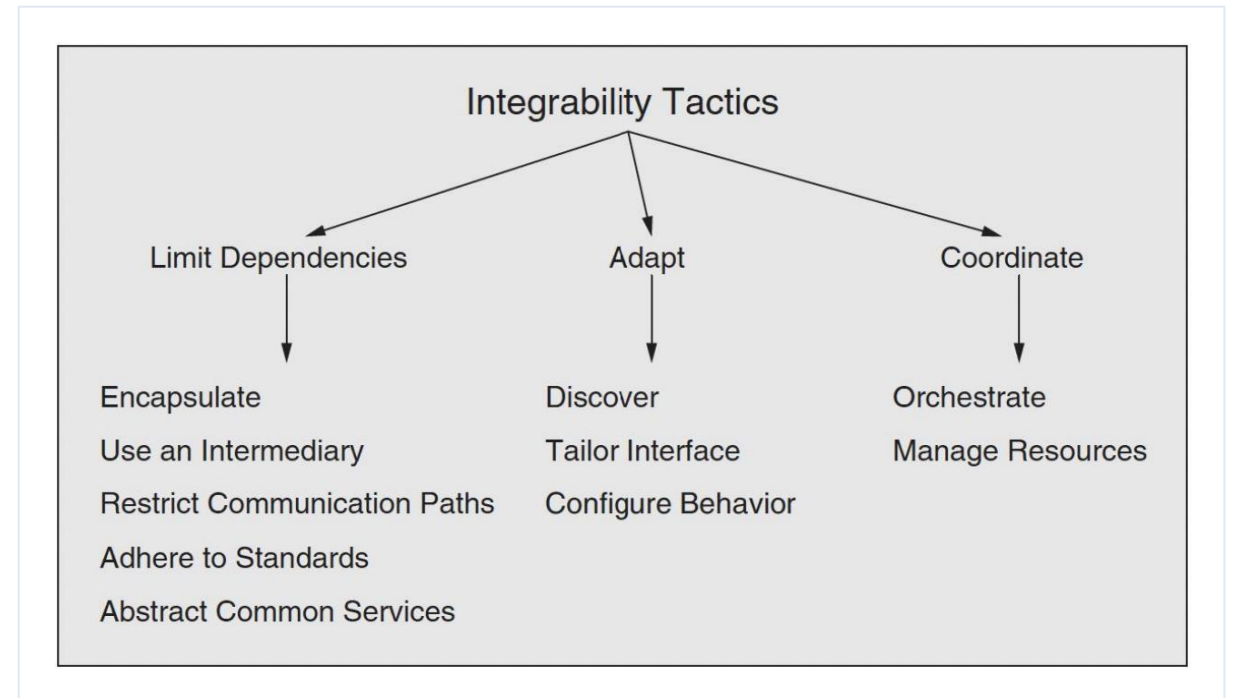
- Encapsulate, Use an Intermediary, Restrict Communication Paths, Adhere to Standards, Abstract Common Services

### ■ Adapt (적응)

- Discover, Tailor Interface, Configure Behavior

### ■ Coordinate (조정)

- Orchestrate, Manage Resources



Integrability Tactics 계층도

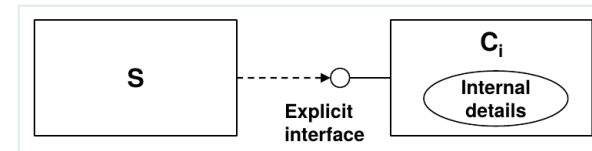
## Limit Dependencies :: Encapsulate

### ■ 기본 개념

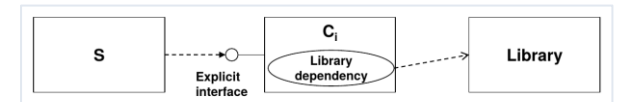
- 캡슐화(Encapsulation)는 어떤 요소에 대한 명시적인 인터페이스를 정의하고, 해당 요소에 대한 모든 접근이 그 인터페이스를 통해서만 이루어지도록 한다

### ■ 의존관계의 수와 거리를 줄인다

- 요소 내부 사항(Internal details)에 대한 의존성을 제거한다
- 특정 통합 작업과 무관한 인터페이스(예: Library dependency)는 숨길 수 있다



내부 사항에 대한 의존성 제거



통합과 무관한 라이브러리 의존성 은닉

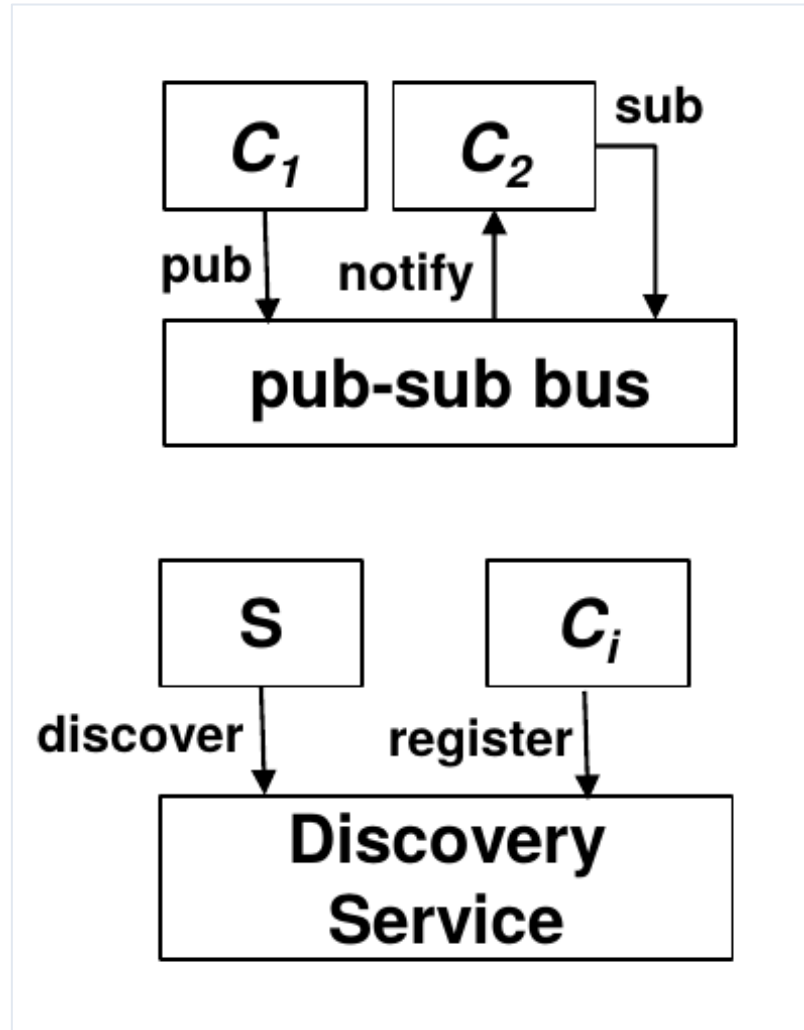
## Limit Dependencies :: Use an Intermediary

### ■ 기본 개념

- 중재자(Intermediary)는 컴포넌트  $C_i$  간의, 혹은 컴포넌트  $C_i$  와 시스템  $S$  간의 의존관계를 깨기 위해 사용된다

### ■ 사례

- 데이터 의존관계를 위한 중재자: 발행-구독 버스(publish-subscribe bus)
  - 데이터 소비자의 데이터 생산자에 대한 의존성을 제거한다
- 서비스 의존관계를 위한 중재자: 동적 서비스 디스커버리(dynamic service discovery)
  - 서비스들은 동적인 조회를 통해 서로의 존재를 인지한다



*pub-sub bus 와 Discovery Service*



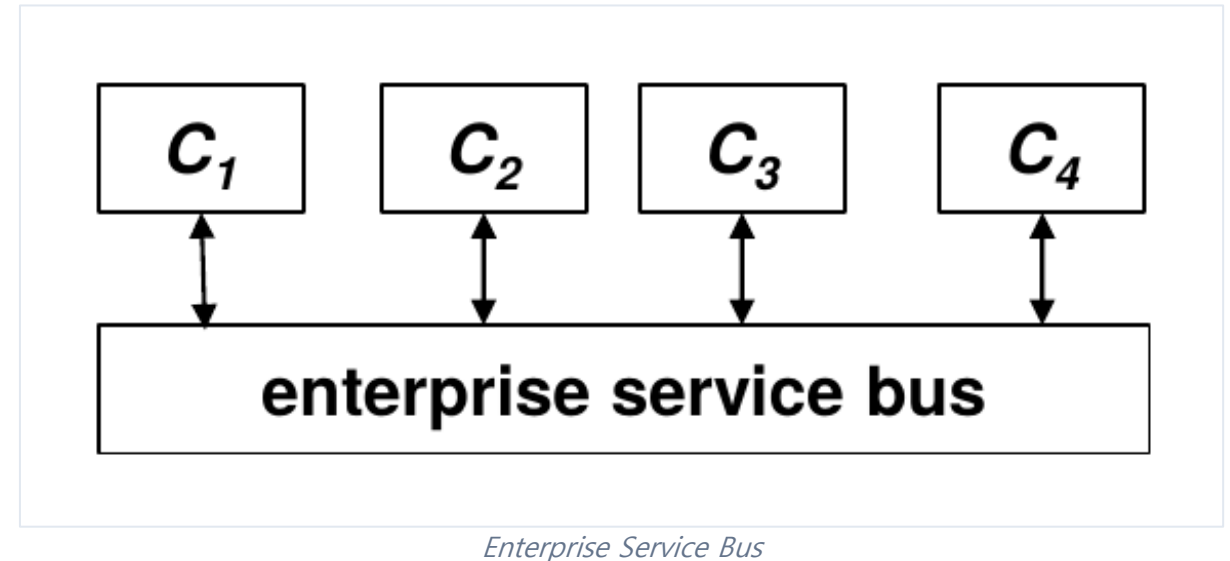
## Limit Dependencies :: Restrict Communication Paths

### ■ 기본 개념

- 어떤 요소가 통신할 수 있는 대상(요소들의 집합)을 제한하는 تاک
- 구체적인 기법
  - 가시성 제한: 인터페이스가 노출된 대상만 통신 가능
  - 권한 부여: 권한이 있는 요소들만 접근 가능

### ■ 사례 — Enterprise Service Bus

- SOA 에서 모든 요청에 대한 라우팅, 중재, 프로토콜 변환 등을 수행한다
- 컴포넌트들은 서로 직접 통신하지 않고 ESB 를 통해서만 통신한다



## Limit Dependencies :: Adhere to Standards

### ■ 기본 개념

- 표준(Standard)은 다양한 플랫폼과 벤더에 걸쳐 효과적인 통합을 가능하게 하는 핵심 تاکтика이다
- 표준이 무엇을 규정하는지에 따라, 표준 준수 تاکтика는 구문적 거리, 데이터 의미론적 거리, 동작적 거리, 시간적 거리 등 다양한 차원의 거리를 해결할 수 있다

### ■ 표준의 효과성을 결정하는 요소

- 해당 표준이 어떤 측면의 차이점을 다루는가
- 향후 컴포넌트 공급사들이 해당 표준을 준수할 것인가

**핵심** 표준은 아직 만나지 않은 컴포넌트와의 거리를 미리 줄여 두는 방법이다

## Limit Dependencies :: Abstract Common Services

### ■ 기본 개념

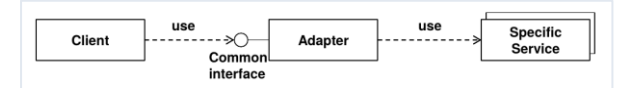
- 유사한 서비스를 제공하는 두 요소를 공통의 추상화(인터페이스) 뒤에 숨겨, 범용적인 서비스로 통합하는 택틱

### ■ 구체적인 방법

- 공통 인터페이스의 구현: Client 는 Common Interface 만 사용하고, 각 구현체가 이를 실현 (realize)한다
- 공통 서비스 요청을 구체적인 서비스 요청으로 변환하는 중재자(예: Adapter)
  - 특정 요소들 간의 구문적·의미론적 차이점을 정규화할 수 있다



공통 인터페이스의 구현



Adapter 를 통한 변환

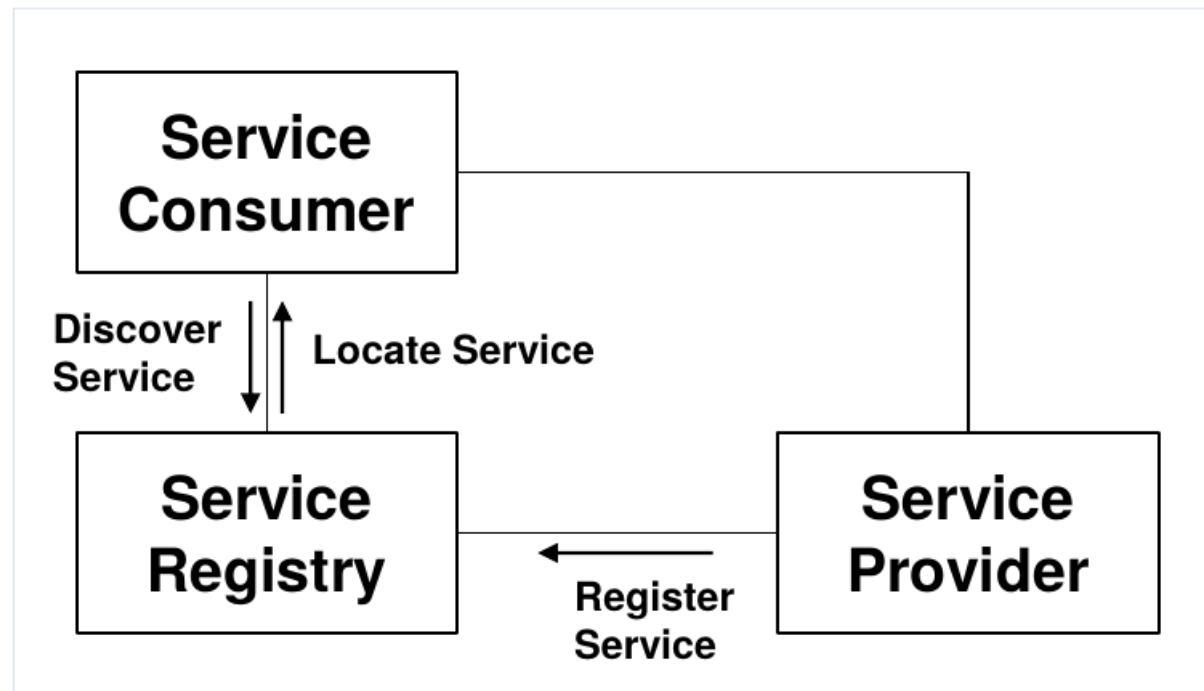
## Adapt :: Discover

### ■ 기본 개념

- 런타임에 필요한 서비스 제공자(Service Provider)나 데이터 소스를 찾아내고 연결하는 택틱

### ■ 작동 원리

- 등록(Registration): 서비스 제공자가 시작될 때 자신의 주소(IP, Port), 서비스 이름, 상태 등을 Service Registry에 등록한다
- 조회(Lookup): 클라이언트는 서비스를 사용하기 전에 레지스트리에 서비스 이름을 질의한다
- 응답(Resolution): 레지스트리는 사용 가능한 인스턴스의 네트워크 주소를 응답한다
- 호출(Invocation): 클라이언트는 응답받은 주소로 서비스 인스턴스를 직접 호출한다



*Service Registry 를 통한 Discovery*

## Adapt :: Discover — 통합용이성 측면의 강점

### ■ 위치 투명성 (Location Transparency)

- 서비스 제공자의 물리적 위치가 바뀌거나 Scale In/Out 으로 인스턴스가 추가·삭제되어도, 요청자는 동적으로 주소를 받아 쓰므로 코드를 수정할 필요가 없다

### ■ 복원력 (Resilience)

- 레지스트리가 장애가 발생한 인스턴스를 목록에서 제외하면, 요청자는 자동으로 정상 인스턴스만 찾아 호출하므로 시스템의 복원력이 높아진다

### ■ 유연한 통합

- 새 서비스 인스턴스를 추가하거나 기존 인스턴스를 교체하는 것이 매우 쉽고 빠르기 때문에 시스템 통합과 확장이 용이하다

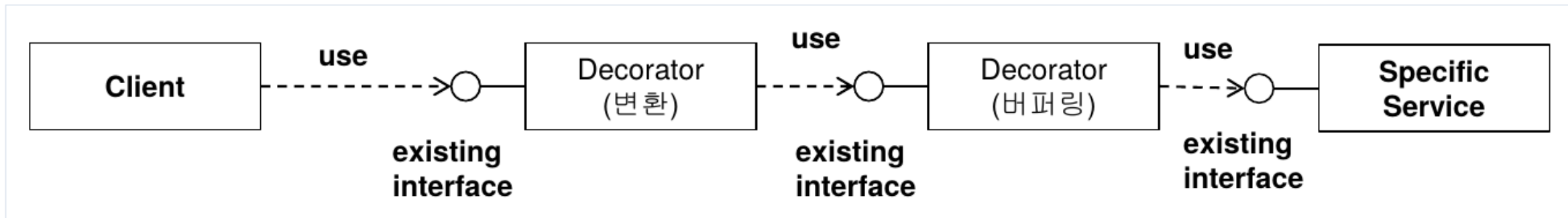
## Adapt :: Tailor Interface

### ■ 기본 개념

- 통합하려는 시스템 간의 차이(Discrepancy)를 줄이기 위해, API 나 구현을 변경하지 않고도 기존 인터페이스에 기능을 추가하거나 기존 기능을 감추는 택틱

### ■ 구체적인 사례 — Decorator 패턴

- 기존 컴포넌트의 인터페이스를 바꾸지 않고, 데이터 포맷 변환(구문적 차이)과 버퍼링(시간적 차이) 기능을 Decorator 로 구현해 원활한 통합을 지원한다



Decorator 를 이용한 인터페이스 맞춤

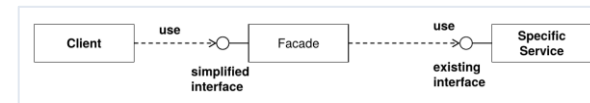
## Adapt :: Tailor Interface — Façade 와 API Gateway

### ■ Façade 패턴

- 제공되는 서비스가 너무 많은 기능을 포함하고 있을 때, 클라이언트가 실제로 필요한 기능만 노출하도록 인터페이스를 간소화한다

### ■ API Gateway

- 불필요한 필드를 제거하고 필요한 데이터만 묶어서 전달한다



*Façade: simplified interface 제공*



*API Gateway: 필요한 데이터만 전달*

## Adapt :: Tailor Interface — 통합용이성 측면의 강점

### ■ 통합 노력 감소 (Reduced Integration Effort)

- 클라이언트가 서비스 제공자의 복잡하거나 이질적인 인터페이스에 억지로 맞춰 코드를 수정할 필요가 없다
- 가장 익숙하고 필요한 형태로 인터페이스가 제공되므로 통합 개발 시간이 단축된다

### ■ 사용 용이성 증대 (Increased Usability)

- 인터페이스가 단순해지면 클라이언트 개발자가 서비스를 오용하거나 잘못 통합할 위험이 줄어든다

### ■ 기존 시스템 보호

- 인터페이스 조정 계층이 추가되므로, 서비스 제공자는 핵심 비즈니스 로직이나 레거시 시스템의 인터페이스를 외부 요구에 맞춰 자주 변경할 필요가 없다



## Adapt :: Configure Behavior

### ■ 기본 개념

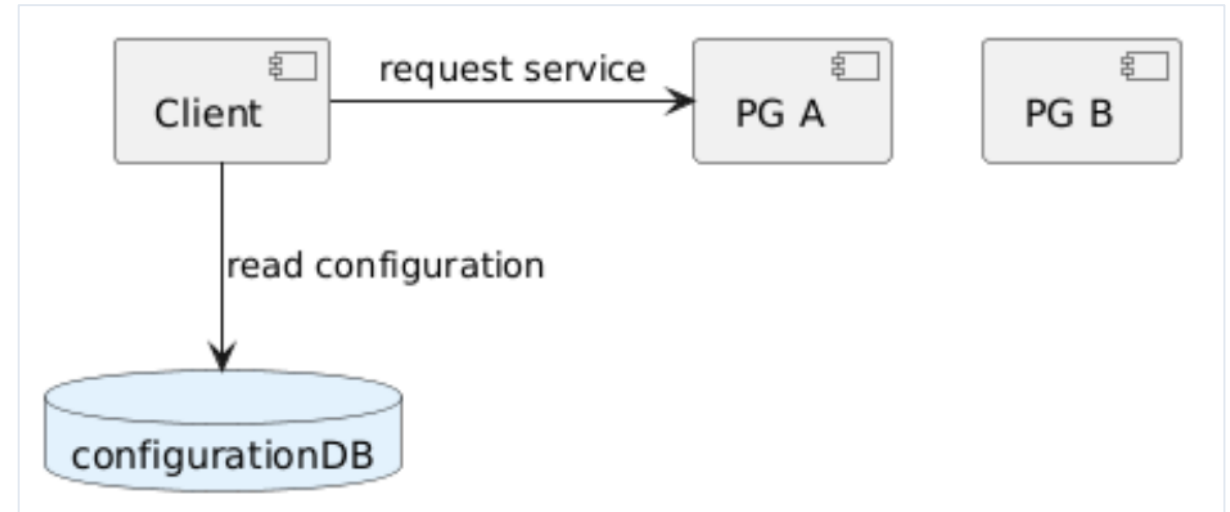
- 시스템 S 의 일부로 환경 설정 가능한 동작을 포함시켜, S 가 더 넓은 범위의 잠재적인 C 들을 지원할 수 있게 하는 택틱

### ■ 작동 방식

- 외부화(Externalization): 통신 프로토콜, 타임아웃, 자격 증명, 서비스 엔드포인트 URL 등 통합에 필수적인 정보를 XML/JSON/YAML 파일, DB, 환경 변수 등 외부 설정에 저장한다
- 런타임 로딩(Runtime Loading): 시스템은 시작 또는 런타임 중에 이 외부 설정을 읽어 자신의 행동 방식을 결정한다

### ■ 사례

- 클라이언트가 두 결제 서비스(PG A, PG B) 중 하나와 통합되어야 할 때, configurationDB 의 설정을 읽어 어느 PG 를 호출할지 결정한다



설정에 따라 PG A / PG B 를 선택

## Coordinate :: Orchestrate

### ■ 기본 개념

- 시스템 S 와 컴포넌트 {Ci} 간 의존관계의 수를 줄이고, {Ci} 간의 명시적인 의존관계를 제거하여 중앙화하는 تاک

### ■ 효과

- S 는 Orchestrator 하나만 호출하고, C1·C2 는 서로를 몰라도 된다
- 호출 순서와 데이터 변환 책임이 Orchestrator 한 곳에 모인다



Orchestrator 도입 전후의 의존관계

## Coordinate :: Orchestrate — 작동 원리

### ■ 작동 원리

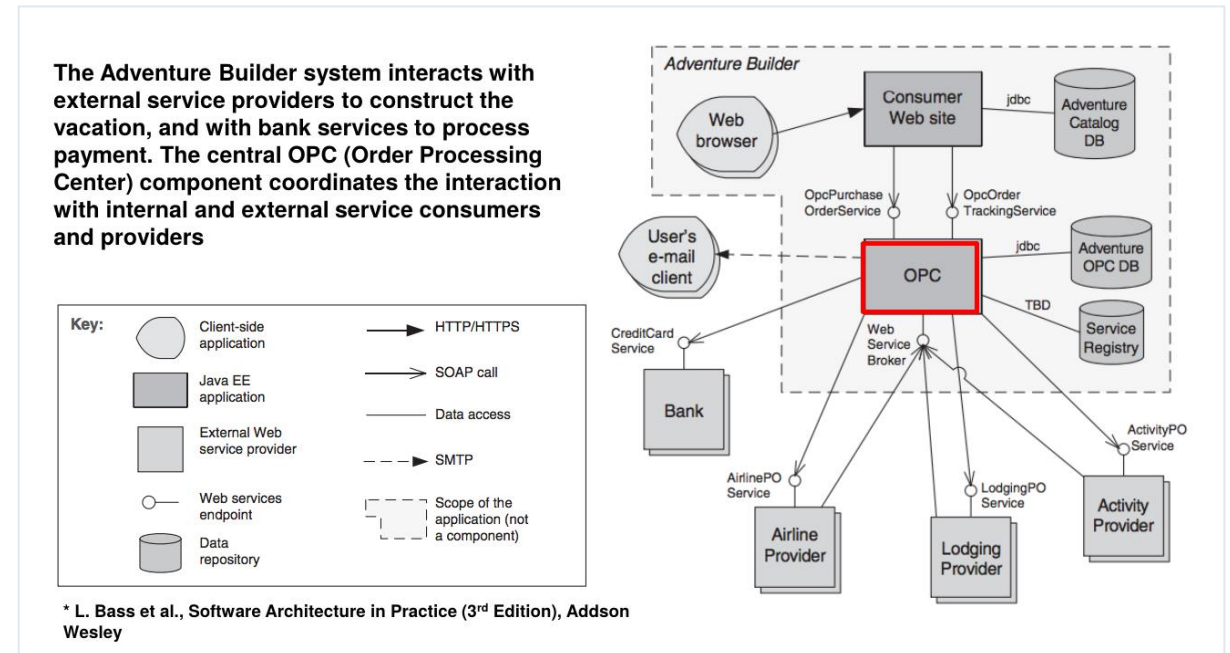
- 단일 요청 수신: 시스템 S 가 Orchestrator 에게 복합 작업(예: "주문 처리")을 요청한다
- 흐름 제어(Flow Control): Orchestrator 는 미리 정의된 비즈니스 프로세스(Workflow)에 따라 서비스 호출 순서를 결정한다
- 순차적 호출 및 데이터 변환: C1 을 호출하고 그 결과를 필요 시 변환하여 C2 의 입력으로 전달한다
- 최종 응답: 모든 호출이 끝나면 결과를 종합하여 S 에게 최종 응답을 제공한다

**핵심** Orchestrator 가 워크플로우를 알고 있으므로, 개별 컴포넌트는 다른 컴포넌트를 알 필요가 없다

## Coordinate :: Orchestrate — 사례

### ■ Adventure Builder 시스템

- 휴가 상품을 구성하기 위해 외부 서비스 제공자(항공, 숙박, 액티비티)와 상호작용하고, 결제를 위해 은행 서비스와 상호작용한다
- 중앙의 OPC(Order Processing Center) 컴포넌트가 내부·외부 서비스 소비자와 제공자 간의 상호작용을 조정(orchestrate)한다
- 출처: L. Bass et al., Software Architecture in Practice (3rd Ed.)



Adventure Builder 의 OPC 가 Orchestrator 역할

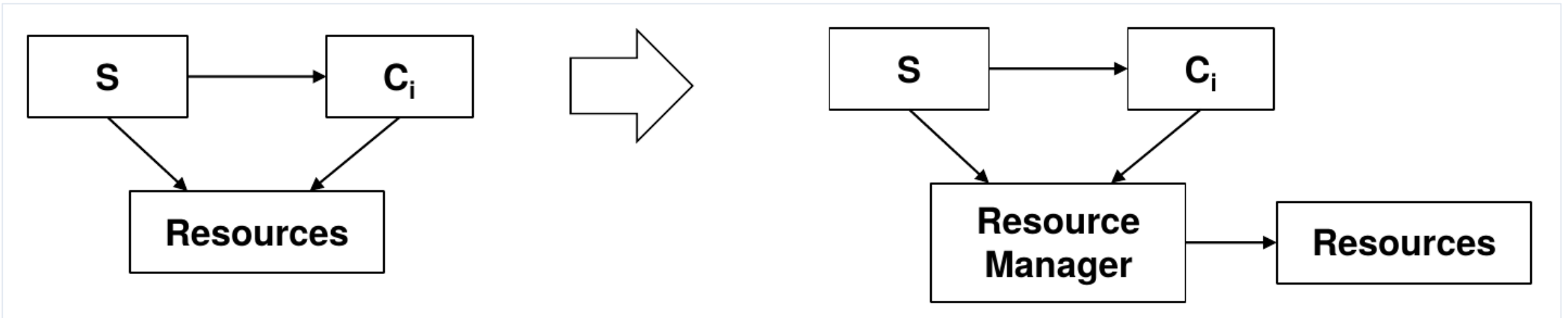
## Coordinate :: Manage Resources

### ■ 기본 개념

- 시스템 S 와 컴포넌트 C 사이의 리소스 거리를 줄이고, 리소스 요구사항을 명확하게 노출하며 해당 리소스의 사용을 관리하는택틱

### ■ 리소스 관리자(Resource Manager)

- 컴퓨팅 리소스에 대한 접근을 관장하는 특화된 형태의 중개자
- S 와 다른 컴포넌트 {C<sub>i</sub>} 가 통합될 때 필요한 자원의 획득·사용·해제 과정을 체계적으로 관리한다



Resource Manager 도입 전후

## Coordinate :: Manage Resources — 풀링(Pooling) 사례

### ■ 풀(Pool)은 대표적인 Resource Manager

- 비싼 자원을 미리 만들어 두고 빌려주고 돌려받는 방식으로, 자원의 획득·사용·해제를 한 곳에서 관리한다

| 풀링 대상                   | 설명                                      | 효과                                                             |
|-------------------------|-----------------------------------------|----------------------------------------------------------------|
| 커넥션 풀 (Connection Pool) | DB 또는 외부 서비스와의 연결을 미리 맺어 놓고 재활용         | 요청마다 발생하는 연결 설정·해제 오버헤드를 제거하여 통합 속도 향상                         |
| 객체 풀 (Object Pool)      | 생성 비용이 높은 객체들을 미리 만들어 저장해 두고 필요할 때 빌려 씀 | 메모리 할당 및 가비지 컬렉션 부담 감소                                         |
| 스레드 풀 (Thread Pool)     | 작업 처리에 사용할 스레드의 개수를 제한하고 관리             | 스레드 생성 오버헤드를 줄이고, 시스템이 처리할 수 있는 동시성(Concurrency)을 통제하여 오버로드 방지 |

# 3.6

## Modifiability Tactics

변경을 더 적은 비용과 위험으로 수행하기 위한 تاک

- Modifiability 의 관심사와 일반/구체 시나리오
- Coupling 과 Cohesion
- Increase Cohesion: Split Module, Redistribute Responsibilities
- Reduce Coupling: Encapsulate, Use an Intermediary, Restrict Dependencies, Abstract Common Services
- Defer Binding: Compile/Build, Startup, Runtime Binding

## Modifiability(변경용이성)는 '변경'에 관한 것

### ■ Modifiability 의 관심사

- 변경(change)을 만드는 데 드는 비용과 위험이 핵심 관심사이다
- 무엇이 변할 수 있는가? (What can change?)
- 그 변경이 일어날 가능성은 얼마나 되는가?
- 변경은 언제, 누가 만드는가?
- 변경 비용은 얼마인가?
  - 시스템을 더 변경하기 쉽게 만드는 메커니즘을 도입하는 비용
  - 그 메커니즘을 이용해 실제 변경을 수행하는 비용

**핵심** Modifiability 태틱은 '변경 메커니즘 도입 비용'과 '변경 수행 비용'의 합을 줄이는 것이 목표이다



## Modifiability(Adaptability) 일반 시나리오

| 부분                      | 설명           | 가능한 값                                                      |
|-------------------------|--------------|------------------------------------------------------------|
| 자극원(Source)             | 누가 변경을 만드는가  | 개발자, 최종 사용자, 시스템 관리자                                       |
| 자극(Stimulus)            | 어떤 변경을 만드는가  | 기능, 품질 속성, 용량(capacity), 기술의 추가/삭제/수정                      |
| 대상(Artifact)            | 무엇이 변경되는가    | 컴포넌트나 모듈 / 시스템의 플랫폼 / 사용자 인터페이스 / 시스템의 환경 / 상호 운영되는 다른 시스템 |
| 환경(Environment)         | 언제 변경할 수 있는가 | 설계 시점, 컴파일 시점, 빌드 시점, 초기화 시점, 런타임                          |
| 응답(Response)            | 시스템의 반응      | 변경을 만든다 / 변경을 테스트한다 / 변경을 배포한다                             |
| 응답 측정(Response Measure) | 어떻게 측정하는가    | 영향받는 요소의 수·크기·복잡도 / 노력 / 달력 시간 / 비용(돈)                     |

## 구체적인 Modifiability 시나리오

### ■ 시나리오 예

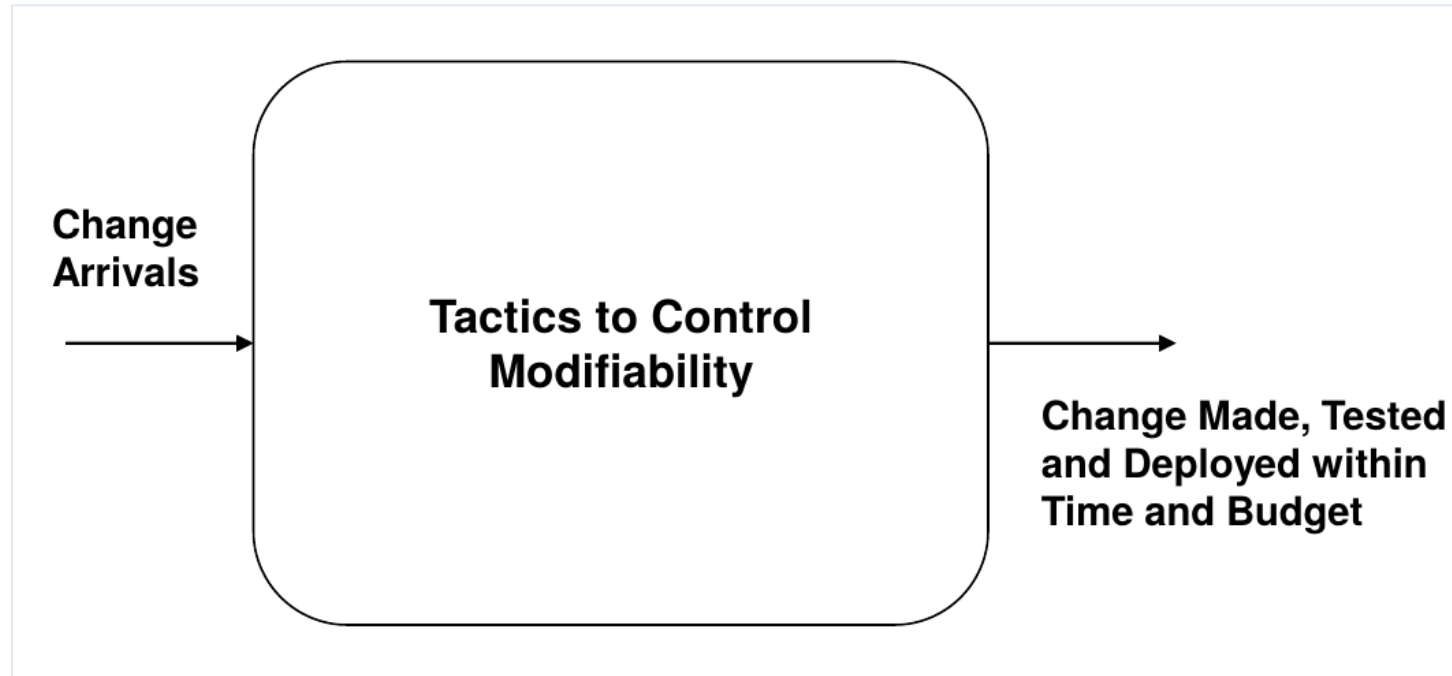
- "개발자가 화면의 배경색을 파란색으로 바꾸기 위해 사용자 인터페이스를 변경하려 한다. 이 변경은 설계 시점에 코드에 가해진다. 변경과 테스트에 3시간 미만이 걸리고, 동작에 어떤 부작용도 생기지 않는다"

| 부분              | 값                     |
|-----------------|-----------------------|
| 자극원(Source)     | 개발자                   |
| 자극(Stimulus)    | 변경을 원함(화면 배경색을 파란색으로) |
| 대상(Artifact)    | 사용자 인터페이스             |
| 환경(Environment) | 설계 시점                 |
| 응답(Response)    | 부작용 없이 변경이 이루어진다      |
| 응답 측정(Measure)  | 3시간 이내                |

## Modifiability 태틱의 목표

### ■ Modifiability 태틱의 목표

- 변경을 만드는 복잡도와, 변경에 드는 시간과 코드를 통제하는 것
- 변경 요청(Change Arrivals)이 들어오면 → 시간과 예산 안에 변경을 만들고 테스트하고 배포한다



Modifiability 태틱의 목표

## Coupling(결합도)과 Cohesion(응집도)

### ■ Coupling(결합도)

- 모듈은 책임(responsibility)을 가지며, 모듈을 바꾸면 그 책임도 어떤 식으로든 바뀐다
- 한 모듈의 변경이 다른 모듈로 전파될 확률을 측정한다 — 높은 결합도는 Modifiability 의 적이다

### ■ Cohesion(응집도)

- 한 모듈의 책임들이 얼마나 강하게 관련되어 있는지를 측정한다
- 응집도가 높을수록 하나의 변경이 다른 모듈의 여러 책임에 영향을 줄 확률이 낮아진다

**핵심** 결합도는 낮게, 응집도는 높게 — 변경이 번지지 않게 하는 기본 원칙

## Modifiability 를 달성하는 네 가지 접근

### ■ 모듈 크기 줄이기 (Reduce size of a module)

- 시스템이 여러 개의 작은 모듈로 이루어져 있으면, 하나의 큰 모듈일 때보다 변경하기 쉽다

### ■ 응집도 높이기 (Increase cohesion)

- 응집도가 낮은 모듈은 응집도가 충분히 높아질 때까지 책임을 떼어내어 개선한다

### ■ 결합도 낮추기 (Reduce coupling)

- 모듈 간 결합도를 줄이면 한 모듈의 변경이 다른 모듈로 전파되지 않도록 보장하는 데 도움이 된다

### ■ 바인딩 시점 늦추기 (Binding time)

- 시스템을 매우 변경하기 쉽게 만들고 싶다면, 가능한 한 늦게 바인딩한다

# Modifiability Tactics 의 계층도

## ■ Increase Cohesion

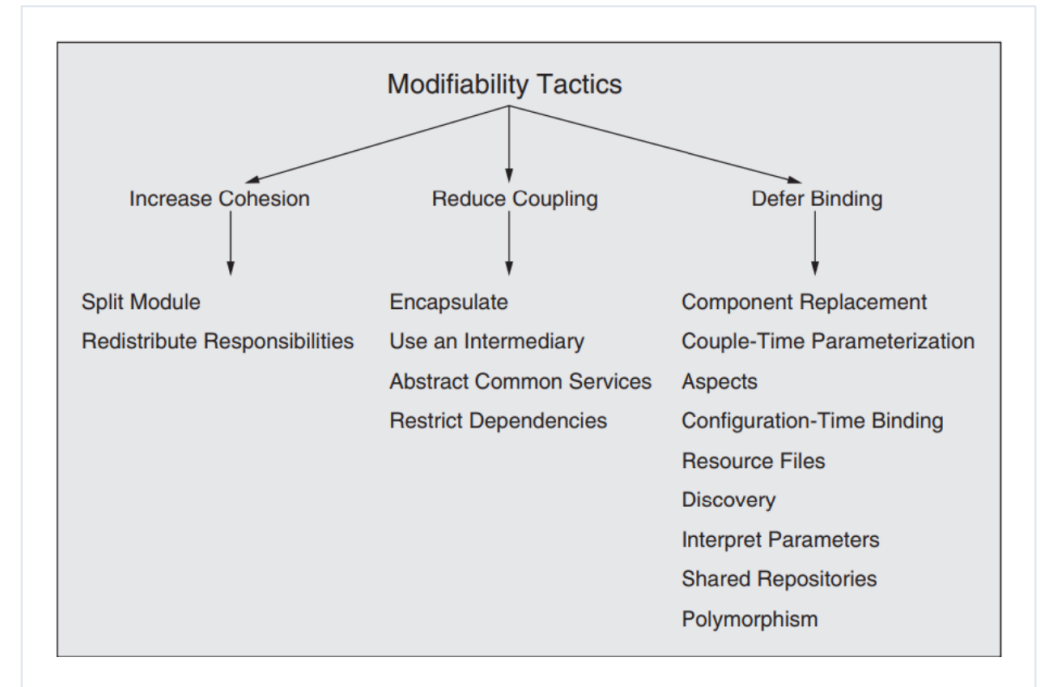
- Split Module, Redistribute Responsibilities

## ■ Reduce Coupling

- Encapsulate, Use an Intermediary, Abstract Common Services, Restrict Dependencies

## ■ Defer Binding

- Component Replacement, Compile-Time Parameterization, Aspects, Configuration-Time Binding, Resource Files, Discovery, Interpret Parameters, Shared Repositories, Polymorphism



Modifiability Tactics 계층도

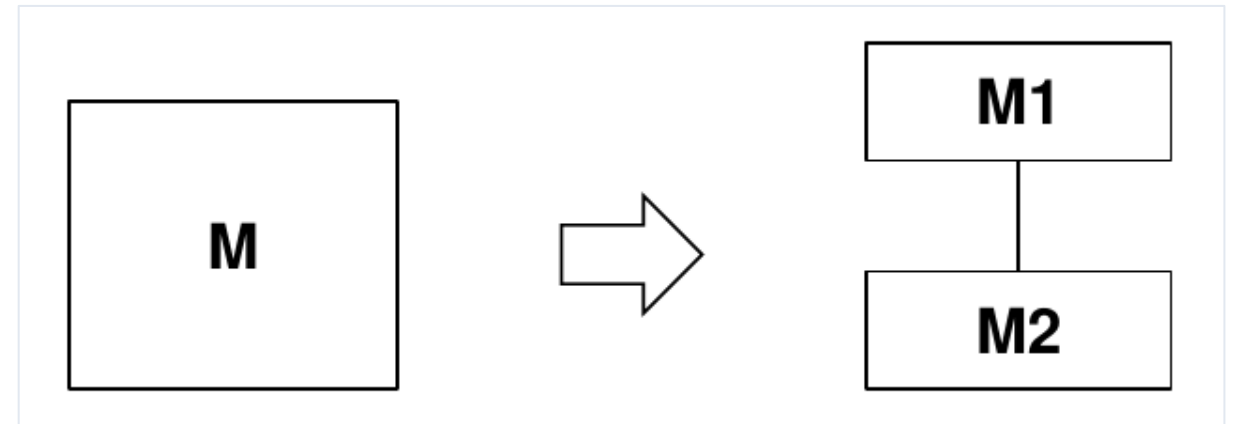
## Increase Cohesion :: Split Module

### ■ 기본 개념

- 변경 대상 모듈이 아주 많은 기능을 포함하고 있다면 변경 비용이 높아지기 쉽다
- 모듈을 여러 개의 작은 모듈로 정제(refine)하면 향후 변경의 평균 비용이 줄어든다

### ■ 단일 책임 원칙 (Single-Responsibility Principle, SRP)

- 프로그램의 모든 모듈, 클래스, 함수는 그 프로그램 기능의 단 한 부분에 대해서만 책임을 져야 한다



모듈 M 을 M1, M2 로 분리

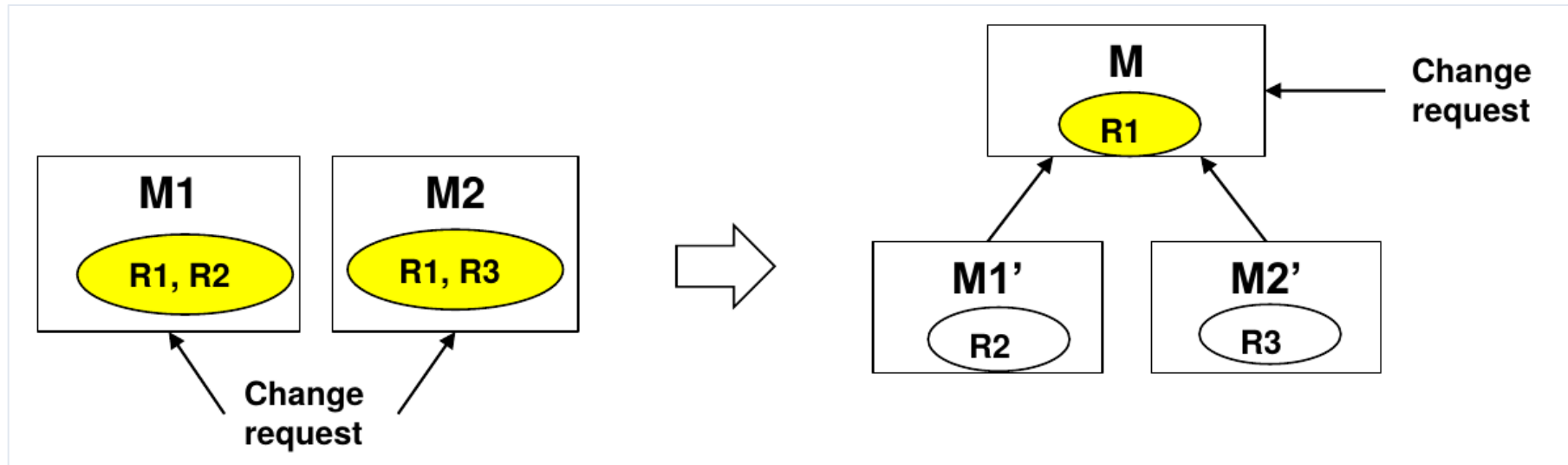
## Increase Cohesion :: Redistribute Responsibilities

### ■ 기본 개념

- 비슷한 책임 A, A', A'' 가 여러 모듈에 흩어져 있다면 한곳에 모아야 한다
- 두 모듈이 (적어도 부분적으로) 서로의 중복이어서 같은 변경에 함께 영향을 받는다면, 공통 책임(과 그 구현 코드)을 모듈에서 분리(factor out)한다

### ■ 그림 설명

- M1(R1, R2)과 M2(R1, R3)가 R1 의 변경에 모두 영향을 받는다 → 공통 책임 R1 을 M 으로 뽑아내면 변경 요청은 M 하나에만 닿는다



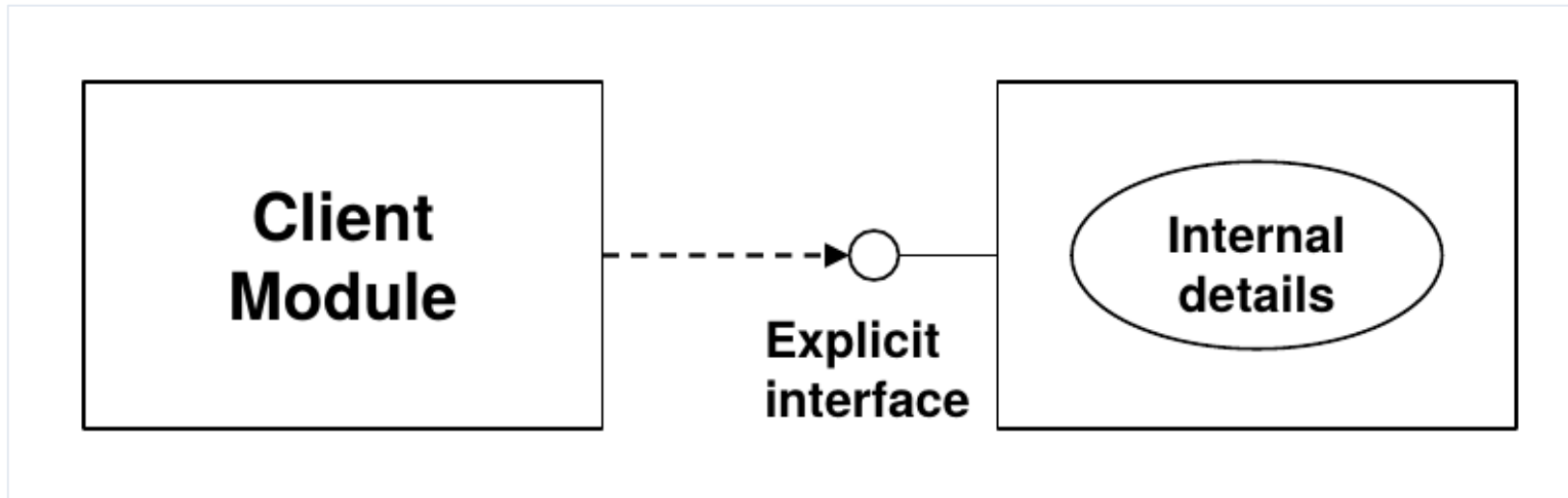
공통 책임 R1 을 별도 모듈 M 으로 분리



## Reduce Coupling :: Encapsulate

### ■ 기본 개념

- 캡슐화(Encapsulation)는 모듈에 명시적인 인터페이스(explicit interface)를 도입한다
  - 이 인터페이스는 모듈의 내부 사항(internal details)을 숨겨야 한다
- 한 모듈의 변경이 다른 모듈로 전파될 확률을 줄인다



*Client Module 은 Explicit interface 만 의존*

## Reduce Coupling :: Use an Intermediary

### ■ 기본 개념

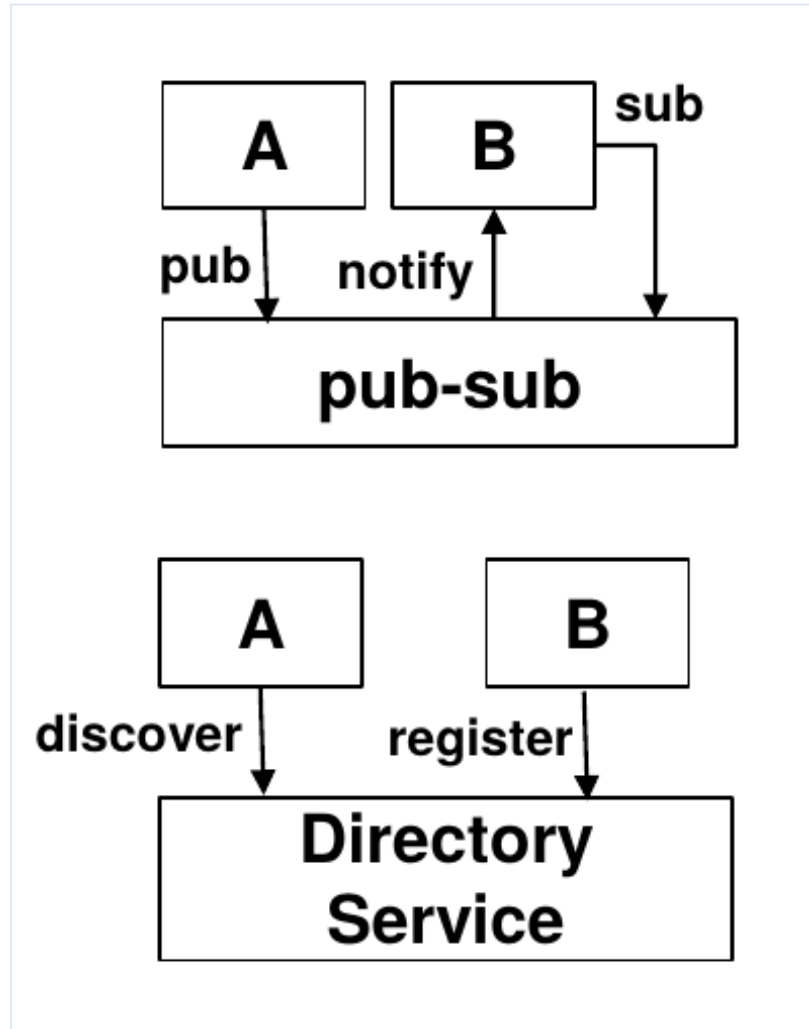
- 책임 A 와 책임 B 사이에 의존관계가 있을 때, 중재자 (Intermediary)를 사용하여 그 의존관계를 끊을 수 있다

### ■ 데이터 의존관계를 위한 중재자

- publish-subscribe 중재자
  - 데이터 생산자가 소비자를 알 필요를 없앤다

### ■ 서비스 의존관계를 위한 중재자

- SOA 의 Directory Service 컴포넌트
  - 서비스들은 동적 조회(dynamic lookup)로 서로를 발견한다



*pub-sub 와 Directory Service 를 통한 의존관계 제거*

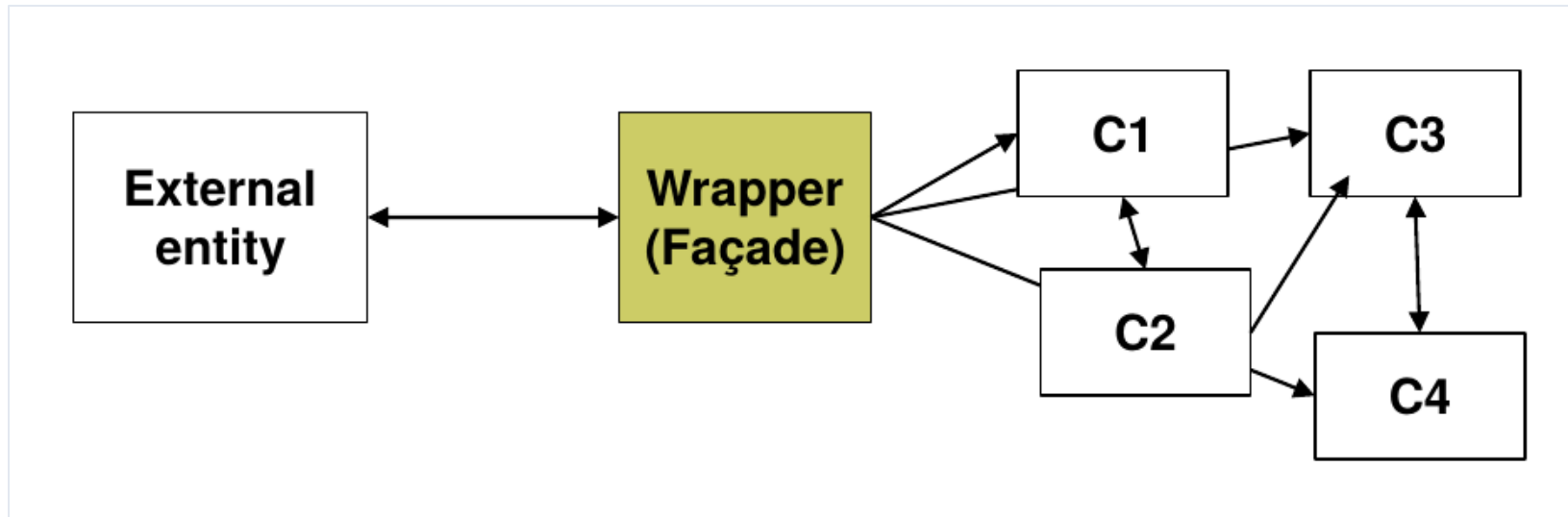
## Reduce Coupling :: Restrict Dependencies

### ■ 기본 개념

- 모듈의 가시성(visibility)이나 접근을 권한이 있는 모듈로만 제한한다

### ■ 예

- Layered Architecture: 한 계층은 자기보다 아래 계층만 사용할 수 있다
- Wrapper(또는 Façade): 외부 요소는 Wrapper 만 볼 수 있고(따라서 Wrapper에만 의존하고), 그 안에 감싸인 내부 기능(C1~C4)은 볼 수 없다



Wrapper(Façade)로 외부 의존을 제한

## Defer Binding Time — 바인딩 시점별 택틱

### ■ 컴파일/빌드 시점 바인딩

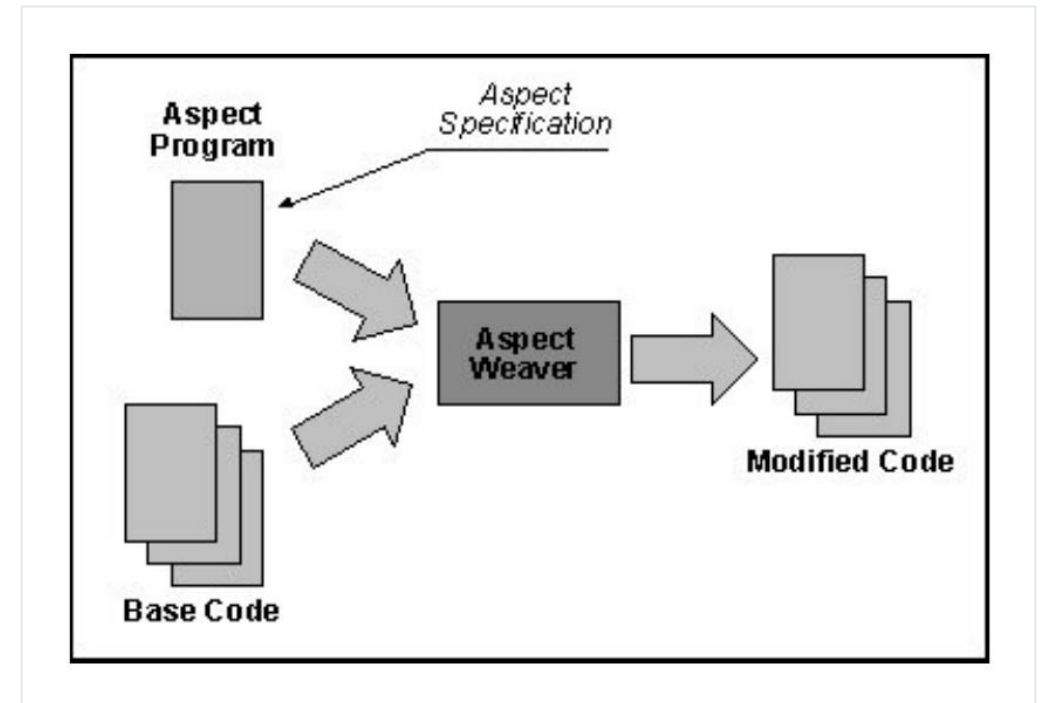
- Component replacement(예: 빌드 스크립트나 makefile 에서 교체)
- Compile-time parameterization(예: 전처리)
- Aspects: Aspect Weaver 가 Base Code 와 Aspect Program 을 엮어 Modified Code 생성

### ■ 시작/초기화 시점 바인딩

- Resource files: 시작 시 파라미터를 설정한다

### ■ 런타임 바인딩

- Service registration and discovery, Publish-subscribe
- Shared repositories, Dynamic plug-ins, Polymorphism, AOP



Aspect Weaver 의 동작

## Modifiability Tactics — Summary

### ■ Modifiability 란

- 변경(change)과, 변경을 만드는 데 드는 시간·비용을 다룬다

### ■ 변경 비용을 줄이는 تاک

- 모듈을 더 작게 만들기(making modules smaller)
- 응집도 높이기(increasing cohesion)
- 결합도 낮추기(reducing coupling)
- 바인딩 늦추기(deferring binding)

### ■ Reduce Coupling 은 대표적인 تاک 범주

- Encapsulate, Use an Intermediary, Restrict Dependencies, Refactor, Abstract Common Services 를 포함한다

**핵심** 작게 나누고, 뭉칠 것은 뭉치고, 결합은 끊고, 결정은 늦춘다

# 3.7

## Performance Tactics

이벤트가 도착했을 때 정해진 시간 안에 응답하도록 만드는 택틱

- Performance 의 정의와 측정 지표
- Performance Scenario
- Control Resource Demand
- Manage Resources

## Performance(성능)란 무엇인가

### ■ Performance 는 시간(time)에 관한 것

- 이벤트가 발생하면 시스템은 제때(in time) 응답해야 한다

### ■ 이벤트 도착 패턴(Arrival pattern)

- Periodic: 일정한 시간 간격으로 이벤트가 도착
- Stochastic: 확률 분포에 따라 이벤트가 도착
- Sporadic: 주기적이지도 확률적이지도 않은 산발적 도착

### ■ 시간 기반 응답 지표

- Latency: 이벤트 도착부터 시스템 응답까지 걸리는 시간
- Jitter: 허용되는 latency 의 변동 폭
- Throughput: 특정 시간 동안 처리할 수 있는 이벤트 수
- Miss rate: 처리하지 못한 이벤트 수 ÷ 전체 이벤트 수

**핵심** Performance 는 '이벤트가 어떤 패턴으로 오는가'와 '얼마나 빨리 응답하는가'로 규정한다

# General Performance Scenario

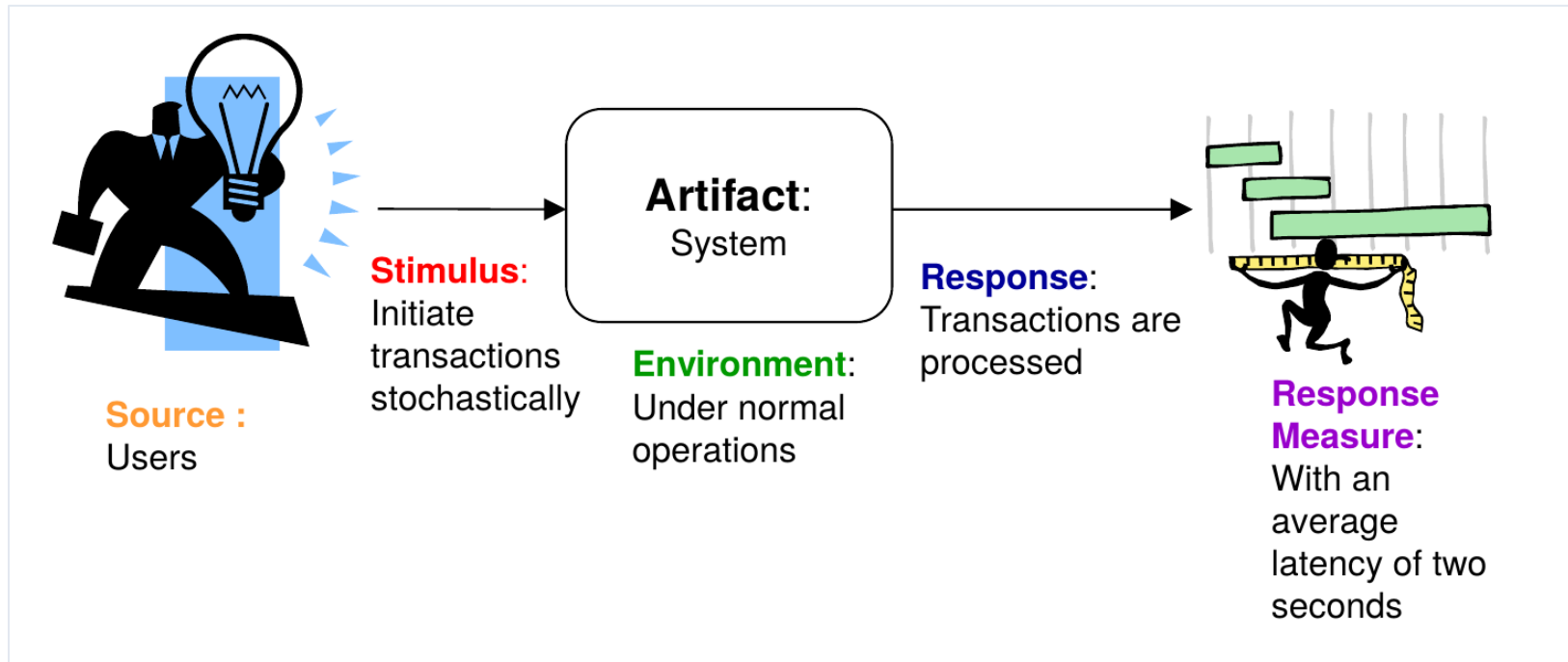
| 부분               | 가능한 값                                                             |
|------------------|-------------------------------------------------------------------|
| Source           | 시스템 내부 또는 외부                                                      |
| Stimulus         | Periodic, Sporadic, Stochastic 이벤트의 도착                            |
| Artifact         | 시스템 전체 / 시스템 내 하나 이상의 Component                                   |
| Environment      | 운영 모드: 정상(Normal), 비상(Emergency), 최대 부하(Peak load), 과부하(Overload) |
| Response         | 이벤트를 처리한다 / 서비스 수준(level of service)을 변경한다                        |
| Response Measure | Latency, Deadline, Jitter, Throughput, Miss rate                  |



## Concrete Performance Scenario 예

### ■ 시나리오

- "사용자가 정상 운영 중에 분당 1,000건의 transaction 을 stochastic 하게 발생시키면, 시스템은 평균 latency 2초로 transaction 을 처리해야 한다"



Source: Users / Stimulus: transaction 발생 / Artifact: System / Environment: 정상 운영 / Response: 처리 / Response Measure: 평균 latency 2초

## 응답 시간을 결정하는 두 가지 요소

### ■ Processing time (처리 시간)

- 처리는 자원(resource)을 소비하고, 자원 사용에는 시간이 걸린다
- HW 자원: CPU, 데이터 저장소, 네트워크 대역폭, 메모리
- SW 자원: 설계 중인 시스템이 정의하는 개체(buffer, thread, lock 등)

### ■ Blocked time (대기 시간)

- 계산이 진행하지 못하고 막혀 있는 시간
- 필요한 자원을 두고 경쟁(contention)이 일어날 때
- 자원을 사용할 수 없을 때(unavailability)
- 의존하는 다른 component 의 결과가 아직 나오지 않았을 때

**핵심** 응답 시간 = 처리 시간 + 대기 시간. 두 값을 줄이는 것이 Performance 택틱의 목표이다

## Performance 태틱의 두 가지 접근

### ■ Control Resource Demand (수요 측)

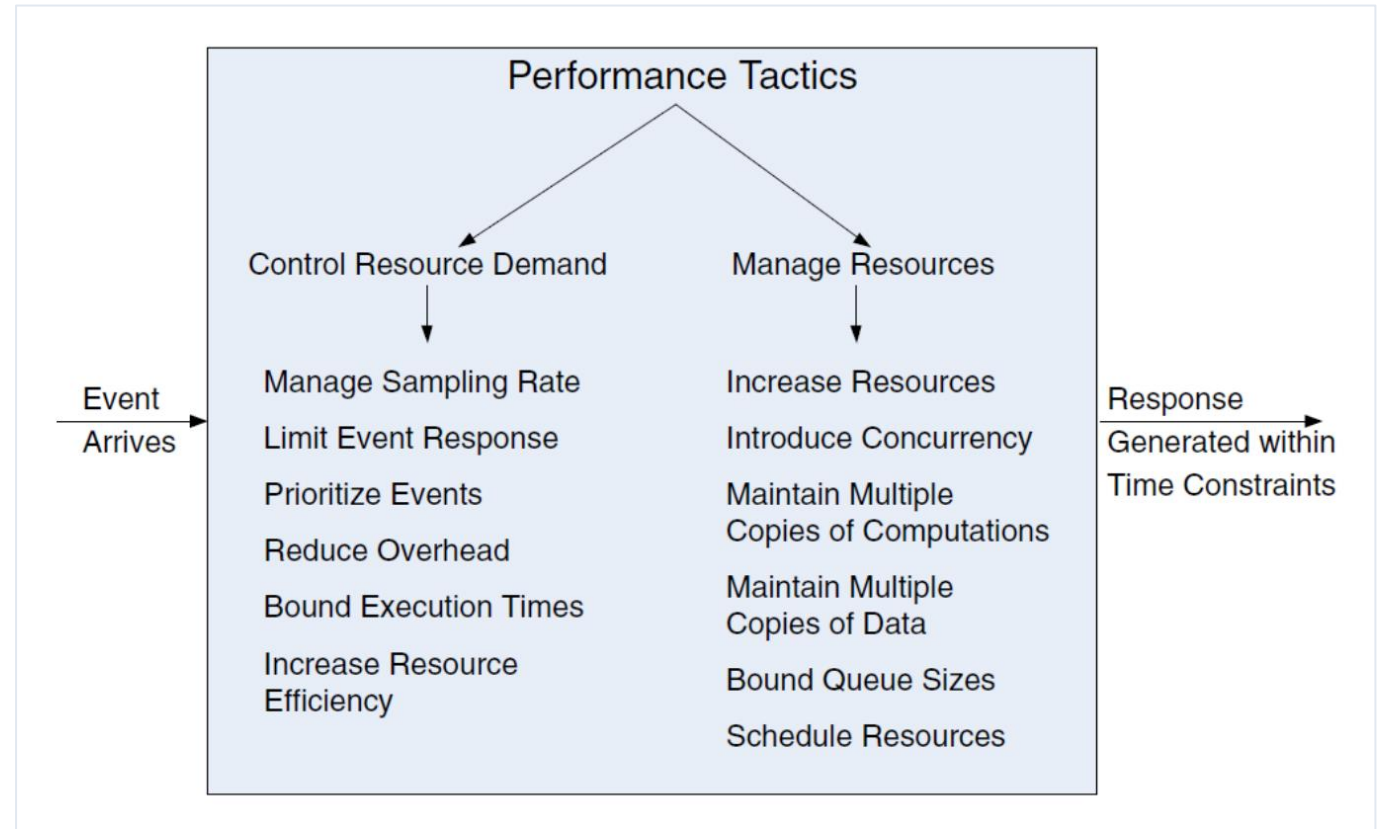
- 이벤트를 처리할 자원에 가해지는 수요 (demand)를 줄인다

### ■ Manage Resources (공급/응답 측)

- 수요를 처리하는 자원을 더 효과적으로 관리한다

### ■ 결과

- 이벤트가 도착하면(Event Arrives) 시간 제약 안에 응답이 만들어진다(Response Generated within Time Constraints)



Performance 태틱 계층도

## Manage Sampling Rate / Limit Event Response

### ■ Manage sampling rate (샘플링 주기 관리)

- 환경 데이터 stream 을 수집하는 샘플링 빈도를 줄일 수 있다면 수요를 줄일 수 있다
- 작업 부하나 자원 가용성 같은 요인에 따라 샘플링 빈도를 조절한다
- 예측 가능한 수준의 latency 를 유지한다
- 연속적인 고빈도 처리가 꼭 필요하지 않은 경우 정확도와 효율 사이의 균형을 잡는다

### ■ Limit event response (이벤트 응답 제한)

- 도착한 이벤트가 최대 queue 크기에 이르거나 프로세서 사용률이 경고 수준을 넘으면, 정해진 최대 비율까지만 이벤트를 처리해 예측 가능한 처리를 보장한다
- 버려진(dropped) 이벤트를 다루는 정책
  - 버려진 이벤트를 기록(logging)한다
  - 그냥 무시한다
  - 사용자나 관리자에게 알린다

## Control Resource Demand **택틱 (2)**

### ■ **Prioritize events (이벤트 우선순위)**

- 모든 이벤트가 똑같이 중요하지 않다면, 서비스의 중요도에 따라 이벤트 순위를 매기는 우선순위 체계를 도입한다

### ■ **Reduce overhead (오버헤드 감소)**

- 중개자(intermediary)나 component 체인을 제거한다
- 협력하는 component 를 같은 프로세서에 두어 네트워크 통신 지연을 피한다
- 비효율적으로 변한 자원을 주기적으로 정리한다 (예: hash table 재초기화)

### ■ **Bound execution times (실행 시간 제한)**

- 이벤트 하나에 응답하는 데 쓰는 실행 시간에 상한을 둔다
- 반복적·데이터 의존적 알고리즘은 반복 횟수를 제한하는 방법을 쓴다
- 대가: 계산의 정확도가 떨어진다

### ■ **Increase resource efficiency (자원 효율 향상)**

- 핵심 영역에서 사용하는 알고리즘을 개선하면 latency 가 줄어든다

## Increase Resources / Introduce Concurrency / Schedule Resources

### ■ Increase resources (자원 증가)

- 더 빠른 프로세서, 프로세서 추가, 메모리 추가, 더 빠른 네트워크는 모두 latency 를 줄일 수 있다
- 자원을 선택할 때는 보통 비용(cost)을 고려해야 한다

### ■ Introduce concurrency (동시성 도입)

- 서로 다른 이벤트 stream 을 서로 다른 thread 에서 처리한다
- 서로 다른 활동 집합을 처리할 thread 를 추가로 만든다
- 적절한 scheduling 정책을 함께 사용해야 한다

### ■ Schedule resources (자원 스케줄링)

- 자원을 두고 경쟁이 있으면 자원을 스케줄링해야 한다 (프로세서, buffer, 네트워크 모두)
- First-in/First-out (FIFO)
- Fixed-priority: Rate monotonic — 주기가 짧은 프로세스에 높은 우선순위
- Dynamic priority: Earliest deadline first
- Static scheduling: cyclic executive 스케줄

## Maintain Multiple Copies of Computations / Data

### ■ Maintain multiple copies of computations (계산 복제)

- 계산의 복제본을 여러 서버에 배치하면, 한 서버에서 모든 계산을 할 때 생기는 경쟁을 줄일 수 있다
- Load balancer 가 새 작업을 사용 가능한 복제 서버 중 하나에 할당한다
  - 할당 기준: round-robin, 가장 한가한 서버에 다음 요청 배정

### ■ Maintain multiple copies of data (데이터 복제)

- Caching: 접근 속도가 다른 저장소에 데이터 사본(한 쪽이 다른 쪽의 부분집합일 수 있음)을 둔다
  - 예: 마이크로프로세서 cache, 웹 브라우저 cache
  - cache 할 데이터 선택: 최근 요청된 데이터, 사용자 행동 패턴에 기반한 예측
- Data replication: 여러 동시 접근으로 인한 경쟁을 줄이기 위해 데이터의 별도 사본을 유지한다
  - 사본들은 일관성(consistency)과 동기화를 유지해야 한다

## Performance Tactics 정리

### ■ 핵심

- Performance 는 수요(demand)를 줄이거나 자원을 더 적절히 관리해서 개선할 수 있다
- 수요를 줄이면 정확도(fidelity)가 떨어지거나 일부 요청을 거절하게 되는 부작용이 있다
- 자원 관리는 scheduling, replication, 또는 단순히 가용 자원을 늘리는 방법으로 이루어진다

### ■ 택틱 목록

- Control Resource Demand: Manage Sampling Rate, Limit Event Response, Prioritize Events, Reduce Overhead, Bound Execution Times, Increase Resource Efficiency
- Manage Resources: Increase Resources, Introduce Concurrency, Maintain Multiple Copies of Computations/Data, Bound Queue Sizes, Schedule Resources



# 3.8

## Safety Tactics

시스템의 오동작이나 장애가 인명·재산·환경에 피해를 주지 않도록 하는 تاک

- Safety 의 개념과 장애 요인
- Safety Scenario
- Unsafe State Avoidance / Detection
- Containment
- Recovery

## Safety(안전성)의 기본 개념

### ■ 정의

- Safety(안전성): 시스템의 잘못된 동작이나 장애로 인해 인명, 재산, 환경 등에 피해가 발생하지 않도록 하는 능력

### ■ 잘못된 동작이나 장애를 일으키는 요인

- 누락(Omission): 이벤트 발생의 실패
- 오발(Commission): 의도하지 않거나 원하지 않은 이벤트의 잘못된 발생
- 타이밍 오류: 필요한 시점보다 이르게 혹은 늦게 동작·이벤트가 발생
- 시스템 값의 오류: 검출 가능한 잘못된 값, 검출하기 어려운 미묘한 잘못된 값
- 순서 오류: 연속적인 이벤트 도착 순서가 잘못된 경우

## Safety General Scenario (1) — 자극원과 자극

| 부분            | 설명                                           | 가능한 값                                                                                                                                         |
|---------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 자극원 (Source)  | 시스템의 잘못된 동작이나 장애를 일으키는 요인의 주체                | 센서 / 소프트웨어 컴포넌트 / 통신 채널 / 장치(예: 시계) 중 하나 이상                                                                                                   |
| 자극 (Stimulus) | 시스템의 잘못된 동작이나 장애를 일으키는 요인 (누락, 오발, 타이밍 오류 등) | 누락: 값이 도착하지 않음, 기능이 수행되지 않음<br>오발: 기능이 잘못 수행됨, 장치가 허위 이벤트를 생성함<br>타이밍 오류: 데이터가 너무 늦거나 일찍 도착, 이벤트가 잘못된 순서로 발생<br>값의 오류: 소프트웨어 컴포넌트가 잘못된 결과를 생성 |

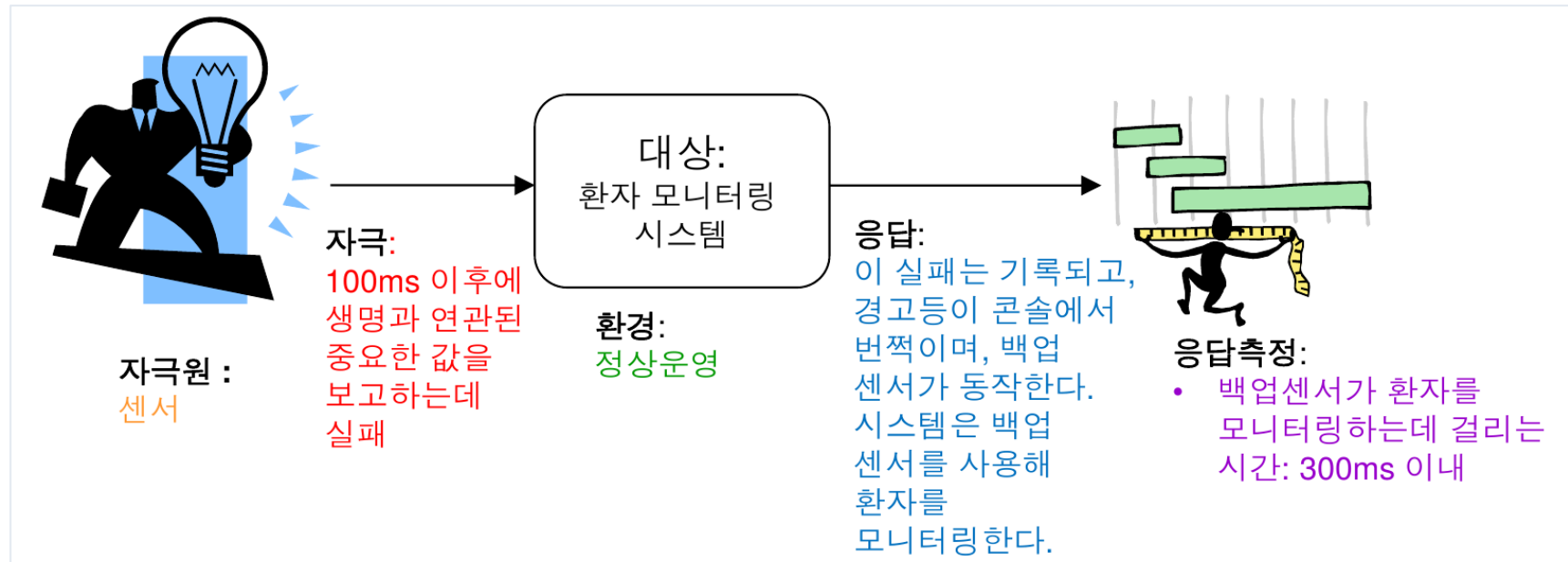
## Safety General Scenario (2) — 대상·환경·응답·응답 측정

| 부분                       | 설명                       | 가능한 값                                                                                                                      |
|--------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 대상 (Artifact)            | 시스템의 어떤 부분이 자극에 영향을 받는가? | 전체 시스템 / 특정 컴포넌트 집합                                                                                                        |
| 환경 (Environment)         | 어떤 상태에서 자극이 발생하는가?       | 정상 운영 / 성능 저하 운영 / 수동 운영 / 복구 모드                                                                                           |
| 응답 (Response)            | 자극에 대한 반응                | 안전하지 않은 상태를 회피 / 복구 / 저하 혹은 안전 모드로 진행 / 섯다운 / 수동 운전·백업 시스템으로 전환 / 적절한 대상(사람 혹은 시스템)에 보고 / 안전하지 않은 상태를 기록                   |
| 응답 측정 (Response Measure) | 응답을 어떻게 측정하는가            | 안전하지 않은 상태 진입을 회피한 횟수나 비율 / 안전하지 않은 상태 중 자동 회복 가능한 상태의 비율 / 복구 가능한 시간 / 저하·안전 모드로 동작하는 시간 / 수동 운영·안전·저하 모드 진입 후 복구까지 걸린 시간 |

## Concrete Safety Scenario 예 — 환자 모니터링

### ■ 시나리오

- "환자 모니터링 시스템의 센서가 100ms 이후에 생명과 연관된 중요한 값을 보고하는 데 실패한다. 이 실패는 기록되고 경고등이 콘솔에서 번쩍이며 백업 센서가 동작한다. 시스템은 300ms 이내에 백업 센서를 사용해 환자를 모니터링한다"

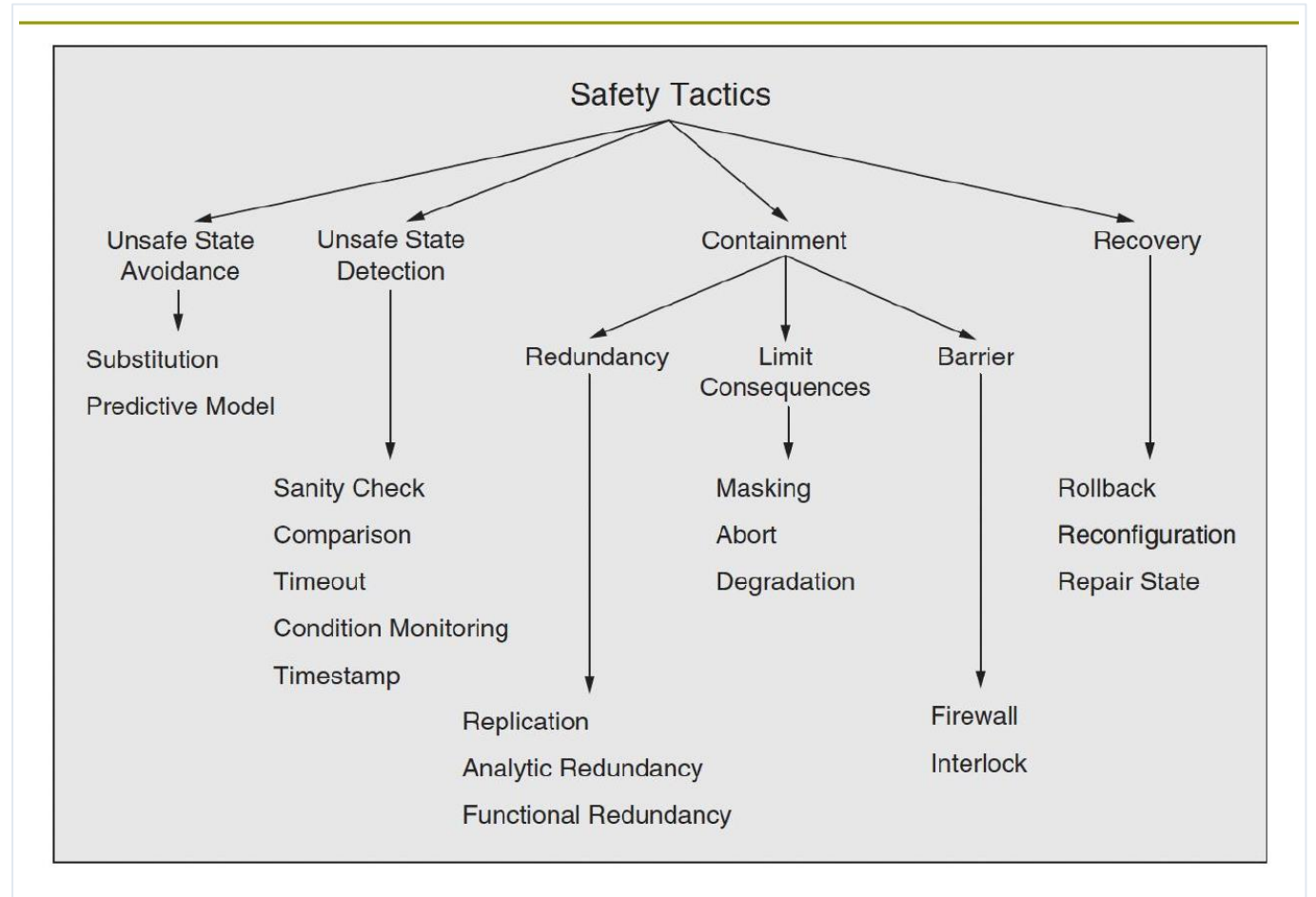


자극원: 센서 / 자극: 값 보고 실패 / 대상: 환자 모니터링 시스템 / 환경: 정상 운영 / 응답: 기록·경고·백업 센서 동작 / 응답 측정: 300ms 이내

## Safety 택틱 계층도

### ■ 4개 범주

- Unsafe State Avoidance: 안전하지 않은 상태에 아예 들어가지 않게 한다
- Unsafe State Detection: 안전하지 않은 상태를 감지한다
- Containment: 이미 진입한 안전하지 않은 상태의 피해를 제한한다
- Recovery: 안전한 상태로 되돌린다



Safety 택틱 계층도

## Unsafe State Avoidance :: Substitution

### ■ 기본 개념

- Substitution(대체) 택틱은 잠재적으로 위험한 소프트웨어 설계 기능을 하드웨어 기반 메커니즘으로 대체해 잠재적 위험 요소를 제거한다

### ■ 사례

- Watchdog Timer, Monitor, Interlock 등의 하드웨어 보호 장치
- 소프트웨어 기반 보호 장치와 달리 자신의 리소스를 기반으로 동작하므로, 리소스를 확보하지 못해 생기는 문제를 회피할 수 있다
- 대개 대체하는 기능이 상대적으로 단순한 경우에 효과가 있다

**핵심** 위험한 소프트웨어 기능을 단순하고 독립적인 하드웨어로 바꾸면 위험 자체가 사라진다

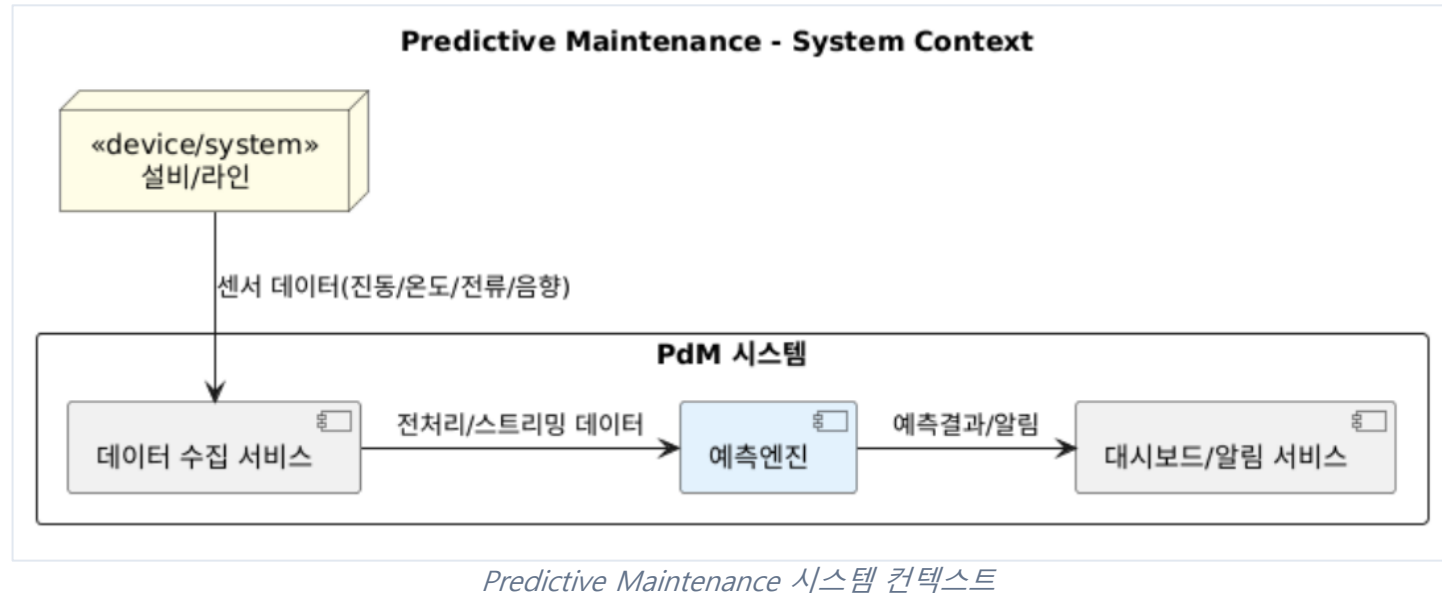
## Unsafe State Avoidance :: Predictive Model

### ■ 기본 개념

- Predictive Model(예측 모델): 다양한 동적 데이터(센서 값, 로그, 성능 메트릭 등)를 분석해 특정 조건이 임박한 미래의 위험 또는 고장 확률을 예측한다
- 주로 Condition Monitoring(상태 모니터링)과 결합한다

### ■ 사례: 예측 정비 (Predictive Maintenance)

- 설비의 센서 데이터(진동/온도/전류/음향)를 수집 → 전처리 → 예측 엔진 → 대시보드/알림 서비스로 전달





## Unsafe State Detection 서버 택틱

### ■ Sanity Checking

- 특정 작업 결과나 component 의 입력·출력의 유효성이나 합리성을 확인한다
- Availability 택틱과 동일

### ■ Comparison

- 동기화되거나 복제된 여러 요소가 만든 출력 값을 비교해 안전하지 못한 상태를 감지한다
- Voting 택틱과 관련

### ■ Timeout

- 타이밍 제약 사항이 만족되지 않음을 감지한다
- Heartbeat, Ping/Echo 같은 Availability 택틱과 관련

### ■ Condition Monitoring

- 위험한 행동으로 이어질 수 있는 시스템 상태를 식별한다
- Availability 택틱과 동일

### ■ Timestamp

- 분산 메시지 전달 시스템에서 잘못된 이벤트 순서를 감지한다
- Availability 택틱과 동일

## Containment(억제) تاک 개요

### ■ Containment 란

- 이미 진입한 안전하지 못한 상태와 관련된 피해를 제한하는 تاک
- 세 가지 서브 تاک: Redundancy, Limit Consequences, Barrier

### ■ Redundancy (다중화)

- 다중화된 component 를 통해 시스템의 동작을 지속 시킨다

### ■ Limit Consequences (영향 제한)

- 안전하지 못한 상태에 들어간 시스템이 초래할 수 있는 나쁜 영향을 제한한다

### ■ Barrier (장벽)

- 문제가 퍼지지 못하도록 예방하고 문제 상황을 완화한다

# Containment :: Redundancy 서브 택틱

|       | Replication                                            | Functional Redundancy                                | Analytic Redundancy                                       |
|-------|--------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------|
| 기본 개념 | 동일 component 를 복수 개 배치                                 | 동일한 기능을 수행하지만 서로 다른 설계, 기술, 혹은 알고리즘을 사용해 구현          | 물리적인 하드웨어 추가 없이 수학적 모델링을 사용해 중복성을 확보                      |
| 사례    | 3개의 동일 센서 중 하나가 이상 값을 출력하면 Voting 방식으로 고장을 감지하고 오류를 무시 | 서로 다른 프로그래밍 언어로 작성된 2개의 소프트웨어 모듈이 동일한 계산을 수행해 결과를 비교 | 시스템의 수학적 모델로 센서 측정값을 예측하고, 예측값과 실제 측정값의 차이가 크면 센서 고장으로 판단 |
| 장점    | 구현이 단순                                                 | 설계 오류 방지에 효과적                                        | 물리적 비용 절감                                                 |
| 단점    | 설계 오류에 취약                                              | 개발 비용이 높음                                            | 모델의 정확도에 의존                                               |

## Containment :: Limit Consequences 서브 택틱

### ■ Abort (취소)

- 안전하지 못하다고 판단된 작업은 피해를 최소화하기 위해 취소한다
- 사례: 로켓 발사 중 심각한 엔진 고장 시 발사 중단(Launch Abort) 후 승무원 탈출 시스템 작동
- 사례: 공장 생산 라인에서 센서 오작동 감지 시 전체 라인 비상 정지

### ■ Degradation (성능 저하)

- 일부 기능을 버리거나 대체함으로써 가장 중요한 시스템 기능을 유지한다
- 사례: 자율 주행 차량에서 센서 하나가 고장 나면 자율 주행 레벨을 낮추고 운전자에게 제어를 요청

### ■ Masking (마스킹)

- 고장이 발생해도 시스템이 자동으로 보정·무시하여, 고장이 없었던 것처럼 정상 기능과 성능을 유지한다
- 사례: 세 개의 동일 센서 중 하나에 오류가 생기면 투표(voting) 메커니즘으로 오류를 마스킹

## Containment :: Barrier 서브 택틱

### ■ Firewall (방화벽)

- 시스템 내 한 영역에서 발생한 오류(Fault)나 고장(Failure)이 다른 안전한 영역으로 확산되는 것을 막아 피해를 격리한다
- 사례: 서브시스템 A 가 오류를 출력해도, 그 출력이 서브시스템 B 에 전달되기 전에 데이터 유효성(범위 검증, timestamp 검증 등)을 검사해 잘못된 데이터를 차단

### ■ Interlock (인터락)

- 특정 조건이 충족되지 않으면 위험한 동작이나 명령의 실행을 물리적 또는 논리적으로 금지하는 안전 메커니즘
- 사례: 모터에 정방향 명령이 활성화된 상태에서는 역방향 명령이 들어와도 실행되지 않도록 제어 시스템에서 차단

## Recovery 서브 택틱 — Rollback / Reconfiguration

### ■ Rollback (롤백)

- 시스템을 최근에 알려진 안전한 상태로 되돌리는 방법
- 1. 오류가 감지되면 안전 모드로 설정
- 2. 최근에 알려진 안전한 상태 스냅샷을 기반으로 복원
- 3. 롤백 후 제어 상태/설정/모델/cache 간의 일관성 검증
- 4. 재가동

### ■ Reconfiguration (재설정)

- 시스템의 일부가 고장 나면 해당 부분을 즉시 분리하고, 남은 리소스에 재연결해 기능을 유지한다
- Failover(장애 전환): 이중화 구조에서 장애 component 를 대기 중인 백업 component 로 전환
- Graceful degradation(점진적 성능 저하): 치명적이지 않은 기능을 비활성화해 남은 자원을 핵심 기능에 집중

## Recovery 서브 택틱 — Repair State

### ■ Repair State (상태 수리)

- 시스템에서 오류 또는 고장이 발생한 후, 해당 시스템을 정상 작동 상태로 되돌리기 위한 일련의 활동

| 활동        | 설명                                                                         |
|-----------|----------------------------------------------------------------------------|
| 물리적 수리/교체 | 고장 난 디스크, 전원 장치, 모듈 등 하드웨어 구성 요소를 교체                                       |
| 데이터 재구성   | RAID 시스템이나 분산 데이터베이스에서 고장 난 디스크로 손실된 데이터를 다른 복제본을 기반으로 재구성(Reconstruction) |
| 소프트웨어 패치  | 소프트웨어 버그가 고장의 원인이었을 경우, 해당 버그를 해결하는 패치를 적용                                 |
| 시스템 재동기화  | 고장/복구 과정에서 불일치해진 주 시스템과 예비 시스템(Active/Standby)의 상태(State)를 다시 일관성 있게 맞춤    |

# 3.9

## Security Tactics

인가되지 않은 접근으로부터 데이터와 정보를 보호하면서, 인가된 사용자에게는 접근을 제공하는 택틱

- Security 의 세 가지 특성 (CIA)
- Security Scenario
- Detect Attacks
- Resist Attacks
- React to Attacks
- Recover from Attacks



## Security(보안)의 정의와 세 가지 특성

### ■ Security 란

- 인가되지 않은(unauthorized) 접근으로부터 데이터와 정보를 보호하면서, 동시에 인가된 사람과 시스템에는 접근을 제공하는 시스템 능력의 척도

### ■ 세 가지 특성

- Confidentiality(기밀성): 데이터나 서비스가 인가되지 않은 접근으로부터 보호되는 성질
- Integrity(무결성): 데이터나 서비스가 인가되지 않은 조작(manipulation)을 받지 않는 성질
- Availability(가용성): 시스템이 정당한(legitimate) 사용을 위해 이용 가능한 성질

**핵심** 보안은 '막는 것'만이 아니라 '정당한 사용자는 쓸 수 있게 하는 것'까지 포함한다

## Security General Scenario (1) — Source·Stimulus·Artifact·Environment

| 부분                 | 가능한 값                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------------|
| <b>Source</b>      | 개인 또는 시스템으로서<br>- 정확히 식별됨 / 잘못 식별됨 / 신원 불명<br>- 내부 / 외부, 인가됨 / 인가되지 않음<br>- 제한된 자원 보유 / 방대한 자원 보유 |
| <b>Stimulus</b>    | 데이터 열람, 데이터 변경·삭제, 시스템 서비스 접근, 시스템 서비스의 가용성 저하를 시도한다                                              |
| <b>Artifact</b>    | 시스템 서비스 / 시스템 내의 데이터                                                                              |
| <b>Environment</b> | 온라인 또는 오프라인 / 연결됨 또는 연결 끊김 / 방화벽 보호 또는 네트워크에 개방 / 완전·부분 운영 또는 비운영                                 |

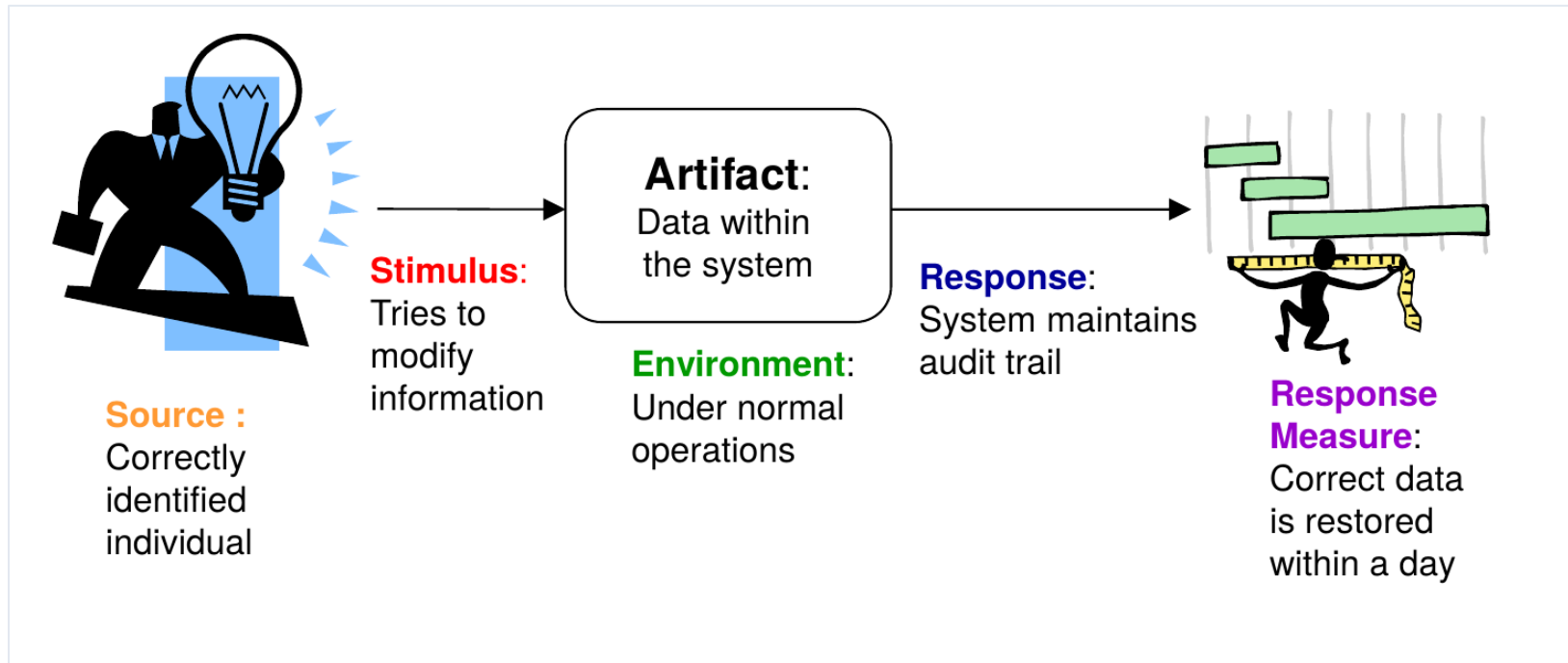
# Security General Scenario (2) — Response·Response Measure

| 부분               | 가능한 값                                                                                                                                                                                                                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Response         | transaction 이 다음과 같이 수행된다<br>- 데이터나 서비스가 인가되지 않은 접근으로부터 보호된다<br>- 데이터나 서비스가 인가 없이 조작되지 않는다<br>- transaction 당사자가 확실하게 식별된다<br>- 당사자가 자신의 관여를 부인할 수 없다(non-repudiation)<br>- 데이터, 자원, 시스템 서비스가 정당한 사용을 위해 이용 가능하다<br>시스템이 내부 활동을 추적한다<br>- 접근이나 수정을 기록 / 데이터·자원·서비스 접근 시도를 기록<br>- 공격이 진행 중으로 보이면 적절한 대상(사람 또는 시스템)에 알림 |
| Response Measure | 특정 component 나 데이터 값이 손상됐을 때 시스템이 얼마나 피해를 입는가 / 공격을 감지하기까지 걸린 시간 / 막아낸 공격 수 / 성공한 공격에서 복구하는 데 걸린 시간 / 특정 공격에 취약했던 데이터 양                                                                                                                                                                                                |

## Concrete Security Scenario 예

### ■ 시나리오

- "정확히 식별된 개인이 외부 사이트에서 시스템 데이터를 수정하려 한다. 시스템은 audit trail 을 유지하고, 올바른 데이터가 하루 안에 복원된다"

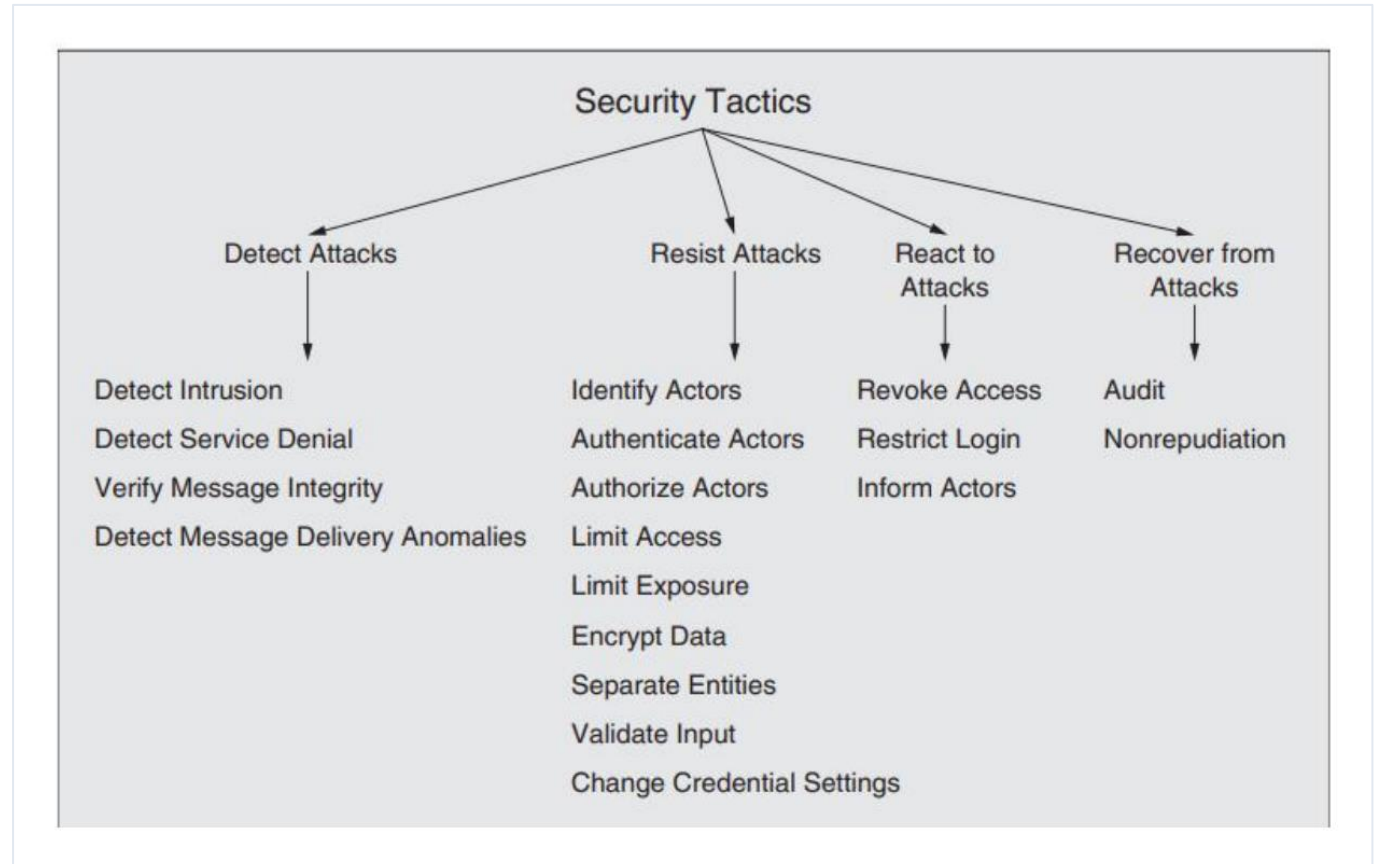


Source: 정확히 식별된 개인 / Stimulus: 정보 수정 시도 / Artifact: 시스템 내 데이터 / Environment: 정상 운영 / Response: audit trail 유지 / Response Measure: 하루 안에 복원

## Security 태틱 계층도

### ■ 4개 범주

- Detect Attacks: 공격을 감지한다
- Resist Attacks: 공격에 저항한다 (가장 많은 태틱)
- React to Attacks: 공격에 대응한다
- Recover from Attacks: 공격에서 복구한다 (Audit, Nonrepudiation)



Security 태틱 계층도

## Detect Attacks **택틱**

### ■ Detect intrusion (침입 감지)

- 시스템 내 네트워크 트래픽이나 서비스 요청 패턴을 데이터베이스에 저장된 악성 행위의 signature·알려진 패턴과 비교한다

### ■ Detect service denial (서비스 거부 감지)

- 시스템으로 들어오는 네트워크 트래픽의 패턴이나 signature 를 알려진 DoS 공격의 과거 프로파일과 비교한다

### ■ Verify message integrity (메시지 무결성 검증)

- checksum 이나 hash 값 같은 기법으로 메시지나 자원 파일의 무결성을 검증한다

### ■ Detect message delay (메시지 지연 감지)

- 메시지 전달에 걸리는 시간을 검사해 잠재적인 man-in-the-middle 공격을 감지한다

## Resist Attacks — Identify / Authenticate Actors

### ■ 공격에 저항하는 잘 알려진 수단들

- Identify actors, Authenticate actors, Authorize actors
- Limit access, Limit exposure, Encrypt data, Separate entities
- Validate input, Change credential settings

### ■ Identify actors (행위자 식별)

- 시스템에 들어오는 모든 외부 입력의 출처를 식별한다
- 사용자는 보통 user ID 로, 다른 시스템은 access code, IP 주소, protocol, port 등으로 식별한다

### ■ Authenticate actors (행위자 인증)

- 행위자(사용자 또는 원격 컴퓨터)가 실제로 자신이 주장하는 그 대상인지 확인한다
- password, one-time password, 디지털 인증서, 생체 인식이 인증 수단이 된다

## Resist Attacks — Authorize Actors

### ■ Authorize actors (행위자 인가)

- 인증된 행위자가 데이터나 서비스에 접근하고 수정할 권한을 가지고 있는지 확인한다
- 보통 시스템 안에 접근 제어(access control) 메커니즘을 두어 구현한다
- 사용자 → Access Control Mechanism → 조직의 자원(Office Files, Test, Build, Code, Work) 순으로 접근이 통제된다



Access Control Mechanism 을 통한 조직 자원 접근 통제



## Resist Attacks — Limit Access / Limit Exposure / Separate Entities

### ■ Limit access (접근 제한)

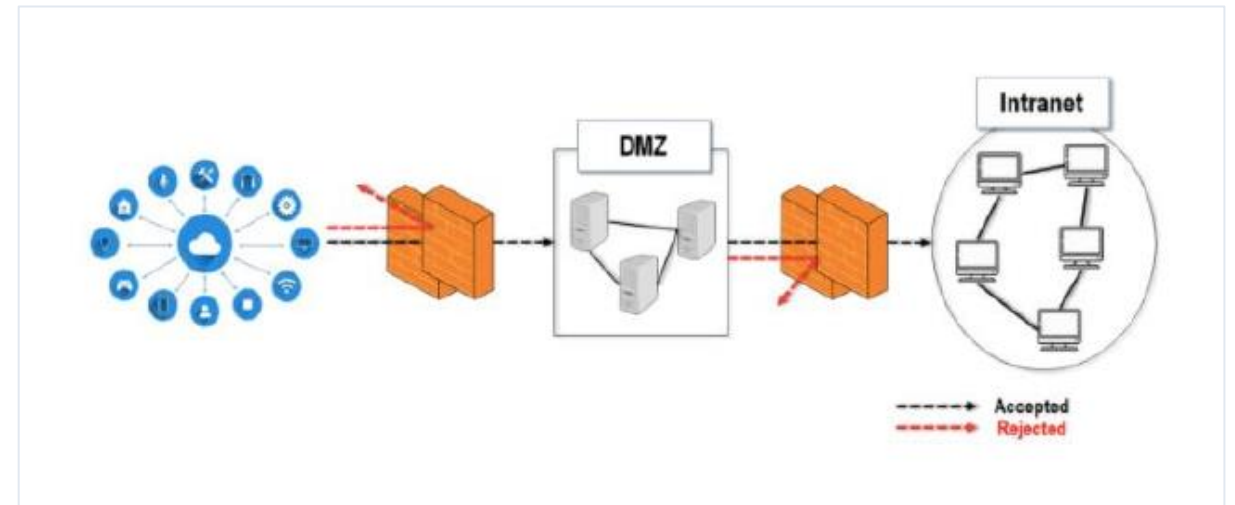
- 자원에 대한 접근을 통제한다
- 예: DMZ(DeMilitarized Zone) — 외부 네트워크와 내부망(Intranet) 사이에 방화벽으로 격리된 완충 구역

### ■ Limit exposure (노출 제한)

- 공격 성공 확률을 줄이거나 잠재적 피해 규모를 제한한다
- 예: 서버를 지리적으로 분산된 여러 데이터 센터에 나누어 배치하기로 결정

### ■ Separate entities (개체 분리)

- 시스템 내의 서로 다른 개체를 서로 다른 서버에 물리적으로 분리한다
- 예: 민감한 데이터를 민감하지 않은 데이터와 분리하면 공격 가능성이 줄어든다



DMZ: 외부 → 방화벽 → DMZ → 방화벽 → Intranet

## Resist Attacks — Encrypt / Validate Input / Credentials

### ■ Encrypt data (데이터 암호화)

- 영구 저장 데이터에 대해 인가(authorization)만으로는 부족한 추가 보호를 제공한다
- 통신 링크에는 인가 통제가 없을 수 있다. 이 경우 공개된 통신 링크로 데이터를 보낼 때 암호화가 유일한 보호 수단이다

### ■ Validate input (입력 검증)

- 입력을 정제(cleaning)하고 검사하는 것은 공격 저항의 중요한 초기 방어선이다
- 예: SQL Injection 방지

### ■ Change credential settings (자격 증명 설정 변경)

- 사용자에게 기본 설정(default settings)을 강제로 바꾸게 한다
- 일반적으로 공개되어 있는 기본 설정을 통해 공격자가 시스템에 접근하는 것을 막는다

## React to Attacks / Recover from Attacks

### ■ React to Attacks (공격 대응)

- Revoke access(접근 회수): 공격이 진행 중이면 평소 정당한 사용자와 용도에 대해서도 민감한 자원 접근을 크게 제한한다
- Lock computer(컴퓨터 잠금): 특정 컴퓨터에서 계정 접근 시도가 반복 실패하면 그 컴퓨터의 접근을 제한한다
- Inform actors(행위자 통보): 시스템이 공격을 감지하면 관련 행위자들에게 알려야 한다

### ■ Recover from Attacks (공격 복구)

- Availability 태틱들을 그대로 가져와 쓸 수 있다
- Maintain audit trail(감사 추적 유지): audit trail(audit log)은 특정 작업·절차·이벤트에 영향을 준 활동의 순서를 시간순으로 기록한 보안 관련 문서 증거
- audit trail 을 분석해 앞으로 더 나은 방어를 만들 수 있다

## Security Tactics 정리

### ■ 목표

- 공격을 감지(detect)하고, 저항(resist)하고, 대응(react)하고, 예방(prevent)한다
- 공격 복구(recover)는 Availability 와 같은 택틱을 많이 사용한다

### ■ Detect attacks

- Detect intrusion, Detect service denial, Verify message integrity, Detect message delay

### ■ Resist attacks

- Identify / Authenticate / Authorize actors
- Limit access, Limit exposure, Separate entities
- Encrypt data, Validate input, Change default settings

### ■ React to attacks

- Revoke access, Lock computer, Inform actors